UniZ: University Administration Platform

Architectural Reference and User Manual

SreeCharan Desu[1], BhanuPrakash[1], Seetharam[1], and Anand[1]

[1]UniZ Engineering Team

`Infra and DevOps | Developer | Cyber Security | AI`

March 31, 2026

Abstract

This document serves as the primary technical specification and user manual for UniZ, a comprehensive university administration platform tailored for RGUKT. It details the role-based access control (RBAC), multi-layered outpass approval workflows, and the distributed microservices architecture powering the system. The report is optimized for clarity, mapping core logic to functional documentation.

Contents

1 User Manual & System Documentation **3**

2 Technical Implementation **79**

2.1 Component: **Uniz Academics** . . . 79
2.2 Component: **Uniz Auth** . . . 80
2.3 Component: **Uniz Cron** . . . 81
2.4 Component: **Uniz Files** . . . 82
2.5 Component: **Uniz Notifications** . . . 83
2.6 Component: **Uniz Outpass** . . . 85
2.7 Component: **Uniz User** . . . 86
2.8 Component: **Uniz Academics** . . . 88
2.9 Component: **Uniz Mail** . . . 90
2.10 Component: **Uniz User** . . . 92
2.11 Component: **Uniz Academics** . . . 94
2.12 Component: **Uniz Auth** . . . 98
2.13 Component: **Uniz Files** . . . 100
2.14 Component: **Uniz Mail** . . . 102
2.15 Component: **Uniz Outpass** . . . 103
2.16 Component: **Uniz User** . . . 107
2.17 Component: **Uniz Academics** . . . 112
2.18 Component: **Uniz Auth** . . . 114
2.19 Component: **Uniz Cron** . . . 115
2.20 Component: **Uniz Files** . . . 118
2.21 Component: **Uniz Gateway** . . . 121
2.22 Component: **Uniz Landing Backend** . . . 123
2.23 Component: **Uniz Landing** . . . 124
2.24 Component: **Uniz Mail** . . . 126

2.25 Component: **Uniz Notifications** 129
2.26 Component: **Uniz Outpass** 132
2.27 Component: **Uniz Portal** 135

1 User Manual & System Documentation

This section provides an overview of the platform from a user and administrator perspective.

UniZ is a university administration platform built specifically for RGUKT students, faculty, and staff. Whether you need to check your grades, request an outpass, approve a student's leave request, or publish a campus announcement — UniZ brings everything into one place, accessible from any device.

The platform is organized around the roles that make up campus life. Students get a personal dashboard for academics and mobility requests. Faculty can upload grades and attendance. Wardens, caretakers, security staff, and administrators each have dedicated workflows that keep campus operations running smoothly. Every action flows through a structured, role-based system so the right people see the right information.

UniZ is a Progressive Web App (PWA), which means you can install it directly on your phone, tablet, or desktop from your browser — no app store required. Once installed, it works like a native app with a fast, focused experience tailored to your role.

Quick Start Get the app installed and log in to your portal in a few minutes.

Roles & Access See what each role can do and find the features built for you.

Student Login Sign in as a student and access your academic dashboard.

API Authentication Explore the authentication API for the UniZ platform.

This guide walks you through accessing UniZ, installing it on your device, and logging in for the first time. The whole process takes under five minutes.

Open the UniZ portal Navigate to the UniZ portal in any modern browser on your phone, tablet, or computer:

```
https://uniz.rguktong.in
```

You will see the UniZ home page with live campus announcements and a getting-started timeline.

Install as a PWA (recommended) UniZ is a Progressive Web App. Installing it gives you a faster, app-like experience without going through an app store.

Android 1. Open `https://uniz.rguktong.in` in Chrome. 2. Tap the **Install** button shown in the browser banner, or tap the three-dot menu and select **Add to Home Screen**. 3. Confirm the installation prompt.

UniZ now appears on your home screen and opens in full-screen mode.

iOS (iPhone / iPad) 1. Open `https://uniz.rguktong.in` in Safari. 2. Tap the **Share** icon at the bottom of the screen. 3. Scroll down and tap **Add to Home Screen**. 4. Tap **Add** to confirm.

Note: *iOS requires Safari for PWA installation. Chrome and other browsers on iOS do not support "Add to Home Screen" for PWAs.*

Desktop (Windows / macOS / Linux) 1. Open `https://uniz.rguktong.in` in Chrome or Edge. 2. Look for the install icon in the browser's address bar (a screen with a download arrow), or open the browser menu and select **Install UniZ**. 3. Click **Install** to confirm.

UniZ opens as a standalone desktop app.

Log in UniZ uses a single login endpoint for all roles. Where you log in depends on whether you are a student or a staff member.

Students 1. From the portal home page, click **Get started** or navigate to the student sign-in page. 2. Enter your university-issued student ID (for example, `O210008`) and your password. 3. Click **Sign In**.

What you see after logging in: - Your profile with name, branch, year, and campus status - Your grades and subject-wise attendance - Outpass and outing request forms - Your request history

```
{
          "username": "O210008",
          "password": "your_password"
        }
```

Pro-Tip: *Your username is your RGUKT student roll number. If you have never logged in before, contact your administrator to get your initial credentials.*

Faculty & Staff 1. From the portal home page, type `admin` on your keyboard to reveal the admin sign-in page, or navigate directly to the admin portal. 2. Enter your assigned username (for example, `dean_cse`, `warden_male`, `security_admin`) and your password. 3. Click **Sign In**.

What you see after logging in: - A dashboard tailored to your role - Pending outpass or outing requests (wardens, caretakers, SWO, dean, director) - Grade and attendance upload tools (faculty) - Gate check-in/check-out controls (security) - Banner and announcement management (webmaster)

```
{
          "username": "director",
          "password": "your_password"
        }
```

Note: *All admin and faculty roles use the same login endpoint. The platform automatically loads the correct dashboard based on the role assigned to your account.*

Take your first action Once you are logged in, here is what to do first for each role:

| Role | First action | |——|————-| | Student | Check your grades and attendance under the **Academics** section | | Caretaker / Warden | Open the **Requests** dashboard to see pending outpass approvals | | Faculty | Go to **Grades** and download the upload template for your batch | | Security | Open the **Gate** dashboard to see currently approved outpasses | | Dean / Director | Use **Search** to look up any student profile | | Webmaster | Go to **Banners** and create or publish a campus announcement |

Forgot your password?

If you cannot log in, you can reset your password using your registered email address.

Request an OTP On the sign-in page, click **Forgot password** and enter your username. UniZ sends a one-time password to your registered email.

Verify the OTP Enter the six-digit OTP from your email on the verification screen.

Set a new password Enter and confirm your new password. You are then redirected to the sign-in page.

Warning: *OTPs expire after a short window. If you do not receive an email within a few minutes, check your spam folder or contact your system administrator.*

Every user in UniZ is assigned a role when their account is created. Your role determines which features you can access, which dashboards you see after logging in, and what actions you are authorized to take. There is no manual role switching — your role is fixed to your account.

Role overview

| Role | Who it's for | Primary function | |——|————-|————————| | `student` | Enrolled RGUKT students | Academics, outpass requests, profile management | | `caretaker_male` / `caretaker_female` | Hostel caretakers | First-level outpass approval | | `warden_male` / `warden_female` | Hostel wardens | Second-level outpass approval | | `swo` | Student Welfare Officer | Approval chain for student requests | | `dean` | Dean of the institution | Final outpass authority, batch grade review | | `director` | Director of the institution | Super authority, student status override | | `security` | Security staff at gates | Gate check-in/check-out, approved outpass verification | | `faculty` | Teaching staff | Grade and attendance uploads, student record access | | `webmaster` | Platform administrator | Banner management, announcements, system health |

—

Detailed role capabilities

Student The default role for all enrolled students.

You can: - Log in and view your profile (name, branch, year, campus status) - View your grades and subject-wise attendance - Submit outpass requests (multi-day leave from campus) - Submit outing requests (short-duration exits) - Track the status and history of your requests - Update personal contact details

You cannot: - View other students' data - Approve any request - Access admin or faculty dashboards

Caretaker Assigned to hostel caretakers (`caretaker_male` or `caretaker_female`).

You can: - View all pending outpass requests for your gender group - Approve a request, moving it forward to the warden level - Reject a request with a comment - Add a comment when approving (for example, "Parent verified via call")

You cannot: - Give final approval for outpasses — caretaker approval is the first step only - View or act on requests for the other gender group

Warden Assigned to hostel wardens (`warden_male` or `warden_female`).

You can: - View pending outpass requests at the warden level - Approve a request, advancing it to the SWO or final authority - Reject a request with a comment - Check academic schedule or disciplinary context before approving

You cannot: - See requests that have not yet been approved by the caretaker

SWO Assigned to the Student Welfare Officer.

You can: - Review outpass requests escalated from the warden level - Approve or reject requests with comments - Act as an intermediary step in the full approval chain

The SWO sits between the warden and the final authority (dean/director) in the approval workflow.

Dean Assigned to the Dean of the institution.

You can: - Give **final approval** on outpass requests — your approval immediately grants the outpass - Reject any pending request at any stage - Search and view any student profile - Review batch grades across branches and years - Filter for failed students to identify academic risks - Override student campus status if needed

Dean approval is binding — no further approval is required after a dean approves a request.

Director Assigned to the Director of the institution. The highest authority on the platform.

You can: - Give **final approval** on outpass and outing requests - Search and view **any** student profile (super search) - Manually override a student's campus status (mark in/out) - View batch grades and all academic data - Trigger result publication emails to students

The director has full override capability across the platform with no access restrictions.

Security Assigned to security staff at campus gates.

You can: - View all approved outpasses and outings to verify students at the gate - Mark a student as **checked out** when they leave campus - Mark a student as **checked in** when they return - View a live list of all students currently outside campus - Search for a specific student by ID to verify their status

You cannot: - Approve or reject outpass requests (you only act after final approval is granted) - Access academic records

Faculty Assigned to teaching staff.

You can: - Download a pre-filled Excel grade template for any batch (by branch, year, semester) - Upload completed grade sheets — processed in bulk with error reporting - Upload attendance records for your subjects - Monitor the status of a grade or attendance upload job - View student records for your assigned subjects

You cannot: - Approve outpass requests - Access student mobility or outpass data

Webmaster Assigned to the platform administrator responsible for content and system health.

You can: - Create campus announcements and banners (title, body text, image) - Publish or unpublish banners — published banners appear to all logged-in users - Manage technical configurations - Monitor real-time system health across all UniZ services

The webmaster role is the only role that can control what announcements the entire user base sees.

—

Outpass approval chain

When a student submits an outpass request, it moves through a structured approval workflow. Each level must approve before it advances to the next.

```
Student submits     Caretaker     Warden     SWO     Dean / Director (final approval)
```

At any stage, the approver can reject the request with a comment. If rejected, the request does not advance further. A dean or director can grant final approval at any point in the chain — their approval immediately marks the outpass as approved regardless of which stage it is currently at.

Note: *Outing requests (short-duration exits, typically a few hours) follow a shorter approval path and are handled separately from multi-day outpass requests.*

Welcome to UniZ, the enterprise-grade university administration platform built for RGUKT. UniZ unifies student academics, outpass management, attendance tracking, and campus operations into a single intelligent ecosystem accessible from any device.

Quick Start Get up and running with UniZ in minutes — install the app and log in.

Roles & Access Understand the different roles and what each user can do on the platform.

API Reference Explore the full REST API with request and response examples.

Student Guide Step-by-step guides for students — grades, outpass, attendance, and more.

What you can do with UniZ

UniZ is designed for every role in the university ecosystem — students, faculty, wardens, security, and administrators all have tailored features built for their workflows.

View Grades & Attendance Students access real-time academic records including grades, GPA, and subject-wise attendance.

Outpass & Outing Requests Submit and track outpass or outing requests through a multi-level approval workflow.

Admin Approvals Caretakers, wardens, and directors approve or reject requests from a unified dashboard.

Security Gate Portal Security staff verify and manage student campus entry and exit in real time.

Get started in three steps

Install the app Open the UniZ portal in your browser and install it as a PWA on your device — works on iOS, Android, and desktop.

Log in with your credentials Use your university-issued username and password to log in. Students use the Student Portal; faculty and staff use the Admin Portal.

Start using your features Students can immediately view grades and request outpasses. Admins and faculty have role-specific dashboards ready on first login.

Note: *UniZ is built by RGUKT students, for RGUKT students. If you run into issues, use the Help & Support section inside the portal or contact your system administrator.*

Use this guide to submit attendance records for your subjects, correct individual entries, and download attendance reports. Once you upload attendance, students can immediately see their attendance percentage per subject in their portal.

Note: *You log in to UniZ with the same endpoint used by admins. Your `role` in the login response determines your access level.*

Upload attendance via Excel template

The recommended workflow for submitting a batch's attendance is to download the pre-filled Excel template, fill in classes attended and total classes, then upload the completed file.

Download the attendance template Send a `GET` request to `/academics/attendance/template` with query parameters for the batch you are recording attendance for. The template comes pre-filled with all student IDs so you do not need to enter them manually.

```
GET https://api.uniz.rguktong.in/api/v1/academics/attendance/template?branch=CSE&year=E3&
    semester=SEM-1&subjectCode=E3-SEM-1-CSE-1
    Authorization: Bearer <your-token>
```

| Parameter | Example | Description | | ————- | ————- | ————————- | | `branch` | `CSE` | Department code | | `year` | `E3` | Batch year (E1–E4, P1–P2) | | `semester` | `SEM-1` | Semester identifier | | `subjectCode` | `E3-SEM-1-CSE-1` | Subject code as registered in UniZ |

The response is an `.xlsx` file download. Save it before filling it in.

Fill in attendance for each student Open the downloaded template. Each row corresponds to one student. Fill in the following columns for each student:

| Column | Description | | ————- | ————————————— | | `attendedClasses` | Number of classes the student physically attended | | `totalClasses` | Total number of classes conducted for the subject |

Example rows:

| Student ID | Attended Classes | Total Classes | | ———- | ————- | ———- | | O210008 | 45 | 50 | | O210045 | 38 | 50 | | O210139 | 50 | 50 |

Do not modify the student ID column or add or remove rows. Both values must be whole numbers. `attendedClasses` cannot exceed `totalClasses`.

Warning: *The file size limit for uploads is **50 MB**. Standard attendance sheets are well within this limit, but avoid embedding images or charts in the spreadsheet.*

Pro-Tip: *If a student was not enrolled in the subject, leave both cells blank rather than entering zeros. Entering `0/50` will register as 0% attendance and will be visible to the student.*

Upload the filled template Post the completed file to `/academics/attendance/upload` as a multipart form.

```
POST https://api.uniz.rguktong.in/api/v1/academics/attendance/upload
    Authorization: Bearer <your-token>
    Content-Type: multipart/form-data

    file=<your-filled-attendance-template.xlsx>
```

Success response:

```
{
      "success": true,
      "processed": 62,
      "successCount": 61,
      "failCount": 1,
      "errors": [
        "Row 22: attendedClasses (55) exceeds totalClasses (50) for student O210077"
      ]
    }
```

Upload is processed as a background batch job. The `successCount` and `failCount` fields give you an immediate summary. Review any entries listed in `errors`, correct them in the sheet, and re-upload only the corrected rows if needed — or fix the values individually using the manual add endpoint below.

Note: *Uploading a file does not overwrite existing attendance for students not included in the new file. Only rows present in the uploaded sheet are processed.*

Monitor upload progress For large batches, poll the progress endpoint until the job completes.

```
GET https://api.uniz.rguktong.in/api/v1/academics/attendance/upload/progress
    Authorization: Bearer <your-token>
```

Response:

```
{
      "success": true,
      "progress": {
        "status": "processing",
        "percent": 68,
        "processed": 210,
        "total": 310
      }
    }
```

Poll every few seconds until `status` is `"completed"` and `percent` reaches `100`.

—

Add attendance manually for individual students

To record or correct attendance for a single student without re-uploading a full sheet, use the add attendance endpoint directly.

```
POST https://api.uniz.rguktong.in/api/v1/academics/attendance/add
Authorization: Bearer <your-token>
Content-Type: application/json
```

Request body:

```
{
  "studentId": "O210008",
  "subjectCode": "E3-SEM-1-CSE-1",
  "semesterId": "SEM-1",
  "attendedClasses": 45,
  "totalClasses": 50
}
```

Success response:

```
{
  "success": true,
  "message": "Attendance added successfully"
}
```

If a record already exists for that student and subject, this call updates it in place.

—

Download an attendance report

To download a consolidated attendance report for a semester, use the download endpoint with the semester ID.

```
GET https://api.uniz.rguktong.in/api/v1/academics/attendance/download/:semesterId
Authorization: Bearer <your-token>
```

Example:

```
GET https://api.uniz.rguktong.in/api/v1/academics/attendance/download/SEM-1
Authorization: Bearer <your-token>
```

The response is a downloadable report file for that semester. Use this to identify students who fall below the required attendance threshold before finalising records.

—

What students see

After you upload attendance, students can view their attendance percentage for each subject in their portal immediately — no additional publish step is required for attendance data.

Each student sees:

- Subject name and code - Number of classes attended - Total classes conducted - Calculated attendance percentage

Example of what the student portal displays (derived from uploaded data):

```
{
  "success": true,
  "attendance": [
    {
      "id": "att-123",
      "attendedClasses": 45,
      "totalClasses": 50,
      "subject": {
        "code": "CS3101",
        "name": "Operating Systems"
      }
    },
    {
      "id": "att-124",
      "attendedClasses": 38,
      "totalClasses": 50,
      "subject": {
        "code": "CS3102",
        "name": "Compiler Design"
      }
    }
  ]
}
```

The attendance percentage displayed to the student is calculated as `(attendedClasses / totalClasses) × 100`. Ensure your `totalClasses` value is accurate before uploading, as corrections after students have viewed their records can cause confusion.

—

Publish attendance summary via email

Warning: *The email publish endpoint (`POST /academics/attendance/publish-email`) is* ***deprecated****. Prefer downloading the attendance report directly using `GET /academics/attendance/download/:semesterId` and sharing it through your department's standard communication channels.*

The endpoint is retained for audit compliance and may be removed in a future release.

If you need to trigger an email summary for audit or legacy purposes:

```
POST https://api.uniz.rguktong.in/api/v1/academics/attendance/publish-email
Authorization: Bearer <your-token>
Content-Type: application/json
```

Request body:

```
{
  "semesterId": "SEM-1"
}
```

Response:

```
{
  "success": true,
  "message": "Attendance summary email job queued"
}
```

—

Attendance format reference

Required columns Each row in the template requires:

- **Student ID** — pre-filled, do not edit - **attendedClasses** — whole number - **totalClasses** — whole number, must be $\{ }ge$ attendedClasses

Accepted file formats The upload endpoint accepts:

- `.xlsx` (recommended) - `.xls` - `.csv`

Maximum file size: **50 MB**

Use this guide to upload grades for your assigned subjects, correct individual records, and download grade reports. Grade results are visible to students only after the Director triggers a publish.

Note: *You log in to UniZ with the same endpoint used by admins. Your `role` in the login response determines your access level.*

Upload grades via Excel template

The recommended workflow for submitting an entire batch's grades is to download the pre-filled Excel template, fill in grades, and upload the completed file.

Download the Excel template Send a `GET` request to `/academics/grades/template` with query parameters for the batch you are grading. The template comes pre-filled with all student IDs for that batch, so you do not need to enter them manually.

```
GET https://api.uniz.rguktong.in/api/v1/academics/grades/template?branch=CSE&year=E3&semester=SEM-1&subjectCode=E3-SEM-1-CSE-1
    Authorization: Bearer <your-token>
```

| Parameter | Example | Description | | ————- | ————- | ———————————— | | `branch` | `CSE` | Department code | | `year` | `E3` | Batch year (E1–E4, P1–P2) | | `semester` | `SEM-1` | Semester identifier | | `subjectCode` | `E3-SEM-1-CSE-1` | Subject code as registered in UniZ |

The response is an `.xlsx` file download. Save it locally before filling it in.

Pro-Tip: *In the UniZ portal, you can download the template directly from the **Add Grades** page without writing the request manually.*

Fill in grades for each student Open the downloaded template. Each row corresponds to one student. Fill in the **Grade** column using the accepted grade values:

| Value | Meaning | | —— | ————— | | `EX` | Excellent | | `A` | Grade A | | `B` | Grade B | | `C` | Grade C | | `D` | Grade D | | `F` | Fail | | `9.5` | Numeric points |

Do not modify the student ID column or add or remove rows. Leave the grade cell blank only if a student was absent for evaluation — do not use `N/A` or similar placeholders.

Warning: *The file size limit for uploads is **50 MB**. Standard grade sheets are well within this limit, but avoid embedding images or charts in the spreadsheet.*

Upload the filled template Post the completed file to `/academics/grades/upload` as a multipart form.

```
POST https://api.uniz.rguktong.in/api/v1/academics/grades/upload
    Authorization: Bearer <your-token>
    Content-Type: multipart/form-data

    file=<your-filled-template.xlsx>
```

Success response:

```
{
      "success": true,
      "processed": 62,
      "successCount": 60,
      "failCount": 2,
      "errors": [
        "Row 14: Invalid grade value 'NA' for student O210045",
        "Row 31: Student ID O219999 not found in this batch"
      ]
    }
```

Upload is processed as a background batch job. The `successCount` and `failCount` fields give you an immediate summary. Review any entries listed in `errors` and correct them before re-uploading.

Note: *Uploading a file does not overwrite previously uploaded grades for students not included in the new file. Only rows present in the uploaded sheet are processed.*

Monitor upload progress For large batches, poll the progress endpoint until the job completes.

```
GET https://api.uniz.rguktong.in/api/v1/academics/grades/upload/progress
    Authorization: Bearer <your-token>
```

Response:

```
{
      "success": true,
      "progress": {
        "status": "processing",
        "percent": 45,
        "processed": 150,
        "total": 330
      }
    }
```

Poll every few seconds until `status` is `"completed"` and `percent` reaches `100`. You can also check this from the **Add Grades** page in the portal.

—

Add grades manually for individual students

To record or correct a grade for a single student without re-uploading a full sheet, use the add grades endpoint directly.

```
POST https://api.uniz.rguktong.in/api/v1/academics/grades/add
Authorization: Bearer <your-token>
Content-Type: application/json
```

Request body:

```
{
  "studentId": "O210008",
  "subjectCode": "E3-SEM-1-CSE-1",
  "semesterId": "SEM-1",
  "grade": "A"
}
```

Success response:

```
{
  "success": true,
  "message": "Grade added successfully"
}
```

Use this for late submissions, re-evaluations, or corrections after the main upload has been processed.

—

Bulk update grades via JSON

When you need to correct multiple grades without uploading a new Excel file — for example, after a re-evaluation — send a JSON array of updates to the bulk update endpoint.

```
PUT https://api.uniz.rguktong.in/api/v1/academics/grades/bulk-update
Authorization: Bearer <your-token>
Content-Type: application/json
```

Request body:

```
{
  "updates": [
    {
      "studentId": "O210008",
      "subjectId": "uuid-of-subject",
      "semesterId": "SEM-1",
      "grade": "EX"
    },
    {
      "studentId": "O210045",
      "subjectId": "uuid-of-subject",
      "semesterId": "SEM-1",
      "grade": 9.5
```

```
    }
  ]
}
```

The `grade` field accepts either a letter grade string (`"EX"`, `"A"`, `"F"`, etc.) or a numeric points value like `9.5`.

Pro-Tip: *Use the batch grades endpoint (`GET /academics/grades/batch`) to retrieve subject IDs before performing bulk updates. Query params: `branch`, `year`, `semesterId`, and optionally `failedOnly=true` to focus on failed records.*

—

Download a grade report

To download a consolidated grade report for a semester, use the download endpoint with the semester ID.

```
GET https://api.uniz.rguktong.in/api/v1/academics/grades/download/:semesterId
Authorization: Bearer <your-token>
```

Example:

```
GET https://api.uniz.rguktong.in/api/v1/academics/grades/download/SEM-1
Authorization: Bearer <your-token>
```

The response is a downloadable report file for that semester. Use this to verify grades before requesting publication, or to keep a local record.

—

Publishing results to students

Warning: ***Only the Director can publish results.*** *After you upload grades, students cannot see them until the Director triggers the publish action. You do not need to take any further steps — notify your department's administration that grades are ready for review and publication.*

When the Director publishes results for a semester, UniZ triggers a background job that sends report card emails to all students for that semester.

You can monitor the publish progress at:

```
GET https://api.uniz.rguktong.in/api/v1/academics/grades/publish/progress
Authorization: Bearer <your-token>
```

Response:

```
{
  "success": true,
  "progress": {
    "status": "processing",
    "percent": 72,
    "processed": 240,
    "total": 330
  }
}
```

—

Grade format reference

Letter grades `EX` — Excellent

`A` — Grade A

`B` — Grade B

`C` — Grade C

`D` — Grade D

`F` — Fail

Numeric points You can also submit grades as numeric point values (e.g., `9.5`, `7.0`). The system accepts both letter and numeric formats in the template and via the API.

The academic management section covers everything from viewing batch grades and running bulk updates to uploading new student cohorts and managing homepage banners. These features are available to the **dean**, **director**, and **webmaster** roles.

Viewing batch grades

Fetch grades for an entire batch at once using the batch grades endpoint. You can filter by branch, year, semester, and optionally limit results to failed students only.

Endpoint: `GET /academics/grades/batch`

```
GET https://api.uniz.rguktong.in/api/v1/academics/grades/batch?branch=CSE&year=E2&semesterId=
    SEM-1&failedOnly=false
Authorization: Bearer <TOKEN>
```

Query parameter	Example	Description
`branch`	`CSE`	Filter by branch (CSE, ECE, EEE, etc.)
`year`	`E2`	Filter by year (E1–E4)
`semesterId`	`SEM-1`	Semester identifier
`failedOnly`	`true` / `false`	Return only failed grade records when `true`

Response:

```
{
  "success": true,
  "summary": {
    "totalStudents": 3,
    "totalRecords": 7,
    "failedRecords": 2,
    "timestamp": "2026-02-01T10:00:00Z"
  },
  "students": [
    {
      "studentId": "O210139",
      "name": "DAMARLA SEETHA RAM PRAVEEN",
      "branch": "CSE",
      "year": "E2",
      "records": [
        {
          "subjectCode": "E2-SEM-1-CSE-3",
          "subjectName": "Design & Analysis of Algorithms",
          "grade": 0,
          "credits": 4,
          "semesterId": "SEM-1"
        }
      ]
    }
  ]
}
```

Bulk updating grades via JSON

Use this endpoint to update multiple grades directly without uploading an Excel file. This is the preferred method for re-evaluations or small corrections.

Endpoint: `POST /academics/grades/bulk-update`

```
POST https://api.uniz.rguktong.in/api/v1/academics/grades/bulk-update
Authorization: Bearer <TOKEN>
Content-Type: application/json
```

```
{
  "updates": [
    {
      "studentId": "O210008",
      "subjectId": "550e8400-e29b-41d4-a716-446655440001",
      "semesterId": "SEM-1",
      "grade": "EX"
    },
    {
      "studentId": "O210009",
      "subjectId": "550e8400-e29b-41d4-a716-446655440002",
      "semesterId": "SEM-1",
      "grade": 9.5
    }
```

```
  ]
}
```

The `grade` field accepts either a letter grade string (`"EX"`, `"A"`, `"B"`, etc.) or a numeric grade point value.

Pro-Tip: *Use `GET /academics/grades/batch` first to retrieve the correct `subjectId` values before constructing a bulk-update payload. Incorrect subject IDs will be silently skipped.*

Uploading grades via Excel

For large-scale grade entry, download a pre-filled template, fill in grades, and upload the file.

Download the grade template Fetch the Excel template for a specific batch. The file comes pre-filled with student IDs for the selected branch, year, and semester.

```
GET https://api.uniz.rguktong.in/api/v1/academics/grades/template?branch=CSE&year=E1&semester=SEM-1&subjectCode=E1-SEM-1-CSE-1
    Authorization: Bearer <TOKEN>
```

Save the returned `.xlsx` file to your computer.

Fill in the grades Open the template in Excel or Google Sheets. Fill in the `grade` column for each student. Do not modify the student ID column or the sheet structure.

Upload the completed file Submit the filled file to the upload endpoint. The system processes the records in batches.

```
POST https://api.uniz.rguktong.in/api/v1/academics/grades/upload
    Authorization: Bearer <TOKEN>
    Content-Type: multipart/form-data
```

The response includes a success count, failure count, and a list of any errors.

Monitor upload progress Poll the upload progress endpoint to track the background job.

```
GET https://api.uniz.rguktong.in/api/v1/academics/grades/upload/progress
    Authorization: Bearer <TOKEN>
```

```
{
      "success": true,
      "progress": {
        "status": "processing",
        "percent": 45,
        "processed": 150,
        "total": 330
      }
    }
```

When `status` returns `"completed"`, the upload is finished.

Downloading grade and attendance reports

Use the template endpoint with specific filters to download grade reports for any combination of branch, year, semester, and subject. Reports are returned as `.xlsx` files.

For attendance data, staff with the appropriate role can access attendance records through the student profile endpoints filtered by batch parameters.

Publishing results by email

Warning: *This action is restricted to the **director** role only. It triggers a background bulk-email job that sends report cards to all students for the specified semester. Confirm the grades are finalized before publishing.*

Endpoint: `POST /academics/grades/publish-email`

```
POST https://api.uniz.rguktong.in/api/v1/academics/grades/publish-email
Authorization: Bearer <TOKEN>
Content-Type: application/json
```

```
{
  "semesterId": "SEM-1"
}
```

After triggering the job, monitor its progress:

```
GET https://api.uniz.rguktong.in/api/v1/academics/grades/publish/progress
Authorization: Bearer <TOKEN>
```

```
{
  "success": true,
  "progress": {
    "status": "processing",
    "percent": 72,
    "processed": 238,
    "total": 330
  }
}
```

Bulk uploading students

Use the Add Students feature to onboard a new batch or add records for a new semester. Supported file formats are `.xlsx`, `.xls`, and `.csv`.

Configure the batch parameters Select the **Year** (E1–E4), **Semester**, and **Branch** for the student cohort you are uploading. These values are attached to every student record in the upload.

Download the student template

```
GET https://api.uniz.rguktong.in/api/v1/admin/student/template
    Authorization: Bearer <TOKEN>
```

The downloaded `.xlsx` file contains the required column headers for the import. Do not modify the headers.

Fill in the spreadsheet Add one row per student. Do not change the column headers.

Column	Example	Notes
`Username`	`O210008`	University roll number
`FullName`	`DESU SABER`	Full name in uppercase
`Email`	`o210008@rguktong.ac.in`	Institutional email

Upload the spreadsheet

```
POST https://api.uniz.rguktong.in/api/v1/admin/student/upload
    Authorization: Bearer <TOKEN>
    Content-Type: multipart/form-data
```

Send the file as `multipart/form-data` with the field name `file`. A successful upload returns a `jobId` you can use with the progress endpoint to track background processing.

Banner management

Webmasters create and manage homepage banners visible to all students in the UniZ portal. Banners support a title, description text, and an image.

Note: *Only the **webmaster** role can create and publish banners. The director and dean can view banners but cannot modify them.*

Creating a banner

Endpoint: `POST /profile/admin/banners`

```
POST https://api.uniz.rguktong.in/api/v1/profile/admin/banners
Authorization: Bearer <TOKEN>
Content-Type: application/json
```

```
{
  "title": "Holiday Notice",
  "text": "The campus will be closed on 15th August for Independence Day.",
  "imageUrl": "https://example.com/banners/holiday.jpg"
}
```

Response:

```
{
  "success": true,
  "banners": [
    {
      "id": "ban-456",
      "title": "Holiday Notice",
      "text": "The campus will be closed on 15th August for Independence Day.",
      "imageUrl": "https://...",
      "isPublished": false,
      "createdAt": "2026-08-01T09:00:00Z"
    }
  ]
}
```

New banners are created as drafts (`isPublished: false`) and are not visible to students until you publish them.

Publishing and unpublishing a banner

Endpoint: `POST /profile/admin/banners/:id/publish`

```
POST https://api.uniz.rguktong.in/api/v1/profile/admin/banners/ban-456/publish
Authorization: Bearer <TOKEN>
Content-Type: application/json
```

To publish:

```
{
  "publish": true
}
```

To unpublish (revert to draft):

```
{
  "publish": false
}
```

Viewing all banners

```
GET https://api.uniz.rguktong.in/api/v1/profile/admin/banners
Authorization: Bearer <TOKEN>
```

This returns all banners regardless of published status. The student-facing view only shows banners where `isPublished: true`.

Student outpass requests move through a fixed multi-level approval chain before the student is cleared to leave campus. Each admin role is responsible for a specific level in this chain.

Approval chain

Outpass requests (multi-day home leave) go through the following sequence:

```
Student submits     Caretaker     Warden     SWO     Dean / Director (final)
```

Outing requests (short duration, same day) follow a lighter path with less overhead. Check your institution's configuration for the exact levels required for outings.

Note: *The `currentLevel` field on each request tells you exactly which role needs to act next. You only see requests assigned to your level — caretakers see `currentLevel: "caretaker"`, wardens see `"warden"`, and so on.*

What each role does

| Role | Level | Effect of approval | |—|—|—| | Caretaker | `caretaker` | Moves request to `warden` level | | Warden | `warden` | Moves request to `swo` level | | SWO | `swo` | Moves request to dean / director level | | Dean / Director | Final | Sets `isApproved: true`; student is notified and security can verify the pass |

Viewing pending requests

Fetch all pending outpass requests. The API returns all records — your frontend filters them to show only those matching your role's `currentLevel`.

Endpoint: `GET /requests/outpass/all`

```
GET https://api.uniz.rguktong.in/api/v1/requests/outpass/all
Authorization: Bearer <TOKEN>
```

Response:

```
{
  "success": true,
  "outpasses": [
    {
      "id": "req-123",
      "studentId": "O210008",
      "studentGender": "M",
      "reason": "Home visit due to family function",
      "currentLevel": "caretaker",
      "requested_time": "2026-02-10T10:00:00Z",
      "fromDay": "2026-02-12T09:00:00Z",
      "toDay": "2026-02-15T18:00:00Z",
      "isApproved": false,
      "isRejected": false
    }
  ]
}
```

To view pending outing requests, use the equivalent endpoint:

```
GET https://api.uniz.rguktong.in/api/v1/requests/outing/all
Authorization: Bearer <TOKEN>
```

Searching and filtering

The approvals UI lets you search by student ID, email, or reason text. Use the search bar at the top of the approvals view to narrow down the queue when you have a large number of pending requests.

Approving a request

Send a `POST` to `/:id/approve` with the request ID. You may include an optional comment.

Endpoint: `POST /requests/:id/approve`

```
POST https://api.uniz.rguktong.in/api/v1/requests/req-123/approve
Authorization: Bearer <TOKEN>
Content-Type: application/json
```

```
{
  "comment": "Parent verified via call. Approved."
}
```

Response:

```
{
  "success": true,
  "msg": "Request approved and forwarded to next level."
}
```

Pro-Tip: *The comment field is optional for approvals. Use it to leave a note for the next approver in the chain, especially if you verified information out-of-band (a phone call, parent visit, etc.).*

After approval, the request automatically advances to the next level. The student is not notified until the final approver (dean or director) grants approval.

Forwarding a request

You can also forward a request directly to the next level without approving it — use this when you want to pass a request up the chain for review without taking an approval action yourself.

Endpoint: `POST /requests/:id/forward`

```
POST https://api.uniz.rguktong.in/api/v1/requests/req-123/forward
Authorization: Bearer <TOKEN>
```

Note: *Forward is an alias for approve on the backend. It has the same effect — the request moves to the next `currentLevel`. The distinction is presentational in the UI.*

Rejecting a request

To reject a request, provide a mandatory reason in the `comment` field. The student is notified immediately when a request is rejected at any level.

Endpoint: `POST /requests/:id/reject`

```
POST https://api.uniz.rguktong.in/api/v1/requests/req-123/reject
Authorization: Bearer <TOKEN>
Content-Type: application/json
```

```
{
  "comment": "Exam period     no leaves permitted this week."
}
```

Response:

```
{
  "success": true,
  "msg": "Request rejected."
}
```

Warning: *The `comment` field is required when rejecting. A rejection without a reason will fail validation. Always provide a clear, specific reason so the student understands and can appeal if necessary.*

After final approval

When a dean or director approves a request, the following happens automatically:

1. The request status changes to `isApproved: true`. 2. The student receives a notification that their outpass has been granted. 3. The request becomes visible to security staff at the gate, who can see it when the student presents their pass for check-out.

Security uses the approved outpass to verify the student's identity and entitlement before allowing them to leave campus. See the Security Portal guide for details.

Expired requests

Requests that have passed their `fromDay` / `fromTime` without being fully approved are marked as expired. Expired requests appear greyed-out in the UI and cannot be actioned. No further action is needed on expired items.

The UniZ admin portal gives non-student staff a dedicated interface for managing approvals, academic data, campus security, and announcements. Every administrative role — caretaker, warden, SWO, dean, director, and webmaster — uses the same portal with a role-appropriate view.

Logging in

All admin accounts use the same login endpoint as students. The `role` field in the response determines which dashboard and features you can access.

Endpoint: `POST /auth/login`

```
POST https://api.uniz.rguktong.in/api/v1/auth/login
Content-Type: application/json
```

```
{
  "username": "warden_male",
  "password": "your_password"
}
```

Response:

```
{
  "success": true,
  "token": "eyJ...",
  "role": "warden_male",
  "username": "warden_male"
}
```

Store the returned token and include it as a `Bearer` token in the `Authorization` header for all subsequent requests.

Note: *Admin usernames follow a fixed pattern:* `director`, `dean_cse`, `warden_male`, `warden_female`, `caretaker_male`, `caretaker_female`, `security_admin`, *and* `webmaster`. *Contact your system administrator if you do not have your credentials.*

Role-based access

Each admin role has a tailored view of the portal. The table below maps roles to their primary responsibilities.

Role	Username pattern	Primary access
Director	`director`	Final approvals, student search, academic publishing
Dean	`dean_cse`	Final approvals, grade management, academic oversight
Warden	`warden_male` / `warden_female`	Outpass approvals (second level)
Caretaker	`caretaker_male` / `caretaker_female`	Outpass approvals (first level)
SWO	`swo`	Outpass approvals (third level)
Security	`security_admin`	Gate check-in and check-out
Webmaster	`webmaster`	Banner management, grade uploads

What the dashboard shows

When you log in, the dashboard surfaces the information most relevant to your role.

- **Caretakers and wardens** see a queue of pending outpass requests assigned to their level. - **Security staff** see the access control terminal with real-time student presence data. - **Dean and director** see pending final-approval requests, student search, and academic controls. - **Webmaster** sees banner management and announcement tools.

The portal automatically hides features your role cannot access. You do not need to configure anything — access is determined by the `role` returned at login.

System health

You can check the status of all UniZ backend services at any time without authentication.

Endpoint: `GET /system/health`

```
GET https://api.uniz.rguktong.in/api/v1/system/health
```

```
{
  "success": true,
  "services": [
    { "name": "Auth Service", "status": "UP", "latency": "120ms" },
    { "name": "Mail Service", "status": "UP", "latency": "45ms" }
```

```
  ]
}
```

Use this endpoint to verify that the platform is operational before performing bulk operations like grade uploads or result publishing.

Key admin sections

Approvals Review, approve, forward, or reject student outpass and outing requests. Each role operates at a specific level in the multi-step chain.

Academic management View batch grades, run bulk updates, upload students, manage semesters, and publish results by email.

Security portal Check students in and out at the campus gate, view approved passes, and monitor who is currently outside campus.

Banners Create and publish homepage announcements visible to all students in the portal.

The Security Portal gives security staff at the campus gate a dedicated interface for verifying approved passes, logging student entry and exit, and tracking who is currently outside campus.

Note: *Log in with the `security_admin` username. The Security Portal is role-restricted — other admin roles cannot access the gate control functions.*

Dashboard overview

When you log in, the Security Portal dashboard shows four real-time metrics:

- **Students outside** — total number of students currently not in campus - **Today's check-ins** — students who have returned to campus today - **Today's check-outs** — students who have left campus today - **Sync status** — whether the terminal is connected to the central server

These counters update automatically. The portal polls the summary endpoint every five minutes when the tab is active, and every 30 seconds as a fallback.

Security summary endpoint

Endpoint: `GET /requests/security/summary`

```
GET https://api.uniz.rguktong.in/api/v1/requests/security/summary
Authorization: Bearer <TOKEN>
```

```
{
  "success": true,
  "summary": {
    "totalOutside": 14,
    "todayCheckins": 8,
    "todayCheckouts": 22
  }
}
```

Viewing approved passes

Approved outpasses

Fetch all outpass records to verify a student's approved long-duration leave.

Endpoint: `GET /requests/outpass/all`

```
GET https://api.uniz.rguktong.in/api/v1/requests/outpass/all
Authorization: Bearer <TOKEN>
```

```
{
  "success": true,
  "outpasses": [
    {
      "id": "req-123",
      "studentId": "O210008",
      "studentGender": "M",
      "reason": "Home visit",
      "isApproved": true,
      "fromDay": "2026-02-12T09:00:00Z",
      "toDay": "2026-02-15T18:00:00Z",
      "currentLevel": "approved"
    }
  ]
}
```

Only show passes where `isApproved: true` to the gate officer. Pending or rejected passes are not valid for exit.

Approved outings

Fetch approved short-duration outing requests.

Endpoint: `GET /requests/outing/all`

```
GET https://api.uniz.rguktong.in/api/v1/requests/outing/all
Authorization: Bearer <TOKEN>
```

Gate check-out process

Use this procedure when a student presents their approved pass at the gate to leave campus.

Search for the student Enter the student's university ID in the scan field on the Security Portal dashboard. You can type the ID manually or scan the student's ID card.

```
POST https://api.uniz.rguktong.in/api/v1/profile/student/search
    Authorization: Bearer <TOKEN>
    Content-Type: application/json
```

```
{
      "username": "O210008",
      "limit": 5
    }
```

The response returns matching student records with name, branch, and year.

Verify the approved pass Confirm that the student has an approved outpass or outing by checking `isApproved: true` on their request record. Cross-check the `fromDay`/`toDay` (outpass) or `fromTime`/`toTime` (outing) against the current date and time.

Log the exit Call the check-out endpoint with the student's username as the request ID. This marks the student as outside campus in the system.

Endpoint: `POST /requests/:id/checkout`

```
POST https://api.uniz.rguktong.in/api/v1/requests/O210008/checkout
    Authorization: Bearer <TOKEN>
```

```
{
      "success": true,
      "msg": "Checked-Out"
    }
```

The student's `is_in_campus` status changes to `false` in the system.

Gate check-in process

When a student returns to campus, log their entry to update their presence status.

Search for the returning student Enter the student's university ID in the scan field.

Log the entry Call the check-in endpoint to mark the student as back on campus.

Endpoint: `POST /requests/:id/checkin`

```
POST https://api.uniz.rguktong.in/api/v1/requests/O210008/checkin
    Authorization: Bearer <TOKEN>
```

```
{
      "success": true,
      "msg": "Checked-In"
    }
```

The student's `is_in_campus` status changes to `true`.

Manually updating student presence status

If you need to correct a student's campus status outside of the normal check-in/check-out flow — for example, if a system error left the status incorrect — use the presence status endpoint directly.

Endpoint: `PUT /profile/student/status`

```
PUT https://api.uniz.rguktong.in/api/v1/profile/student/status
Authorization: Bearer <TOKEN>
Content-Type: application/json
```

Set `isPresent` to `false` when the student is leaving, and `true` when they have returned.

```
{
  "username": "O210008",
  "isPresent": false
}
```

Warning: *Manually updating presence status bypasses the outpass verification check. Only use this endpoint to correct data errors, not as a substitute for the standard check-in/check-out flow.*

Viewing all students currently outside campus

Generate a list of all students who have not yet returned to campus. This is useful for end-of-day reconciliation and for identifying overdue returns.

Endpoint: `POST /profile/student/search`

```
POST https://api.uniz.rguktong.in/api/v1/profile/student/search
Authorization: Bearer <TOKEN>
Content-Type: application/json
```

```
{
  "isPresentInCampus": false,
  "limit": 50
}
```

Response:

```
{
  "success": true,
  "students": [
    {
      "username": "O210008",
      "name": "DESU SABER",
      "branch": "CSE",
      "is_in_campus": false
    }
  ],
  "pagination": {
    "page": 1,
    "totalPages": 1,
    "total": 14
  }
}
```

Increase the `limit` value if you need to fetch more than 50 records in a single request.

Activity logs

The Security Portal includes an **Activity Logs** tab that shows recent entry and exit events recorded at the gate. Use it to review a history of check-in and check-out actions for audit or dispute resolution.

The Academics section shows your grade history, attendance per subject, and the subjects you're registered for in the current semester.

—

Grades

View your grades

Open the **Academic** tab on your profile page. Your grades are shown grouped by semester. Each record includes the subject name, subject code, number of credits, and the grade awarded.

API response example:

```
{
  "success": true,
  "grades": [
    {
      "id": "abc-123",
      "grade": "EX",
      "semesterId": "E3S1",
      "subject": {
        "code": "CS3101",
        "name": "Operating Systems",
        "credits": 4
      }
    },
    {
      "id": "abc-456",
      "grade": "A",
      "semesterId": "E3S1",
      "subject": {
        "code": "CS3102",
        "name": "Compiler Design",
        "credits": 4
      }
    }
  ],
  "source": "cache"
}
```

The `source` field will be `"cache"` or `"db"`. When served from cache, responses load in under 50ms.

Understanding grade values

RGUKT uses a letter-grade system. The grades you may see in UniZ:

| Grade | Description | |———-|————-| | `EX` | Excellent — highest grade | | `A` | Very Good | | `B` | Good | | `C` | Average | | `D` | Below Average | | `F` | Fail | | `AB` | Absent |

Note: *If no grade appears for a subject, results for that semester have not been published yet. See the section below on published results.*

Published results

Faculty and administrators upload and then **publish** results before they become visible to you. Until results are published:

- The subject appears in your registration list but shows no grade. - The portal does not display a placeholder or draft grade — the entry simply remains blank.

When results are published, you receive an email notification and the grades appear automatically in the portal. You don't need to refresh or take any action.

—

Attendance

View your attendance

Your attendance is shown on the **Academic** tab alongside your grades. Each row shows the subject name, how many classes you attended, and the total classes held.

API response example:

```
{
  "success": true,
  "attendance": [
```

```
    {
      "id": "att-123",
      "attendedClasses": 45,
      "totalClasses": 50,
      "subject": {
        "code": "CS3101",
        "name": "Operating Systems"
      }
    }
  ]
}
```

Your attendance percentage for a subject is:

```
(attendedClasses / totalClasses)    100
```

In the example above: `(45 / 50) × 100 = 90%`.

Warning: *RGUKT requires a minimum attendance percentage to be eligible to sit examinations. Check with your faculty for the current threshold — typically 75%. If you are below the threshold, contact your class coordinator immediately.*

—

Current semester

View registered subjects

The **Current Semester** section shows all subjects you are registered for in the active semester, along with each subject's code and credit value.

The page header shows:

- **Semester name** — e.g., `E3 SEM-1` - **Status** — e.g., `ONGOING` - **Total subjects** and **total credits** for the semester

Each subject card shows:

- Subject code (e.g., `CS3101`) - Subject name (e.g., `Operating Systems`) - Credit value

Note: *If you see "No Registered Subjects", you have not completed registration for the active semester. Visit the registration portal while it is open, or contact your academic section.*

If the page shows "No Active Semester", the administration has not yet opened a semester session for your batch. Check back later or contact your academic office.

Semester registration (student view)

Registration is managed by the administration. From your side:

1. The administration activates a new semester and opens registration. 2. You register your subjects through the registration portal (separate from UniZ). 3. Once confirmed, your registered subjects appear in UniZ automatically. 4. You can verify your registration is complete by checking that all expected subjects appear under **Current Semester**.

Pro-Tip: *Cross-check your registered subjects at the start of each semester. If a subject is missing or incorrect, contact your academic section before the registration window closes.*

The UniZ portal is your central hub for academics, leave requests, and your student profile. Your login credentials are your RGUKT roll number and a password set by the administration.

Sign in

Open the portal Navigate to the UniZ student portal URL provided by your institution.

Enter your credentials - **Username** — your roll number (e.g., `O210008`) - **Password** — your current password

Tap Sign In On success, you are redirected to your profile dashboard. The portal stores your session securely in the browser — you do not need to enter your credentials again until you log out or your session expires.

Note: *Your username is always your roll number in uppercase (e.g., `O210008`). It never changes.*

How authentication works

When you sign in, the server issues a JWT (JSON Web Token) that the portal stores in your browser. Every subsequent request automatically includes this token — you don't need to do anything extra in the portal.

If you are building on top of the API directly, include the token in every request:

```
Authorization: Bearer <your-token>
```

Login endpoint (for API use):

```
POST https://api.uniz.rguktong.in/api/v1/auth/login/student
```

```
{
  "username": "O210008",
  "password": "password123"
}
```

Success response:

```
{
  "success": true,
  "token": "eyJhbGciOiJIUz...",
  "role": "student",
  "username": "O210008"
}
```

—

Reset your password

Use this flow if you've forgotten your password. You'll need access to the email address registered with your account.

Go to the login page Tap **Forgot Password** on the sign-in screen.

Enter your roll number Submit your username (`O210008`). The system sends a 6-digit OTP to your registered email address.

```
POST /auth/otp/request
    { "username": "O210008" }
```

Enter the OTP Check your inbox and enter the 6-digit code in the verification field.

```
POST /auth/otp/verify
    {
      "username": "O210008",
      "otp": "123456"
    }
```

Set a new password Enter and confirm your new password. The reset requires a valid OTP and the same username.

```
POST /auth/password/reset
    {
      "username": "O210008",
      "resetToken": "123456",
      "newPassword": "newSecurePassword123"
    }
```

Warning: *OTP requests are rate-limited. If you request too many OTPs in a short period, you'll be temporarily blocked. Wait a few minutes before trying again.*

—

Change your password

Once logged in, you can change your password from the **Reset Password** page in the portal. You'll need your current password.

A strong password must meet all three criteria:

- At least 8 characters - Contains at least one number - Contains at least one special character (e.g., `!@#$%`)

Warning: *After a successful password change, all active sessions are immediately logged out for security. You'll need to sign in again with your new password.*

The portal evaluates your new password in real time and shows a strength indicator (Weak / Moderate / Strong). Weak and Moderate passwords are rejected.

—

Install as a PWA

UniZ is a Progressive Web App (PWA). You can install it on your phone or desktop so it opens like a native app — no app store required.

Android

Open the portal in Chrome Navigate to the UniZ portal URL.

Tap the install prompt Chrome shows an **Add to Home Screen** banner at the bottom, or you can tap the three-dot menu and select **Install app / Add to Home Screen**.

Confirm Tap **Install**. The app icon appears on your home screen and opens in standalone mode.

iOS (Safari)

Open the portal in Safari PWA installation on iOS only works in Safari.

Tap the Share button Tap the share icon at the bottom of the screen (the box with an arrow pointing up).

Select Add to Home Screen Scroll down in the share sheet and tap **Add to Home Screen**, then tap **Add**.

Desktop (Chrome/Edge)

Open the portal Navigate to the UniZ portal URL in Chrome or Edge.

Click the install icon Look for an install icon in the address bar (a monitor with a download arrow), or open the browser menu and select **Install UniZ**.

Confirm Click **Install**. The app opens in its own window.

Pro-Tip: *Installing the PWA gives you a faster experience and keeps your session active between visits without navigating to the URL each time.*

—

Common errors

Invalid credentials Double-check that your roll number is entered in the correct format (e.g., `O210008` — capital letter O, not zero). Passwords are case-sensitive.

Account suspended If your account has been suspended by the administration, you'll see a suspension message on sign-in. Contact your warden or SWO to resolve this.

OTP not received Check your spam folder. If the email still hasn't arrived after a few minutes, confirm your registered email with your hostel admin and try requesting the OTP again.

Session expired JWT tokens expire after a set period. If you're redirected to the login page unexpectedly, simply sign in again.

UniZ handles two types of campus exit permissions:

- **Outpass** — multi-day leave (overnight or longer). Requires full multi-level approval before you can leave. - **Outing** — short-duration leave on the same day. Used for trips that start and end on the same day (e.g., going to town for a few hours).

Note: *You can only have one pending request at a time. You cannot submit a new outpass or outing while a previous request is still awaiting approval.*

—

Request an outpass

Use an outpass for any leave that involves an overnight stay outside campus.

Open the Permissions tab Go to your profile and select the **Permissions** tab. Tap **Request Outpass**.

Enter a reason Write a clear reason for your leave (e.g., "Emergency visit to home due to health issues"). This is reviewed by your caretaker, warden, and the SWO/Dean/Director in sequence.

Set departure and return dates Select your **From** date (when you plan to leave) and **To** date (when you plan to return). The return date must be on or after the departure date.

Submit the request Tap **Request Outpass**. If successful, the system creates a new request at the `caretaker` approval level.

```
POST /requests/outpass
    {
      "reason": "Emergency visit to home due to health issues",
      "fromDay": "2026-02-10T09:00:00Z",
      "toDay": "2026-02-15T18:00:00Z"
    }
```

Response:

```
{
      "success": true,
      "data": {
        "id": "req-789",
        "currentLevel": "caretaker",
        "isApproved": false,
        "createdAt": "2026-01-31T10:00:00Z"
      }
    }
```

—

Request an outing

Use an outing for same-day, short-duration trips that don't require an overnight stay.

Open the Permissions tab Go to your profile and select the **Permissions** tab. Tap **Request Outing**.

Enter a reason Write a brief reason (e.g., "Buying groceries").

Set departure and return times Select your **From** time and **To** time. Both times apply to today's date.

Submit the request Tap **Request Outing**.

```
POST /requests/outing
    {
      "reason": "Buying groceries",
      "fromTime": "2026-02-10T17:00:00Z",
      "toTime": "2026-02-10T20:00:00Z"
    }
```

—

Approval workflow

Both outpass and outing requests go through a multi-level approval chain before you can leave campus.

```
You submit     Caretaker     Warden     SWO / Dean / Director (final approval)
```

| Level | Role | Action | |——-|——|———| | 1 | Caretaker | Reviews and forwards to warden | | 2 | Warden | Reviews academic schedule and forwards to SWO | | 3 | SWO / Dean / Director | Grants final approval |

The `currentLevel` field in your request shows which level is currently reviewing it. Each approver may add a comment.

Note: *Caretakers are gender-specific — male caretakers review male students' requests, and female caretakers review female students' requests. The system routes your request automatically based on your profile.*

—

Track your requests

Open the **Permissions** tab on your profile to see your full request history. Each entry shows:

- Request type (outpass or outing) - Date and time range - Your reason - Current status

Request statuses

| Status | Meaning | |———|———| | **Pending** | Still moving through the approval chain | | **Approved** | Fully approved — you may leave campus | | **Rejected** | Declined by one of the approvers | | **Expired** | The request was not acted on before its departure date |

—

After approval

Once your request reaches `isApproved: true`:

- Security at the gate will see your approved request and allow you to exit. - Your campus presence status (`is_in_campus`) is updated to `false` when you check out at the gate. - When you return, security marks you back in and your status returns to `true`.

Warning: *Do not attempt to leave campus before your request is fully approved. Security verifies each student against the system — unapproved exits are recorded.*

—

If your request is rejected

When an approver rejects your request, you'll see a rejection status and any comment they added. Common reasons include:

- Upcoming examinations or assessments - Incomplete attendance records - Insufficient reason provided - Disciplinary holds

If you believe a rejection was in error, speak to your caretaker or warden directly.

After a rejection, the pending request clears and you can submit a new request if needed.

Your profile page is the central view of your student record in UniZ. It shows your personal details, academic information, family details, and your outpass/outing history.

—

Profile information

Your profile is divided into tabs:

Personal Full name, gender, blood group, phone number, and date of birth.

Academic Student ID (roll number), year, branch, section, and room number.

Family Father's and mother's name, occupation, email, and phone number.

Permissions Outpass and outing request history, and the option to submit new requests.

API response example — your profile:

```
{
  "success": true,
  "student": {
    "_id": "550e8400-e29b-41d4-a716-446655440000",
    "username": "O210008",
    "name": "DESU SABER",
    "year": "3",
    "branch": "CSE",
    "profile_url": "https://...",
    "has_pending_requests": false,
    "is_in_campus": true
  }
}
```

—

Update your details

You can update the following fields from the Personal and Family tabs:

- Phone number - Address - Blood group - Date of birth - Gender - Father's name and occupation - Mother's name and occupation - Father's phone number - Mother's phone number

Tap the edit icon On the profile header, tap the pencil icon next to your name, or use the **Edit** button (desktop: top-right corner).

Modify the fields you want to change Only changed fields are sent to the server — unchanged fields are left as-is.

Tap Save On desktop, tap **Save** in the top-right corner. On mobile, tap **Save Changes** at the bottom of the editing panel.

The update endpoint only accepts the fields you change:

```
PUT /profile/student/update
{
  "phone": "9876543210",
  "address": "New Address, Ongole"
}
```

Note: *Academic fields — your roll number, year, branch, section, and room number — are managed by the administration and cannot be edited from the portal.*

—

Profile photo

You can upload or replace your profile photo directly from the profile page.

Tap the camera icon Tap the small camera button at the bottom-right of your profile picture.

Select a photo Choose an image from your device. Accepted formats are standard image types (JPEG, PNG, etc.).

Wait for upload The photo is uploaded to cloud storage and your profile URL is updated automatically. You'll see a loading spinner while it uploads.

Pro-Tip: *Use a clear, recent photo — it may be used for identification purposes in campus records.*

—

Campus presence status

The `is_in_campus` field on your profile reflects whether you are currently recorded as present on campus.

| Value | Meaning | |——-|———| | `true` | You are currently on campus | | `false` | You have checked out and are outside campus |

This status is updated by security at the gate:

- When you leave on an approved outpass or outing, security marks you as **out** (`is_in_campus: false`). - When you return, security marks you as **in** (`is_in_campus: true`).

You cannot change this status yourself. If there's a discrepancy, contact the security office or your warden.

—

Request history

Your outpass and outing history is available in the **Permissions** tab. It shows a paginated list of all past requests with:

- Request type (outpass or outing) - Date and duration - Reason you submitted - Final status (Pending, Approved, Rejected, or Expired)

API response example:

```
{
  "success": true,
  "history": [
    {
      "_id": "req-123",
      "type": "outpass",
      "status": "pending",
      "reason": "Home visit",
      "requested_time": "2026-02-10T10:00:00Z"
    }
  ],
  "pagination": { "page": 1, "total": 5 }
}
```

Results are paginated at 5 per page. Use the pagination controls at the bottom to navigate older records.

—

Grievances and support

If you have a complaint or need help with something not covered in the portal, use the **Help & Support** section accessible from the main navigation.

Submit a brief description of your issue. It will be routed to the appropriate administrative staff.

Note: *For urgent matters — such as a rejected leave request you believe is incorrect, or a discrepancy in your academic record — contact your caretaker, warden, or academic section directly rather than waiting for a grievance response.*

All attendance endpoints require authentication. The specific role required varies per endpoint and is noted in each section.

—

Get my attendance

`GET /academics/attendance`

Returns the authenticated student's attendance records across all subjects for the current semester.

Auth required: Yes (student)

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/academics/attendance \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "attendance": [
    {
      "id": "att-123",
      "attendedClasses": 45,
      "totalClasses": 50,
      "subject": {
        "code": "CS3101",
        "name": "Operating Systems"
      }
    },
    {
      "id": "att-456",
      "attendedClasses": 38,
      "totalClasses": 50,
      "subject": {
        "code": "CS3102",
        "name": "Compiler Design"
      }
    }
  ]
}
```

Response fields

`true` on success.

Unique attendance record identifier.

Number of classes the student attended.

Total number of classes conducted.

Subject code (e.g. `CS3101`).

Full subject name.

—

Add attendance

`POST /academics/attendance/add`

Records attendance for a student in a subject for a given session.

Auth required: Yes (faculty)

Request body

Student roll number (e.g. `O210008`).

UUID of the subject.

Semester identifier (e.g. `SEM-1`).

`true` if the student was present, `false` if absent.

ISO 8601 date of the class session (e.g. `2026-03-15`). Defaults to today if omitted.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/academics/attendance/add \
  --header 'Authorization: Bearer <token>' \
```

```
  --header 'Content-Type: application/json' \
  --data '{
    "studentId": "O210008",
    "subjectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "semesterId": "SEM-1",
    "attended": true,
    "date": "2026-03-15"
  }'
```

200 OK

```
{
  "success": true,
  "message": "Attendance recorded"
}
```

—

Download attendance template

`GET /academics/attendance/template`

Downloads an Excel template pre-filled with student IDs for the specified batch. Fill in the attendance column and upload the file using the upload endpoint.

Auth required: Yes (faculty, admin)

Query parameters

Branch code (e.g. `CSE`).

Academic year (e.g. `E1`).

Semester label (e.g. `SEM-1`).

Subject code for which to generate the template (e.g. `E1-SEM-1-CSE-1`).

Example

```
curl --request GET \
  --url 'https://api.uniz.rguktong.in/api/v1/academics/attendance/template?branch=CSE&year=E1&
    semester=SEM-1&subjectCode=E1-SEM-1-CSE-1' \
  --header 'Authorization: Bearer <token>' \
  --output attendance_template.xlsx
```

Returns a binary Excel file (`.xlsx`).

—

Upload attendance

`POST /academics/attendance/upload`

Uploads a completed Excel template to import attendance records in bulk.

Auth required: Yes (faculty, admin)

Content type: `multipart/form-data`

Request body

The completed Excel file (`.xlsx`) previously downloaded from the template endpoint.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/academics/attendance/upload \
  --header 'Authorization: Bearer <token>' \
  --form 'file=@/path/to/attendance_template.xlsx'
```

200 OK

```
{
  "success": true,
  "message": "Upload started",
  "jobId": "job-def-456"
}
```

Pro-Tip: *Poll `GET /academics/attendance/upload/progress` with the returned `jobId` to track processing status.*

—

Check upload progress

`GET /academics/attendance/upload/progress`

Polls the status of a background attendance upload job.

Auth required: Yes (faculty, admin)

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/academics/attendance/upload/progress \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "progress": {
    "status": "processing",
    "percent": 60,
    "processed": 200,
    "total": 330
  }
}
```

Response fields

Current job status. One of `queued`, `processing`, `completed`, `failed`.

Completion percentage (0–100).

Number of records processed so far.

Total number of records in the job.

—

Download attendance report

`GET /academics/attendance/download/:semesterId`

Downloads the compiled attendance report for a semester as an Excel file.

Auth required: Yes (admin)

Path parameters

Semester identifier (e.g. `SEM-1`).

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/academics/attendance/download/SEM-1 \
  --header 'Authorization: Bearer <token>' \
  --output attendance_report_SEM-1.xlsx
```

Returns a binary Excel file.

—

Publish attendance via email

`POST /academics/attendance/publish-email`

Warning: *This endpoint is deprecated. Use the download endpoint (`GET /academics/attendance/download/:semeste` to export attendance reports. This endpoint is retained for audit compliance and may be removed in a future version.*

Triggers a background job to email attendance reports to students for a given semester.

Auth required: Yes (director)

Request body

Semester identifier whose attendance should be published (e.g. `SEM-1`).

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/academics/attendance/publish-email \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "semesterId": "SEM-1"
  }'
```

200 OK

```
{
  "success": true,
  "message": "Attendance publish job started"
}
```

All grades endpoints require authentication. The specific role required varies per endpoint and is noted in each section.

Note: *`GET /academics/grades` is served from a cache layer and typically responds in under 50 ms.*

—

Get my grades

`GET /academics/grades`

Returns the authenticated student's grades across all semesters.

Auth required: Yes (student)

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/academics/grades \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "grades": [
    {
      "id": "abc-123",
      "grade": "EX",
      "semesterId": "E3S1",
      "subject": {
        "code": "CS3101",
        "name": "Operating Systems",
        "credits": 4
      }
    },
    {
      "id": "abc-456",
      "grade": "A",
      "semesterId": "E3S1",
      "subject": {
        "code": "CS3102",
        "name": "Compiler Design",
        "credits": 4
      }
    }
  ],
  "source": "cache"
}
```

Response fields

`true` on success.

Unique grade record identifier.

Letter grade assigned (e.g. `EX`, `A`, `B`, `C`, `F`).

Semester the grade belongs to (e.g. `E3S1`).

Subject code (e.g. `CS3101`).

Full subject name.

Credit hours for the subject.

`cache` if served from the cache layer, `db` if fetched directly from the database.

—

Get batch grades

`GET /academics/grades/batch`

Returns grades for an entire batch, with optional filtering. Intended for administrative use.

Auth required: Yes (webmaster, dean, director)

Query parameters

Branch code (e.g. `CSE`, `ECE`, `ME`).

Academic year (e.g. `E1`, `E2`, `E3`, `E4`).

Semester identifier (e.g. `SEM-1`, `SEM-2`).

When `true`, returns only students with failing grades. Defaults to `false`.
Example

```
curl --request GET \
  --url 'https://api.uniz.rguktong.in/api/v1/academics/grades/batch?branch=CSE&year=E2&
    semesterId=SEM-1&failedOnly=true' \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "summary": {
    "totalStudents": 3,
    "totalRecords": 7,
    "failedRecords": 7,
    "timestamp": "2026-02-01T10:00:00.000Z"
  },
  "students": [
    {
      "studentId": "O210139",
      "name": "DAMARLA SEETHA RAM PRAVEEN",
      "branch": "CSE",
      "year": "E2",
      "records": [
        {
          "subjectCode": "E2-SEM-1-CSE-3",
          "subjectName": "Design & Analysis of Algorithms",
          "grade": 0,
          "credits": 4,
          "semesterId": "SEM-1"
        }
      ]
    }
  ]
}
```

Response fields
Number of distinct students in the result set.
Total grade records returned.
Number of failing grade records.
ISO 8601 timestamp of when the data was fetched.
Array of student grade summaries.
—
Add grade
`POST /academics/grades/add`
Records a grade for a single student in a specific subject and semester.
Auth required: Yes (faculty, admin)
Request body
Student roll number (e.g. `O210008`).
UUID of the subject.
Semester identifier (e.g. `SEM-1`).
Letter grade or numeric points to assign (e.g. `EX`, `A`, `9.5`).
Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/academics/grades/add \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "studentId": "O210008",
    "subjectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "semesterId": "SEM-1",
    "grade": "EX"
  }'
```

200 OK

```
{
  "success": true,
  "message": "Grade added successfully"
}
```

—

Bulk update grades

`PUT /academics/grades/bulk-update`

Updates multiple grade records in a single request. Useful for re-evaluations or corrections without uploading an Excel file.

Auth required: Yes (webmaster, dean)

Request body

Array of grade update objects.

Student roll number.

UUID of the subject.

Semester identifier.

New letter grade or numeric points.

Example

```
curl --request PUT \
  --url https://api.uniz.rguktong.in/api/v1/academics/grades/bulk-update \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "updates": [
      {
        "studentId": "O210008",
        "subjectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "semesterId": "SEM-1",
        "grade": "EX"
      },
      {
        "studentId": "O210139",
        "subjectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "semesterId": "SEM-1",
        "grade": "A"
      }
    ]
  }'
```

200 OK

```
{
  "success": true,
  "message": "Grades updated successfully",
  "updated": 2
}
```

—

Download grade template

`GET /academics/grades/template`

Downloads an Excel template pre-filled with student IDs for the specified batch. Fill in the grade column and upload the file using the upload endpoint.

Auth required: Yes (faculty, admin)

Query parameters

Branch code (e.g. `CSE`).

Academic year (e.g. `E1`).

Semester label (e.g. `SEM-1`).

Subject code to generate the template for (e.g. `E1-SEM-1-CSE-1`).

Example

```
curl --request GET \
  --url 'https://api.uniz.rguktong.in/api/v1/academics/grades/template?branch=CSE&year=E1&
    semester=SEM-1&subjectCode=E1-SEM-1-CSE-1' \
  --header 'Authorization: Bearer <token>' \
  --output grades_template.xlsx
```

Returns a binary Excel file (`.xlsx`).

—

Upload grades

`POST /academics/grades/upload`

Uploads a filled-in Excel template to import grades in bulk. The file is processed in batches in the background.

Auth required: Yes (faculty, admin)

Content type: `multipart/form-data`

Request body

The completed Excel file (`.xlsx`) previously downloaded from the template endpoint.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/academics/grades/upload \
  --header 'Authorization: Bearer <token>' \
  --form 'file=@/path/to/grades_template.xlsx'
```

200 OK

```
{
  "success": true,
  "message": "Upload started",
  "jobId": "job-abc-123"
}
```

Pro-Tip: *Poll `GET /academics/grades/upload/progress` with the returned `jobId` to track processing status.*

—

Check upload progress

`GET /academics/grades/upload/progress`

Polls the status of a background grade upload job.

Auth required: Yes (faculty, admin)

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/academics/grades/upload/progress \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "progress": {
    "status": "processing",
    "percent": 45,
    "processed": 150,
    "total": 330
  }
}
```

Response fields

Current job status. One of `queued`, `processing`, `completed`, `failed`.

Completion percentage (0–100).

Number of records processed so far.

Total number of records in the job.

—

Check publish progress

`GET /academics/grades/publish/progress`

Polls the status of a background grade publishing (email dispatch) job.

Auth required: Yes (director)

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/academics/grades/publish/progress \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "progress": {
    "status": "processing",
    "percent": 72,
    "processed": 240,
    "total": 330
  }
}
```

—

Download grade report

`GET /academics/grades/download/:semesterId`

Downloads the compiled grade report for a semester as a file.

Auth required: Yes (admin)

Path parameters

Semester identifier (e.g. `SEM-1`).

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/academics/grades/download/SEM-1 \
  --header 'Authorization: Bearer <token>' \
  --output grades_report_SEM-1.xlsx
```

Returns a binary Excel file.

—

Publish results via email

`POST /academics/grades/publish-email`

Triggers a background job that emails grade report cards to all students for the given semester.

Auth required: Yes (director only)

Request body

Semester identifier whose results should be published (e.g. `SEM-1`).

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/academics/grades/publish-email \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "semesterId": "SEM-1"
  }'
```

200 OK

```
{
  "success": true,
  "message": "Publishing job started"
}
```

Pro-Tip: *Poll `GET /academics/grades/publish/progress` to monitor email dispatch status.*

All subjects and semester endpoints require authentication. Administrative operations (add, update, delete) require elevated roles as noted per endpoint.

—

List subjects

`GET /academics/subjects`

Returns all subjects configured in the system.

Auth required: Yes

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/academics/subjects \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "subjects": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "code": "E1-SEM-1-CSE-1",
      "name": "Engineering Mathematics I",
      "credits": 4,
      "branch": "CSE",
      "year": "E1",
      "semesterId": "SEM-1"
    }
  ]
}
```

Response fields

UUID of the subject.

Subject code (e.g. `E1-SEM-1-CSE-1`).

Full subject name.

Credit hours.

Branch this subject belongs to (e.g. `CSE`).

Academic year this subject is offered (e.g. `E1`).

Semester identifier (e.g. `SEM-1`).

—

Add a subject

`POST /academics/subjects/add`

Creates a new subject.

Auth required: Yes (dean, director, webmaster)

Request body

Unique subject code (e.g. `E1-SEM-1-CSE-1`).

Full subject name (e.g. `Engineering Mathematics I`).

Number of credit hours.

Branch the subject belongs to (e.g. `CSE`).

Academic year the subject is offered (e.g. `E1`).

Semester the subject is part of (e.g. `SEM-1`).

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/academics/subjects/add \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "code": "E1-SEM-1-CSE-1",
    "name": "Engineering Mathematics I",
    "credits": 4,
    "branch": "CSE",
    "year": "E1",
```

```
    "semesterId": "SEM-1"
  }'
```

200 OK

```
{
  "success": true,
  "message": "Subject added successfully",
  "subject": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "code": "E1-SEM-1-CSE-1",
    "name": "Engineering Mathematics I",
    "credits": 4,
    "branch": "CSE",
    "year": "E1",
    "semesterId": "SEM-1"
  }
}
```

—

Update a subject

`PUT /academics/subjects/:id`

Updates an existing subject by its UUID.

Auth required: Yes (dean, director, webmaster)

Path parameters

UUID of the subject to update.

Request body

All fields are optional. Provide only the fields you want to change.

Updated subject name.

Updated credit hours.

Updated subject code.

Example

```
curl --request PUT \
  --url https://api.uniz.rguktong.in/api/v1/academics/subjects/3fa85f64-5717-4562-b3fc-2
    c963f66afa6 \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "credits": 5
  }'
```

200 OK

```
{
  "success": true,
  "message": "Subject updated successfully"
}
```

—

Delete a subject

`DELETE /academics/subjects/:id`

Permanently removes a subject by its UUID.

Auth required: Yes (director)

Path parameters

UUID of the subject to delete.

Example

```
curl --request DELETE \
  --url https://api.uniz.rguktong.in/api/v1/academics/subjects/3fa85f64-5717-4562-b3fc-2
    c963f66afa6 \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "message": "Subject deleted successfully"
}
```

—

List semesters

`GET /academics/semester`

Returns all semesters in the system.

Auth required: Yes

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/academics/semester \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "semesters": [
    {
      "id": "sem-uuid-001",
      "semesterId": "SEM-1",
      "status": "active",
      "startDate": "2026-01-01T00:00:00.000Z",
      "endDate": "2026-06-30T00:00:00.000Z"
    }
  ]
}
```

Response fields

Internal UUID of the semester record.

Human-readable semester label (e.g. `SEM-1`).

Current status. One of `active`, `inactive`, `archived`.

ISO 8601 start date.

ISO 8601 end date.

—

Initialize a semester

`POST /academics/semester/init`

Creates and activates a new semester.

Auth required: Yes (director, dean)

Request body

Identifier for the new semester (e.g. `SEM-2`).

ISO 8601 start date (e.g. `2026-07-01`).

ISO 8601 end date (e.g. `2026-12-31`).

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/academics/semester/init \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "semesterId": "SEM-2",
    "startDate": "2026-07-01",
    "endDate": "2026-12-31"
  }'
```

200 OK

```
{
  "success": true,
  "message": "Semester initialized",
```

```
  "semester": {
    "id": "sem-uuid-002",
    "semesterId": "SEM-2",
    "status": "active",
    "startDate": "2026-07-01T00:00:00.000Z",
    "endDate": "2026-12-31T00:00:00.000Z"
  }
}
```

—

Update semester status

`PATCH /academics/semester/status/:id`

Changes the status of an existing semester (e.g. mark it as inactive or archived).

Auth required: Yes (director, dean)

Path parameters

UUID of the semester record to update.

Request body

New status. One of `active`, `inactive`, `archived`.

Example

```
curl --request PATCH \
  --url https://api.uniz.rguktong.in/api/v1/academics/semester/status/sem-uuid-001 \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "status": "inactive"
  }'
```

200 OK

```
{
  "success": true,
  "message": "Semester status updated"
}
```

—

Semester overview

`GET /academics/semester/overview`

Returns a high-level overview of the current semester, including registration counts and grade publication status.

Auth required: Yes (dean, director, webmaster)

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/academics/semester/overview \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "overview": {
    "activeSemester": "SEM-1",
    "totalRegistrations": 420,
    "totalSubjects": 12,
    "gradesPublished": false,
    "attendancePublished": false
  }
}
```

—

Available subjects for registration

`GET /academics/student/available`

Returns the list of subjects available for the authenticated student to register in the current semester.

Auth required: Yes (student)

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/academics/student/available \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "subjects": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "code": "E3-SEM-1-CSE-1",
      "name": "Operating Systems",
      "credits": 4
    }
  ]
}
```

—

Register for subjects

`POST /academics/student/register`

Registers the authenticated student for one or more subjects in the current semester.

Auth required: Yes (student)

Request body

Array of subject UUIDs to register for.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/academics/student/register \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "subjectIds": [
      "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "7cb96g75-6828-5673-c4gd-3d074g77bgb7"
    ]
  }'
```

200 OK

```
{
  "success": true,
  "message": "Subjects registered successfully",
  "registered": 2
}
```

Error responses

Status	Meaning
`400 Bad Request`	One or more subject IDs are invalid or the registration window is closed.
`409 Conflict`	Student is already registered for one or more of the provided subjects.

—

Get current registered subjects

`GET /academics/student/current/:studentId`

Returns the subjects a student is currently registered for in the active semester.

Auth required: Yes

Path parameters

Student roll number (e.g. `O210008`). Students may only query their own record; admin roles may query any student.

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/academics/student/current/O210008 \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "semesterId": "SEM-1",
  "subjects": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "code": "E3-SEM-1-CSE-1",
      "name": "Operating Systems",
      "credits": 4
    }
  ]
}
```

Response fields

The active semester these registrations belong to.

List of currently registered subjects.

All login endpoints are **unauthenticated**. On success, you receive a `token` that must be sent as `Authorization: Bearer` on every subsequent request.

Warning: *Login endpoints are rate-limited per IP. Repeated failed attempts will result in a `429 Too Many Requests` response.*

—

General login

`POST /auth/login`

Authenticates any user — student or admin — and returns a JWT with the caller's role. Use the role-specific endpoints (`/auth/login/student`, `/auth/login/admin`) when you need to restrict which account types are accepted.

Auth required: No

Request body

The student roll number (e.g. `O210008`) or admin username (e.g. `director`).

The account password.

Response

`true` on successful authentication.

Signed JWT. Pass this value in the `Authorization: Bearer` header for all authenticated requests.

The role assigned to this account. Possible values: `student`, `director`, `dean`, `warden_male`, `warden_female`, `caretaker_male`, `caretaker_female`, `security`, `webmaster`.

The username that was authenticated.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/auth/login \
  --header 'Content-Type: application/json' \
  --data '{
    "username": "O210008",
    "password": "password123"
  }'
```

200 OK — student

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "role": "student",
  "username": "O210008"
}
```

200 OK — admin

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "role": "director",
  "username": "director"
}
```

Error responses

Status	Meaning
`401 Unauthorized`	Incorrect password.
`403 Forbidden`	Account is suspended.
`429 Too Many Requests`	Rate limit exceeded for this IP.

—

Student login

`POST /auth/login/student`

Identical to general login but rejects credentials that do not belong to a `student` role. Use this endpoint in student-facing clients to prevent admins from signing in through the student app.

Auth required: No

Request body

Student roll number (e.g. `O210008`).
The student's password.
Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/auth/login/student \
  --header 'Content-Type: application/json' \
  --data '{
    "username": "O210008",
    "password": "password123"
  }'
```

200 OK

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "role": "student",
  "username": "O210008"
}
```

Error responses
Status	Meaning
`401 Unauthorized`	Incorrect password.
`403 Forbidden`	Account is suspended, or the account belongs to a non-student role.
`429 Too Many Requests`	Rate limit exceeded for this IP.

Admin login
`POST /auth/login/admin`
Authenticates administrative accounts only. Rejects student credentials.
Auth required: No
Request body
Admin username (e.g. `director`, `dean_cse`, `warden_male`, `caretaker_female`, `security_admin`).
The admin's password.
Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/auth/login/admin \
  --header 'Content-Type: application/json' \
  --data '{
    "username": "director",
    "password": "director@uniz"
  }'
```

200 OK

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "role": "director",
  "username": "director"
}
```

Error responses
Status	Meaning
`401 Unauthorized`	Incorrect password.
`403 Forbidden`	Account is suspended, or the account belongs to a student role.
`429 Too Many Requests`	Rate limit exceeded for this IP.

Logout
`POST /auth/logout`
Clears the server-side session. The client should also discard the stored JWT.
Auth required: No
Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/auth/logout \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true
}
```

OTP verification is the first step of the password reset flow. The full flow is:

Request an OTP Call `POST /auth/otp/request` (push notification) or `POST /auth/otp/request-email` (email) with the account username.

Receive the code The user receives a 6-digit code via the chosen channel.

Verify the OTP Call `POST /auth/otp/verify` with the username and the 6-digit code. On success you receive a `resetToken`.

Reset the password Pass the `resetToken` to `POST /auth/password/reset` along with the new password. See the Password page for details.

Warning: *OTP endpoints are rate-limited. Each account is subject to an hourly limit that applies across both channels, a separate per-request limit for push notifications, and a separate per-request limit for email. Exceeding any limit returns `429 Too Many Requests`.*

—

Request OTP via push notification

`POST /auth/otp/request`

Sends a 6-digit OTP to the device registered for push notifications on the account.

Auth required: No

Rate limits: hourly limit (shared with email channel) + per-push limit

Request body

The student roll number or admin username that needs to reset their password.

Response

`true` when the OTP was dispatched successfully.

Human-readable confirmation that the OTP was sent.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/auth/otp/request \
  --header 'Content-Type: application/json' \
  --data '{
    "username": "O210008"
  }'
```

200 OK

```
{
  "success": true,
  "message": "OTP sent via push notification"
}
```

Error responses

Status	Meaning
`404 Not Found`	No account found for the given username.
`429 Too Many Requests`	Hourly or per-push rate limit exceeded.

—

Request OTP via email

`POST /auth/otp/request-email`

Sends a 6-digit OTP to the email address registered on the account.

Auth required: No

Rate limits: hourly limit (shared with push channel) + per-email limit

Request body

The student roll number or admin username that needs to reset their password.

Response

`true` when the OTP email was queued successfully.

Human-readable confirmation that the OTP was sent.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/auth/otp/request-email \
  --header 'Content-Type: application/json' \
  --data '{
    "username": "O210008"
  }'
```

200 OK

```
{
  "success": true,
  "message": "OTP sent to registered email"
}
```

Error responses

Status	Meaning
`404 Not Found`	No account found for the given username.
`429 Too Many Requests`	Hourly or per-email rate limit exceeded.

Verify OTP

`POST /auth/otp/verify`

Validates the 6-digit code and, if correct, returns a short-lived `resetToken`. You must pass this token to `POST /auth/password/reset` to complete the password reset.

Auth required: No

Request body

The username that the OTP was issued for.

The 6-digit one-time password received via push or email. Must be exactly 6 characters.

Response

`true` when the OTP was accepted.

A short-lived token authorising a single password reset. Pass this to `POST /auth/password/reset`.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/auth/otp/verify \
  --header 'Content-Type: application/json' \
  --data '{
    "username": "O210008",
    "otp": "482910"
  }'
```

200 OK

```
{
  "success": true,
  "resetToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Error responses

Status	Meaning
`400 Bad Request`	OTP is invalid or has expired.
`404 Not Found`	No account found for the given username.
`429 Too Many Requests`	Rate limit exceeded.

UniZ provides two password management endpoints:

- **Reset** — for users who have forgotten their password. Requires a `resetToken` obtained through the OTP verification flow. No active session needed. - **Change** — for logged-in users who want to update their current password. Requires a valid Bearer token.

Note: *All new passwords must be at least **6 characters** long.*

—

Reset password

`POST /auth/password/reset`

Sets a new password using the `resetToken` issued by `POST /auth/otp/verify`. This endpoint does not require an active session.

Auth required: No

Request body

The username whose password is being reset.

The short-lived token returned by `POST /auth/otp/verify`. Tokens are single-use and expire shortly after they are issued.

The new password. Minimum 6 characters.

Response

`true` when the password was updated successfully.

Human-readable confirmation.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/auth/password/reset \
  --header 'Content-Type: application/json' \
  --data '{
    "username": "O210008",
    "resetToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "newPassword": "newSecurePassword123"
  }'
```

200 OK

```
{
  "success": true,
  "message": "Password reset successfully"
}
```

Error responses

| Status | Meaning | |———|———| | `400 Bad Request` | `resetToken` is invalid, expired, or already used. New password does not meet the minimum length requirement. | | `404 Not Found` | No account found for the given username. | | `429 Too Many Requests` | Rate limit exceeded. |

—

Change password

`POST /auth/password/change`

Updates the password for the currently authenticated user. The caller must supply their existing password to confirm the change.

Auth required: Yes — `Authorization: Bearer`

Request body

The user's current password.

The desired new password. Minimum 6 characters.

Response

`true` when the password was changed successfully.

Human-readable confirmation.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/auth/password/change \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "currentPassword": "oldPassword123",
    "newPassword": "newSecurePassword456"
  }'
```

200 OK

```
{
  "success": true,
  "message": "Password changed successfully"
}
```

Error responses

Status	Meaning
`400 Bad Request`	`currentPassword` is incorrect. New password does not meet the minimum length requirement.
`401 Unauthorized`	Missing or invalid Bearer token.

Requests flow through a sequential approval chain. Each role can only act at the level assigned to them. Approving a request moves it to the next level; rejecting it ends the chain.

Note: *All endpoints on this page require `Authorization: Bearer` in the request header.*

Approval chain

Caretaker First to review. Confirms the student's family is informed. Approving moves the request to the warden.

Warden Second review. Checks the student's academic schedule and disciplinary record. Approving moves the request to the SWO.

SWO (Student Welfare Officer) Third review. Approving forwards the request to the dean or director.

Dean / Director Final authority. Their approval sets `isApproved: true` and allows the student to exit campus.

Once the dean or director approves, security can use the check-out endpoint to record the student leaving, and the check-in endpoint when they return.

—

Approve a request

`POST /requests/:id/approve`

Advances the request to the next approval level. The action taken depends on the caller's role:

| Caller role | Effect | |————————-|———| | `caretaker_male` / `caretaker_female` | Moves request from `caretaker` $\{}rightarrow$ `warden` level. | | `warden_male` / `warden_female` | Moves request from `warden` $\{}rightarrow$ `swo` level. | | `swo` | Moves request from `swo` $\{}rightarrow$ `dean/director` level. | | `dean` / `director` | Sets `isApproved: true`. Final approval granted. |

`POST /requests/:id/forward` is an alias for this endpoint.

Auth required: Yes (caretaker, warden, swo, dean, director)

Path parameters

The unique identifier of the outpass or outing request.

Request body

Optional note from the approver. Shown to the student and subsequent approvers.

Response

`true` when the approval was recorded.

Request identifier.

Updated approval level after this action.

`true` if the dean or director has now given final approval.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/requests/req-789/approve \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "comment": "Parent verified via call."
  }'
```

200 OK

```
{
  "success": true,
  "data": {
    "id": "req-789",
    "currentLevel": "warden",
    "isApproved": false
  }
```

```
}
```

Error responses

| Status | Meaning | |———|———| | `403 Forbidden` | Your role is not permitted to approve at this request's current level. | | `404 Not Found` | No request found with the given `id`. |

—

Reject a request

`POST /requests/:id/reject`

Rejects the request at the current level and ends the approval chain. The student's request will show `status: "rejected"` in their request history.

Auth required: Yes (caretaker, warden, swo, dean, director)

Path parameters

The unique identifier of the request to reject.

Request body

The reason for rejection. Shown to the student.

Response

`true` when the rejection was recorded.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/requests/req-789/reject \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "comment": "Invalid reason provided."
  }'
```

200 OK

```
{
  "success": true
}
```

—

Check out a student

`POST /requests/:id/checkout`

Records that a student has physically left campus. Call this after the request is fully approved and the security guard verifies the student at the gate.

Auth required: Yes (security)

Path parameters

The unique identifier of the approved outpass or outing request.

Response

`true` when the checkout was recorded.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/requests/req-789/checkout \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true
}
```

—

Check in a student

`POST /requests/:id/checkin`

Records that a student has returned to campus. Call this when the student arrives back at the gate after an approved outpass or outing.

Auth required: Yes (security)
Path parameters
The unique identifier of the request associated with this student's exit.
Response
`true` when the check-in was recorded.
Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/requests/req-789/checkin \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true
}
```

—
Get security summary
`GET /requests/security/summary`
Returns a summary of students currently outside campus — those who have checked out but not yet checked back in. Use this for a gate-level overview or end-of-day report.
Auth required: Yes (security, director, admin)
Response
`true` on success.
Total number of students currently outside campus.
Array of students currently outside.
Student roll number.
Student full name.
ISO 8601 timestamp of when the student exited.
ISO 8601 timestamp of the approved return time.
Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/requests/security/summary \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "summary": {
    "totalOut": 2,
    "students": [
      {
        "studentId": "O210008",
        "name": "DESU SABER",
        "checkedOutAt": "2026-02-10T17:05:00Z",
        "expectedReturn": "2026-02-10T20:00:00Z"
      }
    ]
  }
}
```

An outing is a short-duration, same-day exit request. Unlike an outpass — which covers multi-day absences and requires a full approval chain — an outing covers a brief window on a single day with a simpler workflow.

| Feature | Outpass | Outing | |———|———|———| | Duration | Multi-day | Same day, short window | | Approval chain | Caretaker $\{ }rightarrow$ Warden $\{ }rightarrow$ SWO $\{ }rightarrow$ Dean/Director | Simplified | | Use case | Home visits, medical trips | Groceries, errands, brief outings |

Note: *All endpoints on this page require `Authorization: Bearer` in the request header.*

Warning: *The submission endpoint is rate-limited. If you exceed the limit you will receive a `429 Too Many Requests` response. Wait before retrying.*

—

Submit an outing request

`POST /requests/outing`

Creates a new outing request for the authenticated student.

Auth required: Yes (student)

Request body

The reason for the outing (e.g. `"Buying groceries"`).

Start time as an ISO 8601 datetime string (e.g. `2026-02-10T17:00:00Z`). Must be on the same calendar day as `toTime`.

End time as an ISO 8601 datetime string (e.g. `2026-02-10T20:00:00Z`). Must be on the same calendar day as `fromTime`.

Response

`true` when the request was created successfully.

Unique identifier for the outing request.

Initial status of the request. Typically `"pending"` immediately after creation.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/requests/outing \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "reason": "Buying groceries",
    "fromTime": "2026-02-10T17:00:00Z",
    "toTime": "2026-02-10T20:00:00Z"
  }'
```

200 OK

```
{
  "success": true,
  "data": {
    "id": "out-456",
    "status": "pending"
  }
}
```

Error responses

| Status | Meaning | |———|———| | `400 Bad Request` | Missing or invalid request body fields. | | `401 Unauthorized` | Missing or invalid token. | | `429 Too Many Requests` | Submission rate limit exceeded. |

—

List all outings

`GET /requests/outing/all`

Returns all outing requests in the system. Security personnel use this endpoint to verify students at the gate before allowing them to exit.

Auth required: Yes (security, director, admin)

Note: *Security guards use this endpoint to look up a specific student's approved outing before permitting exit. Cross-reference the student's ID and the current time against* `fromTime` *and* `toTime`*.*

Response

`true` on success.

Array of outing request objects.

Unique outing request identifier.

Roll number of the requesting student.

The stated reason for the outing.

Approved exit start time (ISO 8601).

Expected return time (ISO 8601).

Current status: `"pending"`, `"approved"`, or `"rejected"`.

ISO 8601 timestamp of when the request was submitted.

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/requests/outing/all \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "outings": [
    {
      "id": "out-456",
      "studentId": "O210008",
      "reason": "Buying groceries",
      "fromTime": "2026-02-10T17:00:00Z",
      "toTime": "2026-02-10T20:00:00Z",
      "status": "approved",
      "requested_time": "2026-02-10T14:30:00Z"
    }
  ]
}
```

Error responses

| Status | Meaning | |———|———| | `401 Unauthorized` | Missing or invalid token. | | `403 Forbidden` | The authenticated role does not have access to this endpoint. |

An outpass is a multi-day leave request. After you submit one, it moves through a chain of approvers before the student is permitted to exit. Use the Approvals endpoints to advance or reject a request at each level.

Note: *All endpoints on this page require `Authorization: Bearer` in the request header.*

Warning: *Submission endpoints are rate-limited. If you exceed the limit you will receive a `429 Too Many Requests` response. Wait before retrying.*

—

Submit an outpass request

`POST /requests/outpass`

Creates a new outpass request for the authenticated student. The request enters the approval queue at the `caretaker` level.

Auth required: Yes (student)

Request body

The reason for the leave. Shown to each approver in the chain.

Start of the leave period as an ISO 8601 datetime string (e.g. `2026-02-10T09:00:00Z`).

End of the leave period as an ISO 8601 datetime string (e.g. `2026-02-15T18:00:00Z`).

The student's gender: `"M"` or `"F"`. Optional — the server derives this from the student's profile when omitted.

Response

`true` when the request was created successfully.

Unique identifier for the outpass request.

The approval level the request is waiting at. See approval levels below.

`true` only when the dean or director has given final approval.

ISO 8601 timestamp of when the request was created.

Approval levels

The `currentLevel` field tells you which role must act next:

Value	Meaning
——-	———
`caretaker`	Awaiting first-level approval from the caretaker.
`warden`	Caretaker approved; awaiting warden review.
`swo`	Warden approved; awaiting Student Welfare Officer review.
`dean` / `director`	SWO approved; awaiting final approval.
`approved`	Dean or director granted final approval.
`rejected`	Rejected at some level in the chain.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/requests/outpass \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "reason": "Emergency visit to home due to health issues",
    "fromDay": "2026-02-10T09:00:00Z",
    "toDay": "2026-02-15T18:00:00Z",
    "studentGender": "M"
  }'
```

200 OK

```
{
  "success": true,
  "data": {
    "id": "req-789",
    "currentLevel": "caretaker",
    "isApproved": false,
    "createdAt": "2026-01-31T10:00:00Z"
  }
}
```

Error responses

| Status | Meaning | |———|———|| `400 Bad Request` | Missing or invalid request body fields. | | `401 Unauthorized` | Missing or invalid token. || `429 Too Many Requests` | Submission rate limit exceeded. |

—

List all outpasses

`GET /requests/outpass/all`

Returns all outpass requests in the system. Intended for caretakers, wardens, and administrators to view the full queue.

Auth required: Yes (caretaker, warden, dean, director, admin)

Note: *The response includes all requests regardless of `currentLevel`. Your client should filter by `currentLevel` to show only the requests relevant to the current user's role (e.g. a caretaker sees only `currentLevel: "caretaker"` entries).*

Response

`true` on success.

Array of outpass request objects.

Unique request identifier.

The roll number of the student who submitted the request.

`"M"` or `"F"`.

The stated reason for the leave.

The approval level the request is currently waiting at.

ISO 8601 timestamp of when the request was submitted.

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/requests/outpass/all \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "outpasses": [
    {
      "id": "req-123",
      "studentId": "O210008",
      "studentGender": "M",
      "reason": "Home visit",
      "currentLevel": "caretaker",
      "requested_time": "2026-02-10T10:00:00Z"
    }
  ]
}
```

—

Get request history

`GET /requests/history`

Returns the authenticated student's own outpass and outing request history, newest first.

Auth required: Yes (student)

Response

`true` on success.

Array of past requests.

Unique request identifier.

`"outpass"` or `"outing"`.

Current status of the request: `"pending"`, `"approved"`, or `"rejected"`.

The reason provided when submitting.

ISO 8601 timestamp of submission.

Current page number.

Total number of requests across all pages.

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/requests/history \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "history": [
    {
      "_id": "req-123",
      "type": "outpass",
      "status": "pending",
      "reason": "Home visit",
      "requested_time": "2026-02-10T10:00:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "total": 5
  }
}
```

—

List students currently outside

`GET /requests/outside`

Returns the list of students who currently have an approved outpass and have not yet checked back in. Intended for administrators and security personnel.

Auth required: Yes (admin, security, director)

Response

`true` on success.

Array of student records currently outside campus.

The roll number of the student.

Full name of the student.

Reason stated on the approved outpass.

Approved leave start date.

Approved leave end date.

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/requests/outside \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "students": [
    {
      "studentId": "O210008",
      "name": "DESU SABER",
      "reason": "Home visit",
      "fromDay": "2026-02-10T09:00:00Z",
      "toDay": "2026-02-15T18:00:00Z"
    }
  ]
}
```

Note: *All endpoints on this page require* `Authorization: Bearer` *in the request header unless noted otherwise.*

—

Get your admin profile

`GET /admin/me`

Returns the profile of the authenticated admin user.

Auth required: Yes (any admin role)

Response

`true` on success.

Admin username.

Full name.

Admin email address.

System role (e.g. `"director"`, `"dean"`, `"warden_male"`, `"security"`, `"webmaster"`).

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/admin/me \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "admin": {
    "username": "director",
    "name": "Director Name",
    "email": "director@rguktong.ac.in",
    "role": "director"
  }
}
```

—

Update your admin profile

`PUT /admin/me/update`

Updates the authenticated admin's own profile. All fields are optional.

Auth required: Yes (any admin role)

Request body

Full name.

Email address.

Example

```
curl --request PUT \
  --url https://api.uniz.rguktong.in/api/v1/admin/me/update \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "email": "new.director@rguktong.ac.in"
  }'
```

200 OK

```
{
  "success": true
}
```

—

List available batches

`GET /admin/batches`

Returns all available student batches (combinations of branch and year) in the system. Use this to populate dropdowns when uploading grades or filtering students.

Auth required: Yes (admin, director, dean)

Response

`true` on success.

Array of batch objects.

Branch name (e.g. `"CSE"`).

Academic year (e.g. `"E1"`, `"E2"`, `"E3"`, `"E4"`).

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/admin/batches \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "batches": [
    { "branch": "CSE", "year": "E1" },
    { "branch": "CSE", "year": "E2" },
    { "branch": "ECE", "year": "E1" }
  ]
}
```

—

Download student import template

`GET /admin/student/template`

Downloads an Excel template pre-populated with the expected column structure for bulk student imports.

Auth required: Yes (admin, director)

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/admin/student/template \
  --header 'Authorization: Bearer <token>' \
  --output students_template.xlsx
```

The response is an Excel file (`.xlsx`). Open it, fill in student data, and upload it using the bulk upload endpoint.

—

Bulk upload students

`POST /admin/student/upload`

Imports students in bulk from an Excel file. The request must be sent as `multipart/form-data`.

Auth required: Yes (admin, director)

Request body

The Excel file (`.xlsx`) containing student data. Use the template to ensure correct column formatting.

Response

`true` when the file was accepted and processing has started.

Identifier for the background processing job. Use this with the upload progress endpoint to track completion.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/admin/student/upload \
  --header 'Authorization: Bearer <token>' \
  --form 'file=@students.xlsx'
```

200 OK

```
{
  "success": true,
  "jobId": "job-abc123"
}
```

—

Check upload progress

`GET /admin/student/upload/progress`

Returns the status of an ongoing or completed student bulk upload job.

Auth required: Yes (admin, director)

Response

`true` on success.

Current status of the job: `"queued"`, `"processing"`, or `"completed"`.

Percentage of records processed (0–100).

Number of student records processed so far.

Total number of student records in the file.

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/admin/student/upload/progress \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "progress": {
    "status": "processing",
    "percent": 45,
    "processed": 150,
    "total": 330
  }
}
```

—

Export student data

`GET /admin/student/export`

Exports student records as a downloadable file. Use query parameters to filter the export to a specific branch, year, or other criteria.

Auth required: Yes (admin, director, dean)

Query parameters

Filter by branch (e.g. `CSE`).

Filter by academic year (e.g. `E2`).

Example

```
curl --request GET \
  --url 'https://api.uniz.rguktong.in/api/v1/admin/student/export?branch=CSE&year=E2' \
  --header 'Authorization: Bearer <token>' \
  --output students_cse_e2.xlsx
```

The response is an Excel file containing the filtered student records.

—

List all banners

`GET /profile/admin/banners`

Returns all banners, both published and unpublished. Cached for performance.

Auth required: Yes (any authenticated role)

Note: *This endpoint returns all banners. Student-facing clients should filter by `isPublished: true` before displaying banners to students.*

Response

`true` on success.

Array of banner objects.

Unique banner identifier.

Banner headline.

Banner body text.
URL of the banner image.
`true` if the banner is visible to all users.
ISO 8601 timestamp of when the banner was created.
Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/profile/admin/banners \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "banners": [
    {
      "id": "ban-123",
      "title": "Result Day Announcement",
      "text": "Results will be out at 5 PM.",
      "imageUrl": "https://example.com/banners/result-day.jpg",
      "isPublished": true,
      "createdAt": "2026-01-30T10:00:00Z"
    }
  ]
}
```

—
Create a banner
`POST /profile/admin/banners`
Creates a new banner. Newly created banners are unpublished by default. Use the publish endpoint to make a banner visible to all users.
Auth required: Yes (webmaster)
Request body
Banner headline (e.g. `"Holiday Notice"`).
Banner body text.
URL of an image to display with the banner.
Response
`true` when the banner was created.
Unique identifier of the new banner.
Always `false` on creation. Publish separately.
Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/profile/admin/banners \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "title": "Holiday Notice",
    "text": "College is closed tomorrow for a public holiday.",
    "imageUrl": "https://example.com/banners/holiday.jpg"
  }'
```

200 OK

```
{
  "success": true,
  "banner": {
    "id": "ban-456",
    "isPublished": false
  }
}
```

—
Publish or unpublish a banner

`POST /profile/admin/banners/:id/publish`

Toggles the published state of a banner. Set `publish: true` to make it visible to all users, or `publish: false` to hide it.

Auth required: Yes (webmaster)

Path parameters

The unique identifier of the banner.

Request body

`true` to publish the banner, `false` to unpublish it.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/profile/admin/banners/ban-456/publish \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "publish": true
  }'
```

200 OK

```
{
  "success": true
}
```

—

System health check

`GET /system/health`

Returns the real-time health status and latency of all services in the UniZ platform. Use this to diagnose service degradation or verify that the platform is fully operational.

Auth required: No

Response

`true` when the health check completed.

Array of service status objects.

Human-readable service name (e.g. `"Auth Service"`, `"Mail Service"`).

Current status: `"UP"` or `"DOWN"`.

Response time measured during the health probe (e.g. `"120ms"`).

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/system/health
```

200 OK

```
{
  "success": true,
  "services": [
    { "name": "Auth Service", "status": "UP", "latency": "120ms" },
    { "name": "Mail Service", "status": "UP", "latency": "45ms" },
    { "name": "Outpass Service", "status": "UP", "latency": "88ms" },
    { "name": "User Service", "status": "UP", "latency": "60ms" }
  ]
}
```

Note: *All endpoints on this page require* `Authorization: Bearer` *in the request header.*

—

Get your faculty profile

`GET /profile/faculty/me`

Returns the profile of the authenticated faculty member.

Auth required: Yes (faculty)

Response

`true` on success.

Unique faculty identifier.

Full name of the faculty member.

Institutional email address.

Department the faculty member belongs to (e.g. `"CSE"`).

Job title (e.g. `"Assistant Professor"`).

System role assigned to the account (e.g. `"faculty"`, `"dean"`).

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/profile/faculty/me \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "faculty": {
    "username": "faculty_cse_01",
    "name": "Dr. A. Sharma",
    "email": "a.sharma@rguktong.ac.in",
    "department": "CSE",
    "designation": "Assistant Professor",
    "role": "faculty"
  }
}
```

—

Update your faculty profile

`PUT /faculty/me/update`

Updates the authenticated faculty member's own profile. All fields are optional.

Auth required: Yes (faculty)

Request body

Full name.

Email address.

Department name.

Job title.

Example

```
curl --request PUT \
  --url https://api.uniz.rguktong.in/api/v1/faculty/me/update \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "designation": "Associate Professor"
  }'
```

200 OK

```
{
  "success": true
}
```

—

Get a faculty member (admin)

`GET /admin/faculty/:username`

Returns the full profile of any faculty member. Only accessible to administrators.

Auth required: Yes (admin, director, dean)

Path parameters

The faculty member's username.

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/admin/faculty/faculty_cse_01 \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "faculty": {
    "username": "faculty_cse_01",
    "name": "Dr. A. Sharma",
    "email": "a.sharma@rguktong.ac.in",
    "department": "CSE",
    "designation": "Assistant Professor",
    "role": "faculty"
  }
}
```

—

Update a faculty member (admin)

`PUT /admin/faculty/:username`

Updates any faculty member's profile. Only accessible to administrators.

Auth required: Yes (admin, director)

Path parameters

The username of the faculty member to update.

Request body

Full name.

Email address.

Department name.

Job title.

System role to assign (e.g. `"faculty"`, `"dean"`, `"warden_male"`).

Example

```
curl --request PUT \
  --url https://api.uniz.rguktong.in/api/v1/admin/faculty/faculty_cse_01 \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "designation": "Associate Professor",
    "role": "dean"
  }'
```

200 OK

```
{
  "success": true
}
```

—

Delete a faculty member (admin)

`DELETE /admin/faculty/:username`

Permanently removes a faculty member's account.

Auth required: Yes (admin, director)

Warning: *This action is irreversible. The faculty member will lose access to the system immediately.*

Path parameters

The username of the faculty member to remove.

Example

```
curl --request DELETE \
  --url https://api.uniz.rguktong.in/api/v1/admin/faculty/faculty_cse_01 \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true
}
```

—

Search faculty members

`POST /faculty/search`

Searches faculty members with optional filters. Returns matching results.

Auth required: Yes (admin, director, dean)

Request body

Partial name to search by.

Filter by department (e.g. `"CSE"`).

Filter by designation.

Filter by system role.

Page number. Defaults to `1`.

Number of results per page. Defaults to `10`.

Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/faculty/search \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "department": "CSE",
    "page": 1,
    "limit": 10
  }'
```

200 OK

```
{
  "success": true,
  "faculty": [
    {
      "username": "faculty_cse_01",
      "name": "Dr. A. Sharma",
      "department": "CSE",
      "designation": "Assistant Professor",
      "role": "faculty"
    }
  ],
  "pagination": {
    "page": 1,
    "totalPages": 2,
    "total": 18
  }
}
```

—

Bulk create faculty (admin)

`POST /admin/faculty/bulk-create`

Creates multiple faculty accounts in a single request. Each object in the array must include the required fields.

Auth required: Yes (admin, director)
Request body
The request body is an array of faculty objects.
Unique username for the faculty member.
Full name.
Institutional email address.
Department name.
Job title.
System role to assign. Defaults to `"faculty"` if omitted.
Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/admin/faculty/bulk-create \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '[
    {
      "username": "faculty_ece_01",
      "name": "Dr. B. Rao",
      "email": "b.rao@rguktong.ac.in",
      "department": "ECE",
      "designation": "Associate Professor",
      "role": "faculty"
    },
    {
      "username": "faculty_ece_02",
      "name": "Prof. C. Reddy",
      "email": "c.reddy@rguktong.ac.in",
      "department": "ECE",
      "designation": "Assistant Professor"
    }
  ]'
```

200 OK

```
{
  "success": true,
  "created": 2,
  "failed": 0,
  "errors": []
}
```

Error responses
| Status | Meaning | |———|———| | `400 Bad Request` | One or more entries are missing required fields. | | `403 Forbidden` | Your role does not have permission to create faculty accounts. | | `404 Not Found` | Faculty member not found. |

Note: *All endpoints on this page require* `Authorization: Bearer` *in the request header.*

—

Get your profile

`GET /profile/student/me`

Returns the profile of the authenticated student.

Auth required: Yes (student)

Response

`true` on success.

Internal unique identifier.

The student's roll number (e.g. `O210008`).

Full name of the student.

Current academic year (e.g. `"3"`).

Branch of study (e.g. `"CSE"`).

URL of the student's profile photo.

`true` if the student has an outpass or outing request awaiting approval.

`true` if the student is currently recorded as present on campus.

Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/profile/student/me \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "student": {
    "_id": "550e8400-e29b-41d4-a716-446655440000",
    "username": "O210008",
    "name": "DESU SABER",
    "year": "3",
    "branch": "CSE",
    "profile_url": "https://example.com/photos/O210008.jpg",
    "has_pending_requests": false,
    "is_in_campus": true
  }
}
```

—

Update your profile

`PUT /profile/student/update`

Updates the authenticated student's own profile fields. All fields are optional; only the fields you include will be changed.

Auth required: Yes (student)

Request body

Student's phone number (10–15 digits).

Student's home address.

Blood group (e.g. `"O+"`).

Date of birth in ISO 8601 format.

URL of the student's profile photo.

Father's full name.

Mother's full name.

Father's occupation.

Mother's occupation.

Father's email address.

Mother's email address.

Father's address.

Mother's address.
Hostel room number.
Example

```
curl --request PUT \
  --url https://api.uniz.rguktong.in/api/v1/profile/student/update \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "phone": "9876543210",
    "address": "New Address, Ongole"
  }'
```

200 OK

```
{
  "success": true
}
```

—
Update campus presence status
`PUT /profile/student/status`
Sets whether a student is currently present on campus. Security personnel use this when a student exits or enters at the gate. Administrators and wardens may also use it for manual corrections.
Auth required: Yes (security, warden, director)
Request body
The roll number of the student whose status you are updating.
Set to `false` when the student leaves campus; `true` when they return.
Example

```
curl --request PUT \
  --url https://api.uniz.rguktong.in/api/v1/profile/student/status \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "username": "O210008",
    "isPresent": false
  }'
```

200 OK

```
{
  "success": true
}
```

—
Search students
`POST /profile/student/search`
Searches students with optional filters. Returns paginated results. Administrators, directors, and security personnel use this endpoint for lookups and gate-level reporting (e.g. find all students currently outside campus).
Auth required: Yes (admin, director, dean, security)
Request body
Partial roll number to search by (e.g. `"O21"` matches all roll numbers starting with `O21`).
Filter by branch (e.g. `"CSE"`).
Filter by academic year (e.g. `"3"`).
Filter by campus presence. Set to `false` to find all students currently outside.
Page number for pagination. Defaults to `1`.
Number of results per page. Defaults to `10`. Use a higher value (e.g. `50`) to fetch larger batches.
Response
`true` on success.

Array of matching student objects.
Student roll number.
Student full name.
Branch of study.
Current academic year.
Whether the student is currently on campus.
Current page number.
Total number of pages.
Total number of matching students.
Example

```
curl --request POST \
  --url https://api.uniz.rguktong.in/api/v1/profile/student/search \
  --header 'Authorization: Bearer <token>' \
  --header 'Content-Type: application/json' \
  --data '{
    "username": "O21",
    "branch": "CSE",
    "page": 1,
    "limit": 10
  }'
```

200 OK

```
{
  "success": true,
  "students": [
    {
      "username": "O210008",
      "name": "DESU SABER",
      "branch": "CSE",
      "year": "3",
      "is_in_campus": true
    }
  ],
  "pagination": {
    "page": 1,
    "totalPages": 5,
    "total": 50
  }
}
```

—
Get a specific student profile (admin)
`GET /admin/student/:username`
Returns the full profile of any student by their roll number. Only accessible to administrators and directors.
Auth required: Yes (admin, director, dean)
Path parameters
The student's roll number (e.g. `O210008`).
Example

```
curl --request GET \
  --url https://api.uniz.rguktong.in/api/v1/admin/student/O210008 \
  --header 'Authorization: Bearer <token>'
```

200 OK

```
{
  "success": true,
  "student": {
    "_id": "550e8400-e29b-41d4-a716-446655440000",
    "username": "O210008",
    "name": "DESU SABER",
    "year": "3",
    "branch": "CSE",
```

```
    "profile_url": "https://example.com/photos/O210008.jpg",
    "has_pending_requests": false,
    "is_in_campus": true
  }
}
```

Error responses

Status	Meaning
`403 Forbidden`	Your role does not permit access to other students' profiles.
`404 Not Found`	No student found with the given roll number.

2 Technical Implementation

The following subsections document the critical microservices architecture, including data schemas and core controllers.

2.1 Component: Uniz Academics

apps/uniz-academics/prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
  binaryTargets = ["native", "rhel-openssl-3.0.x", "linux-musl-arm64-openssl-3.0.x"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Subject {
  id                String             @id
  code              String             @unique
  name              String
  credits           Float
  department        String
  semester          String
  attendance        Attendance[]
  grades            Grade[]
  branchAllocations   BranchAllocation[]
  registrations       Registration[]
  seatingArrangements SeatingArrangement[]
}

model Grade {
  id            String   @id @default(uuid()) @db.Uuid
  studentId     String
  subjectId     String
  semesterId    String
  attemptNumber Int      @default(1)
  grade         Float
  batch         String? // e.g. "O21"
  isRemedial    Boolean  @default(false)
  passDate            DateTime?
  subjectNameOverride String? // For electives (e.g. "Cloud Computing" instead of "Elective-VI")
  createdAt           DateTime @default(now())
  updatedAt           DateTime @updatedAt
  subject             Subject  @relation(fields: [subjectId], references: [id])

  @@unique([studentId, subjectId, semesterId, attemptNumber])
  @@index([studentId])
  @@index([batch])
  @@index([semesterId])
}

model Attendance {
  id              String   @id @default(uuid()) @db.Uuid
  studentId       String
  subjectId       String
  semesterId      String
  totalClasses    Int
  attendedClasses Int
  batch           String? // e.g. "O21"
  subjectNameOverride String? // For electives (e.g. "Cloud Computing" instead of "Elective-VI")
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt
  subject         Subject  @relation(fields: [subjectId], references: [id])

  @@unique([studentId, subjectId, semesterId])
  @@index([studentId])
```

```
  @@index([batch])
  @@index([semesterId])
}

enum SemesterStatus {
  DRAFT
  DEAN_REVIEW
  APPROVED
  REGISTRATION_OPEN
  REGISTRATION_CLOSED
}

model AcademicSemester {
  id                String             @id
  name              String // e.g. "AY 2024-25 SEM-II"
  status            SemesterStatus     @default(DRAFT)
  branchAllocations   BranchAllocation[]
  registrations       Registration[]
  seatingArrangements SeatingArrangement[]
  createdAt           DateTime           @default(now())
  updatedAt         DateTime           @updatedAt
}

model Faculty {
  id          String   @id @default(uuid()) @db.Uuid
  name        String
  email       String   @unique
  photo       String?
  department  String
  designation String?  @default("Assistant Professor")
  role        String   @default("FACULTY") // "FACULTY", "HOD", "DEAN", "WEBMASTER"
  bio         Json?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@index([department])
}

enum AllocationStatus {
  DEAN_PENDING
  HOD_REVIEW
  APPROVED
}

model BranchAllocation {
  id            String           @id @default(uuid()) @db.Uuid
  branch        String // e.g. "CSE", "ECE"
  academicYear  String?          @default("E1") // e.g. "E1", "E2", "E3", "E4"
  batch         String?          @default("")   // e.g. "O21"
  subjectId     String
  semesterId    String
  customName    String? // For electives (e.g. ELECTIVE (NPTEL Course))
  customCredits Int? // Ability to override default subject credits
  status        AllocationStatus @default(DEAN_PENDING)
  isApproved    Boolean          @default(false)
  isMandatory   Boolean          @default(true)
  electiveGroupId String?        @default("")
  electiveLimit Int?             @default(1)
  subject       Subject          @relation(fields: [subjectId], references: [id])
  semester      AcademicSemester @relation(fields: [semesterId], references: [id])

// ... [Truncated to optimize compile time] ...
```

2.2 Component: Uniz Auth

apps/uniz-auth/prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
  binaryTargets = ["native", "linux-musl-openssl-3.0.x", "linux-musl-arm64-openssl-3.0.x"]
}
```

```

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model AuthCredential {
  id           String   @id @default(uuid()) @db.Uuid
  username     String   @unique
  passwordHash String
  role         String   @default("student")
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
  isDisabled   Boolean  @default(false)
}

model OtpLog {
  id         String    @id @default(uuid()) @db.Uuid
  username   String
  otp        String
  expiresAt  DateTime
  createdAt  DateTime  @default(now())
  consumedAt DateTime?
}
```

2.3 Component: Uniz Cron

apps/uniz-cron/prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
  binaryTargets = ["native", "rhel-openssl-3.0.x", "linux-musl-arm64-openssl-3.0.x"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}


model Outpass {
  id            String   @id @default(uuid()) @db.Uuid
  studentId     String   // Reference to User ID
  studentGender String   @default("M") // Added for gender-based filtering
  reason        String
  fromDay       DateTime
  toDay         DateTime
  days          Int      @default(0)
  requestedTime DateTime @default(now())

  isExpired     Boolean  @default(false)
  isApproved    Boolean  @default(false)
  isRejected    Boolean  @default(false)

  issuedBy       String   @default("none")
  issuedTime     DateTime @default(now())
  message        String?
  rejectedBy     String   @default("none")
  rejectedTime   DateTime @default(now())

  checkedOutTime DateTime?
  checkedInTime  DateTime?

  currentLevel   String   @default("caretaker")
  approvalLogs   Json?    // Typed as ApprovalLogsSchema in code

  @@index([studentId])
  @@index([isApproved, isRejected, isExpired])
}

model Outing {
```

```
  id             String   @id @default(uuid()) @db.Uuid
  studentId      String   // Reference to User ID
  studentGender  String   @default("M") // Added for gender-based filtering
  reason         String
  fromTime       DateTime
  toTime         DateTime
  hours          Int      @default(0)
  requestedTime  DateTime @default(now())

  isExpired      Boolean  @default(false)
  isApproved     Boolean  @default(false)
  isRejected     Boolean  @default(false)

  issuedBy       String   @default("none")
  issuedTime     DateTime @default(now())
  message        String?
  rejectedBy     String   @default("none")
  rejectedTime   DateTime @default(now())

  checkedOutTime DateTime?
  checkedInTime  DateTime?

  currentLevel   String   @default("caretaker")
  approvalLogs  Json?

  @@index([studentId])
  @@index([isApproved, isRejected, isExpired])
}

model Grievance {
  id            String   @id @default(uuid()) @db.Uuid
  studentId     String?  // Optional: Null if truly anonymous
  studentEmail  String?
  category      String   // e.g., "Hostel", "Mess", "Academic", "Other"
  description   String
  isAnonymous   Boolean  @default(false)

  status        String   @default("pending") // pending, resolved
  resolvedBy    String?
  resolution    String?
  resolvedAt    DateTime?

  createdAt     DateTime @default(now())

  @@index([status])
  @@index([createdAt])
}


model OtpLog {
  id         String    @id @default(uuid()) @db.Uuid
  username   String
  otp        String
  expiresAt  DateTime
  createdAt  DateTime  @default(now())
  consumedAt DateTime?
}
```

2.4 Component: Uniz Files

apps/uniz-files/prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
  binaryTargets = ["native", "rhel-openssl-3.0.x", "linux-musl-arm64-openssl-3.0.x"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

```

model Subject {
  id          String   @id
  code        String   @unique
  name        String
  credits     Int
  department  String
  semester    String

  grades      Grade[]
  attendance  Attendance[]
}

model Grade {
  id          String   @id @default(uuid()) @db.Uuid
  studentId   String
  subjectId   String
  semesterId  String
  grade       Float

  subject     Subject  @relation(fields: [subjectId], references: [id])

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@unique([studentId, subjectId, semesterId])
}

model Attendance {
  id              String   @id @default(uuid()) @db.Uuid
  studentId       String
  subjectId       String
  semesterId      String
  totalClasses    Int
  attendedClasses Int

  subject         Subject  @relation(fields: [subjectId], references: [id])

  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  @@unique([studentId, subjectId, semesterId])
}
```

2.5 Component: Uniz Notifications

apps/uniz-notifications/prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
  binaryTargets = ["native", "rhel-openssl-3.0.x", "linux-musl-arm64-openssl-3.0.x"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model StudentProfile {
  id                  String    @id @default(uuid()) @db.Uuid
  username            String    @unique
  name                String    @default("")
  email               String    @default("")
  gender              String    @default("")
  phone               String    @default("")
  fatherName          String    @default("")
  motherName          String    @default("")
  fatherOccupation    String    @default("")
  motherOccupation    String    @default("")
  fatherEmail         String    @default("")
  motherEmail         String    @default("")
```

```
24   fatherAddress        String    @default("")
25   motherAddress        String    @default("")
26   bloodGroup           String    @default("")
27   dateOfBirth          DateTime?
28   profileUrl           String    @default("")
29   year                 String    @default("")
30   branch               String    @default("")
31   section              String    @default("")
32   roomno               String    @default("")
33   isPresentInCampus    Boolean   @default(true)
34   isApplicationPending Boolean   @default(false)
35   telegramChatId       String?
36   createdAt            DateTime  @default(now())
37   updatedAt            DateTime  @updatedAt
38
39   @@index([branch, year])
40 }
41
42 model FacultyProfile {
43   id          String   @id @default(uuid()) @db.Uuid
44   username    String   @unique
45   name        String
46   email       String   @unique
47   contact     String   @default("")
48   department  String
49   designation String
50   role        String   @default("teacher")
51   profileUrl  String   @default("")
52   createdAt   DateTime @default(now())
53   updatedAt   DateTime @updatedAt
54
55   @@index([department])
56 }
57
58 model AdminProfile {
59   id        String   @id @default(uuid()) @db.Uuid
60   username  String   @unique
61   email     String   @unique @default("")
62   role      String   @default("webmaster")
63   createdAt DateTime @default(now())
64   updatedAt DateTime @updatedAt
65 }
66
67 model Banner {
68   id        String   @id @default(uuid()) @db.Uuid
69   title     String
70   text      String
71   imageUrl  String
72   isVisible Boolean  @default(true)
73   createdAt DateTime @default(now())
74   updatedAt DateTime @updatedAt
75 }
76
77 model PublicNotification {
78   id        String   @id @default(uuid()) @db.Uuid
79   title     String
80   content   String
81   link      String?
82   isVisible Boolean  @default(true)
83   createdAt DateTime @default(now())
84   updatedAt DateTime @updatedAt
85 }
86
87 model Tender {
88   id          String    @id @default(uuid()) @db.Uuid
89   title       String
90   description String
91   pdfUrl      String?
92   deadline    DateTime?
93   isVisible   Boolean   @default(true)
94   createdAt   DateTime  @default(now())
95   updatedAt   DateTime  @updatedAt
96 }
```

```

model UploadHistory {
  id           String   @id @default(uuid()) @db.Uuid
  type         String // "STUDENTS" | "GRADES" | "ATTENDANCE" | "FACULTY"
  filename     String?
  totalRows    Int      @default(0)
  successCount Int      @default(0)
  failCount    Int      @default(0)
  errors       Json? // Summary of errors or specific row failures
  status       String   @default("COMPLETED") // "COMPLETED" | "FAILED" | "PARTIAL"
  uploadedBy   String // Username of the webmaster
  createdAt    DateTime @default(now())

  @@index([type])
  @@index([uploadedBy])
}

model PushSubscription {
  id        String   @id @default(uuid()) @db.Uuid
  username  String   // Link to StudentProfile.username, etc.
  endpoint  String   @unique
  p256dh    String
  auth      String
  createdAt DateTime @default(now())

// ... [Truncated to optimize compile time] ...
```

2.6 Component: Uniz Outpass

apps/uniz-outpass/prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
  binaryTargets = ["native", "rhel-openssl-3.0.x", "linux-musl-arm64-openssl-3.0.x"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Outpass {
  id            String   @id @default(uuid())
  studentId     String   // Reference to User ID
  studentGender String   @default("M") // Added for gender-based filtering
  reason        String
  fromDay       DateTime
  toDay         DateTime
  days          Int      @default(0)
  requestedTime DateTime @default(now())

  isExpired     Boolean  @default(false)
  isApproved    Boolean  @default(false)
  isRejected    Boolean  @default(false)

  issuedBy       String   @default("none")
  issuedTime     DateTime @default(now())
  message        String?
  rejectedBy     String   @default("none")
  rejectedTime   DateTime @default(now())

  checkedOutTime DateTime?
  checkedInTime  DateTime?

  currentLevel   String   @default("caretaker")
  approvalLogs   Json?    // Typed as ApprovalLogsSchema in code

  @@index([studentId])
  @@index([isApproved, isRejected, isExpired])
}

```

```
model Outing {
  id             String   @id @default(uuid())
  studentId      String   // Reference to User ID
  studentGender  String   @default("M") // Added for gender-based filtering
  reason         String
  fromTime       DateTime
  toTime         DateTime
  hours          Int      @default(0)
  requestedTime  DateTime @default(now())

  isExpired      Boolean  @default(false)
  isApproved     Boolean  @default(false)
  isRejected     Boolean  @default(false)

  issuedBy       String   @default("none")
  issuedTime     DateTime @default(now())
  message        String?
  rejectedBy     String   @default("none")
  rejectedTime   DateTime @default(now())

  checkedOutTime DateTime?
  checkedInTime  DateTime?

  currentLevel   String   @default("caretaker")
  approvalLogs  Json?

  @@index([studentId])
  @@index([isApproved, isRejected, isExpired])
}

model Grievance {
  id            String   @id @default(uuid())
  studentId     String?  // Optional: Null if truly anonymous
  studentEmail  String?
  category      String   // e.g., "Hostel", "Mess", "Academic", "Other"
  description   String
  isAnonymous   Boolean  @default(false)

  status        String   @default("pending") // pending, resolved
  resolvedBy    String?
  resolution    String?
  resolvedAt    DateTime?

  createdAt     DateTime @default(now())

  @@index([status])
  @@index([createdAt])
}

model AuthCredential {
  id           String   @id @default(uuid())
  username     String   @unique
  passwordHash String
  role         String   @default("student")
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
  isDisabled   Boolean  @default(false)
}
```

2.7 Component: Uniz User

apps/uniz-user/prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
  binaryTargets = ["native", "rhel-openssl-3.0.x", "linux-musl-arm64-openssl-3.0.x"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
```

```
}

model StudentProfile {
  id                    String    @id @default(uuid()) @db.Uuid
  username              String    @unique
  name                  String    @default("")
  email                 String    @default("")
  gender                String    @default("")
  phone                 String    @default("")
  fatherName            String    @default("")
  motherName            String    @default("")
  fatherOccupation      String    @default("")
  motherOccupation      String    @default("")
  fatherEmail           String    @default("")
  motherEmail           String    @default("")
  fatherAddress         String    @default("")
  motherAddress         String    @default("")
  bloodGroup            String    @default("")
  dateOfBirth           DateTime?
  profileUrl            String    @default("")
  year                  String    @default("")
  semester              String    @default("")
  branch                String    @default("")
  section               String    @default("")
  batch                 String    @default("")
  roomno                String    @default("")
  isPresentInCampus     Boolean   @default(true)
  isApplicationPending  Boolean   @default(false)
  isSuspended           Boolean   @default(false)
  telegramChatId        String?
  createdAt             DateTime  @default(now())
  updatedAt             DateTime  @updatedAt

  // New Fields from O21 Allocation Data
  category              String?   @default("")
  campus                String?   @default("")

  // Denormalized Academic Fields for High-Speed Analytics
  cgpa                  Float     @default(0.0)
  totalBacklogs         Int       @default(0)

  @@index([branch, year, batch])
  @@index([cgpa, totalBacklogs])
}

model FacultyProfile {
  id          String   @id @default(uuid()) @db.Uuid
  username    String   @unique
  name        String
  email       String   @unique
  contact     String   @default("")
  department  String
  designation String
  role        String   @default("teacher")
  profileUrl  String   @default("")
  bio         Json?
  isSuspended Boolean  @default(false)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@index([department])
}

model AdminProfile {
  id          String   @id @default(uuid()) @db.Uuid
  username    String   @unique
  name        String   @default("")
  email       String   @unique @default("")
  contact     String   @default("")
  profileUrl  String   @default("")
  role        String   @default("webmaster")
  designation String   @default("")
  department  String   @default("")
```

```
  bio         Json?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model Banner {
  id        String   @id @default(uuid()) @db.Uuid
  title     String
  text      String
  imageUrl  String
  isVisible Boolean  @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

model PublicNotification {
  id        String   @id @default(uuid()) @db.Uuid
  title     String
  content   String
  link      String?
  isVisible Boolean  @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

model Tender {
  id          String    @id @default(uuid()) @db.Uuid
  title       String
  description String
  pdfUrl      String?
  deadline    DateTime?
  isVisible   Boolean   @default(true)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}

model UploadHistory {
  id           String   @id @default(uuid()) @db.Uuid
  type         String // "STUDENTS" | "GRADES" | "ATTENDANCE" | "FACULTY"

// ... [Truncated to optimize compile time] ...
```

2.8 Component: Uniz Academics

apps/uniz-academics/src/services/upload.service.ts

```
import { redis } from "../utils/redis.util";
import prisma from "../utils/prisma.util";
import { mapGradeToPoint } from "../utils/helpers.util";
import axios from "axios";

const GATEWAY_URL = (
  process.env.GATEWAY_URL ||
  (process.env.DOCKER_ENV === "true"
    ? "http://uniz-gateway-api:3000/api/v1"
    : "http://localhost:3000/api/v1")
).replace(/\/$/, "");
const USER_SERVICE_URL = (
  process.env.USER_SERVICE_URL || `${GATEWAY_URL}/profile`
).replace(/\/$/, "");

const recordUploadHistory = async (data: any, retry = 0) => {
  try {
    // Attempt direct service call first with Loop Buster
    const url = `${USER_SERVICE_URL}/internal/upload-history?lb=${Math.random().toString(36).
    substring(7)}`;
    console.log(`[Worker] Recording upload history externally: ${url}`);
    await axios.post(url, data, {
      headers: {
        "x-internal-secret": process.env.INTERNAL_SECRET || "uniz-core",
      },
```

```
      timeout: 10000, // Increased timeout for history recording
    });
    console.log(`[Worker] Successfully recorded upload history.`);
  } catch (err: any) {
    if (retry < 1) {
      console.warn(
        `[Worker] History record failed: ${err.message}. Retrying in 2s...`,
      );
      await new Promise((r) => setTimeout(r, 2000));
      return recordUploadHistory(data, retry + 1);
    }
    console.error(
      "[Worker] Failed to record upload history after retry:",
      err.message,
    );
  }
};

export async function processNextBatch() {
  const jobString = await redis.lpop("job:queue");
  if (!jobString) return null;

  let job;
  try {
    job = JSON.parse(jobString);
  } catch (e) {
    console.error("Invalid job JSON", e);
    return null;
  }

  const {
    type,
    rows: remainingRows,
    user,
    uploadId,
    processed: initialProcessed = 0,
    total,
    filename,
    fileUrl,
    startTime = Date.now(),
    successCount: initialSuccess = 0,
    failCount: initialFail = 0,
    errors: initialErrors = [],
  } = job;

  let successCount = initialSuccess;
  let failCount = initialFail;
  let errors = initialErrors;

  // Ensure these are available in scope (redundant but safe)
  const totalRows = total;
  const currentUser = user;

  // Process up to 6 batches (300 rows) in one execution to provide more frequent UI updates
  const BATCHES_PER_RUN = 6;
  const CHUNK_SIZE = 50;

  let processedInThisRun = 0;

  // Cache subjects once for all batches in this run
  const allSubjects = await prisma.subject.findMany();
  const subjectMap = new Map<string, any>(
    allSubjects.map((s: any) => [s.code.trim().toUpperCase(), s]),
  );

  const getVal = (row: any, keys: string[]) => {
    // Assuming rowKeys is available in the scope, which it isn't in the provided snippet.
    // This part of the code seems to be missing context or `rowKeys` is defined elsewhere.
    // For now, I'll assume `rowKeys` is meant to be `Object.keys(row)`.
    const rowKeys = Object.keys(row);
    const normalizedKeys = rowKeys.map((k) =>
      k.trim().toLowerCase().replace(/\s+/g, " "),
    );
```

```
    const foundIdx = normalizedKeys.findIndex((nk) => {
      return keys.some(
        (target) =>
          nk === target.toLowerCase() || nk.includes(target.toLowerCase()),
      );
    });

    if (foundIdx !== -1) {
      const actualKey = rowKeys[foundIdx];
      console.log(
        `[Worker] Matched header '${actualKey}' for keys: ${keys.join(", ")}`,
      ); // Added log
      return String(row[actualKey]).trim();
    }
    console.log(`[Worker] No header matched for keys: ${keys.join(", ")}`); // Added log
    return "";
  };

  if (processedInThisRun === 0) {
    console.log(
      `[Worker] [${uploadId}] Normalized Subject Map size: ${subjectMap.size}.`,
    );
  }

// ... [Truncated to optimize compile time] ...
```

2.9 Component: Uniz Mail

apps/uniz-mail/src/services/email.service.ts

```
import nodemailer from "nodemailer";
import * as sesv2 from "@aws-sdk/client-sesv2";
import * as SESv2Saves from "@aws-sdk/client-sesv2";
import {
  generateResultPdf,
  ResultData,
  generateAttendancePdf,
  AttendanceData,
} from "../utils/pdf.util";

const emailUser = process.env.EMAIL_USER;
const emailPass = process.env.EMAIL_PASS;

// --- PROVIDER SELECTION ---
const isProduction = process.env.DOCKER_ENV === "true";
const forceGmail = process.env.FORCE_GMAIL === "true";

let sesTransporter: nodemailer.Transporter | null = null;

if (isProduction && !forceGmail) {
  try {
    const accessKeyId =
      process.env.PROD_AWS_ACCESS_KEY_NEWONE_AFTER_LEAK ||
      process.env.AWS_ACCESS_KEY_ID;
    const secretAccessKey =
      process.env.PROD_AWS_ACCESS_KEY_SECRET_NEWONE_AFTER_LEAK ||
      process.env.AWS_SECRET_ACCESS_KEY;
    const region = process.env.AWS_REGION || "ap-south-1";

    if (!accessKeyId || !secretAccessKey) {
      throw new Error("AWS Credentials missing for SES production.");
    }

    const sesClient = new sesv2.SESv2Client({
      region,
      credentials: {
        accessKeyId,
        secretAccessKey,
      },
    });

```

```
    sesTransporter = nodemailer.createTransport({
      SES: {
        sesClient,
        SendEmailCommand: sesv2.SendEmailCommand,
      },
    } as any);

    console.log(
      `[MAIL-SES] Production SES v3 Transporter Initialized with key prefix: ${accessKeyId.substring(0, 5)}...`,
    );
  } catch (error) {
    console.error(`[MAIL-SES] initialization failed: ${error}`);
  }
}

// --- GMAIL SETUP (Local / Fallback) ---
const localGmailTransporter = nodemailer.createTransport({
  service: "gmail",
  auth: {
    user: process.env.EMAIL_USER || "noreplycampusschield@gmail.com",
    pass: (process.env.EMAIL_PASS || "acix rfbi kujh xwtj").replace(/\s/g, ""),
  },
});

const getTransporter = () => {
  if (sesTransporter) return sesTransporter;
  return localGmailTransporter;
};

console.log(`[MAIL] Transporter Initialized.`);
console.log(`[MAIL] Active Provider: ${sesTransporter ? "AWS SES" : "Gmail"}`);

// --- RESEND SETUP ---
// --- EMAIL DELIVERY SETUP ---
// Unified email sender using the Gmail Pool (Google Workspace / Organizational)
const sendEmailUnified = async (options: {
  from: string;
  to: string;
  subject: string;
  html: string;
  attachments?: any[];
}): Promise<boolean> => {
  const provider = sesTransporter ? "SES" : "GMAIL";
  try {
    const info = await getTransporter().sendMail({
      ...options,
      headers: {
        "X-Entity-Ref-ID": `uniz-${Date.now()}`,
        "X-Priority": "1 (Highest)",
        "X-Mailer": "UniZ-Mail-Engine",
      },
    });

    console.log(
      `[MAIL-SUCCESS] [${provider}] ID=${info.messageId}, To=${options.to}, Subject="${options.subject}"`,
    );
    return true;
  } catch (err: any) {
    // Classification of errors for robustness during surges
    if (err.message?.includes("Throttling") || err.code === "Throttling") {
      console.error(
        `[MAIL-CRITICAL] [${provider}] SES Rate Limit Exceeded. Surge protection active.`,
      );
    } else if (
      err.message?.includes("Blacklisted") ||
      err.name === "MessageRejected"
    ) {
      console.error(
        `[MAIL-BOUNCE] [${provider}] Recipient ${options.to} rejected/bounced. Flagging for review.`,
      );
```

```
      // TODO: Implement DB flag for invalid email if we add a persistence layer here
    } else {
      console.error(
        `[MAIL-ERROR] [${provider}] Delivery failed for ${options.to}:`,
        err.message,
      );
    }
    return false;
  }

// ... [Truncated to optimize compile time] ...
```

2.10 Component: Uniz User

apps/uniz-user/src/services/bulk-worker.service.ts

```
import { PrismaClient } from "@prisma/client";
import { redis } from "../utils/redis.util";
import axios from "axios";

const prisma = new PrismaClient();
const AUTH_SERVICE_URL =
  process.env.AUTH_SERVICE_URL || "http://localhost:3001";
const INTERNAL_SECRET = process.env.INTERNAL_SECRET || "uniz-core";

const BRANCH_MAP: Record<string, string> = {
  "COMPUTER SCIENCE AND ENGINEERING": "CSE",
  "ELECTRONICS AND COMMUNICATION ENGINEERING": "ECE",
  "ELECTRICAL AND ELECTRONICS ENGINEERING": "EEE",
  "MECHANICAL ENGINEERING": "MECH",
  "CIVIL ENGINEERING": "CIVIL",
  "CHEMICAL ENGINEERING": "CHEM",
  "METALLURGICAL AND MATERIALS ENGINEERING": "MME",
  "METALLURGY AND MATERIALS ENGINEERING": "MME",
};

const mapBranch = (name: string) => {
  const upper = (name || "").trim().toUpperCase();
  return BRANCH_MAP[upper] || upper;
};

export async function processNextStudentBatch() {
  const jobString = await redis.lpop("student:job:queue");
  if (!jobString) return null;

  let job;
  try {
    job = JSON.parse(jobString);
  } catch (e) {
    console.error("[Bulk] Invalid student job JSON", e);
    return null;
  }

  const {
    rows: remainingRows,
    username,
    uploadId,
    processed: initialProcessed = 0,
    total,
    fileUrl,
    startTime = Date.now(),
    successCount: initialSuccess = 0,
    failCount: initialFail = 0,
    errors: initialErrors = [],
    majorityBatch,
  } = job;

  let successCount = initialSuccess;
  let failCount = initialFail;
  let errors = initialErrors;

```

```
  // Process up to 6 batches (300 rows) in one execution to provide frequent UI updates
  const BATCHES_PER_RUN = 6;
  const CHUNK_SIZE = 50;
  let processedInThisRun = 0;

  console.log(
    `[Bulk] [${uploadId}] Starting Student Worker Run for ${username}. Total: ${total}`,
  );

  while (
    remainingRows.length > 0 &&
    processedInThisRun < BATCHES_PER_RUN * CHUNK_SIZE
  ) {
    const currentBatch = remainingRows.splice(0, CHUNK_SIZE);
    processedInThisRun += currentBatch.length;

    await Promise.all(
      currentBatch.map(async (row: any, indexInChunk: number) => {
        const getVal = (keys: string[]) => {
          const found = Object.keys(row).find((k) =>
            keys.includes(k.trim().toLowerCase()),
          );
          if (!found) return "";
          const val = row[found];
          if (val && typeof val === "object") {
            return (val as any).text ? String((val as any).text).trim() : "";
          }
          return val ? String(val).trim() : "";
        };

        const id = getVal([
          "student id",
          "studentid",
          "student_id",
          "id",
          "username",
        ]).toUpperCase();
        const name = getVal(["name", "student name", "student_name"]);
        const email = getVal(["email", "mail", "student_email"]);
        const gender = getVal(["gender", "sex"]);
        const branch = mapBranch(
          getVal(["branch", "department", "allocation_branch"]),
        );
        const year = getVal(["year", "class", "academic_year"]).toUpperCase();
        const section = getVal(["section", "sec"]).toUpperCase();
        const phone = getVal(["phone", "mobile", "contact"]);
        const batchCol = getVal(["batch", "acad_batch", "academic_batch"]);

        const category = getVal(["category", "admission category", "caste"]);
        const campus = getVal(["campus", "center", "location"]);

        if (!id) {
          failCount++;
          errors.push({ row: "unknown", error: "Missing Student ID" });
          return;
        }

        const finalEmail = email || `${id.toLowerCase()}@rguktong.ac.in`;
        const finalYear = year || "E1";
        const finalSection =
          !section || ["UNKNOWN", "GENERAL"].includes(section)
            ? Math.floor(Math.random() * 4 + 1).toString()
            : section;

        const finalBatch = (batchCol || "").toUpperCase();

// ... [Truncated to optimize compile time] ...
```

2.11 Component: Uniz Academics

apps/uniz-academics/src/controllers/academic.controller.ts

```
import { Response } from "express";
import { AuthenticatedRequest } from "../middlewares/auth.middleware";
import ExcelJS from "exceljs";
import axios from "axios";
import prisma from "../utils/prisma.util";
import { ErrorCode } from "../shared/error-codes";

const GATEWAY_URL = (
  (process.env.DOCKER_ENV === "true"
    ? "http://uniz-gateway-api:3000/api/v1"
    : process.env.GATEWAY_URL) || "http://localhost:3000/api/v1"
).replace(/\/$/, "");

const USER_SERVICE_URL = (
  (process.env.DOCKER_ENV === "true"
    ? "http://uniz-user-service:3002"
    : process.env.USER_SERVICE_URL) || "http://localhost:3002"
).replace(/\/$/, "");

const getHeaders = (token: string) => ({ headers: { Authorization: token } });
const getInternalHeaders = () => ({
  headers: { "x-internal-secret": process.env.INTERNAL_SECRET || "uniz-core" },
});

import { mapGradeToPoint, getGpaDialogue } from "../utils/helpers.util";
import { randomUUID } from "crypto";
import { redis, notificationQueue } from "../utils/redis.util";
import { processNextBatch } from "../services/upload.service";
import { generateResultPdf, generateAttendancePdf } from "../utils/pdf.util";

// GPA calculation and templates now use database-provided subject credits.
// GRADE_MAP and mapGradeToPoint are now imported from helpers.util
// const GRADE_MAP: Record<string, number> = {
//   EX: 10,
//   A: 9,
//   B: 8,
//   C: 7,
//   D: 6,
//   E: 5,
//   R: 0,
// };

// const mapGradeToPoint = (val: string | number): number => {
//   if (typeof val === "number") return val;
//   if (!val) return 0;
//   const upper = String(val).toUpperCase().trim();
//   return GRADE_MAP[upper] ?? (parseFloat(upper) || 0);
// };

export const getUploadProgress = async (
  req: AuthenticatedRequest,
  res: Response,
) => {
  const user = req.user;
  if (!user)
    return res.status(401).json({ success: false, message: "Unauthorized" });

  try {
    res.setHeader(
      "Cache-Control",
      "no-store, no-cache, must-revalidate, proxy-revalidate",
    );
    res.setHeader("Pragma", "no-cache");
    res.setHeader("Expires", "0");

    const uploadId = req.query.uploadId as string;

    if (uploadId) {
      const uploadKey = `upload:progress:${uploadId}`;
```

```
      const progressData = await redis.get(uploadKey);

      if (progressData) {
        const progress = JSON.parse(progressData);

        // --- SELF-HEALING: JUMP START STUCK JOBS ---
        // If status is 'queued' or 'processing' but no activity for > 15 seconds,
        // it means the trigger likely failed or timed out. Poke it!
        const isStuck =
          (progress.status === "queued" || progress.status === "processing") &&
          Date.now() - (progress.lastActive || 0) > 15000;

        if (isStuck) {
          console.log(
            `[Academics]      Job ${uploadId} appears stuck. Jump-starting via Progress Poll (
    Direct)...`,
          );
          // Fire and forget direct call to prevent blocking the user
          // Using a recursive helper to handle the 'continued' state
          const runStuck = async () => {
            let result;
            do {
              result = await processNextBatch();
            } while (result && result.status === "continued");
          };
          runStuck().catch((e) =>
            console.error("[Academics] Stuck jump-start error:", e),
          );
        }

        return res.json({
          success: true,
          progress,
        });
      }
      return res.status(404).json({
        success: false,
        message: "Upload not found or expired",
      });
    }

    // Fallback: Check by username for backwards compatibility
    const uploadKey = `upload:progress:${user.username}`;
    const uploadProgress = await redis.get(uploadKey);
    if (uploadProgress) {
      return res.json({ success: true, progress: JSON.parse(uploadProgress) });
    }

    // Fallback to Publish Progress
    const publishKey = `publish:progress:${user.username}`;
    const publishProgress = await redis.get(publishKey);
    if (publishProgress) {

// ... [Truncated to optimize compile time] ...
```

apps/uniz-academics/src/controllers/faculty.controller.ts

```
import { Request, Response } from "express";
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

// Get all faculty
export const getFaculties = async (req: Request, res: Response) => {
  try {
    const { department } = req.query;
    const filter: any = {};
    if (department && department !== "all") {
      filter.department = department as string;
    }

```

```
    const faculties = await prisma.faculty.findMany({
      where: filter,
      orderBy: { name: "asc" },
    });
    res.json(faculties);
  } catch (error: any) {
    res.status(500).json({ error: error.message });
  }
};

// Get single faculty profile
export const getFacultyProfile = async (req: Request, res: Response) => {
  try {
    const { id } = req.params;
    const faculty = await prisma.faculty.findUnique({
      where: { id },
    });
    if (!faculty) return res.status(404).json({ error: "Faculty not found" });
    res.json(faculty);
  } catch (error: any) {
    res.status(500).json({ error: error.message });
  }
};

// Create faculty (Webmaster or HOD for their dept)
export const createFaculty = async (req: any, res: Response) => {
  try {
    const { name, email, department, photo, bio, role, designation } = req.body;
    const user = req.user;

    if (!user) return res.status(401).json({ error: "Unauthorized" });

    // HODs can only create for their own department
    if (user.role === "hod") {
      // For now, let's just allow HODs to create faculty.
    } else if (user.role !== "webmaster" && user.role !== "dean") {
      return res.status(403).json({ error: "Permission denied" });
    }

    const faculty = await prisma.faculty.create({
      data: {
        name,
        email: email.toLowerCase(),
        department: department.toUpperCase(),
        photo,
        bio: bio || {},
        designation: designation || "Assistant Professor",
        role: role || "FACULTY",
      },
    });
    res.status(201).json(faculty);
  } catch (error: any) {
    res.status(500).json({ error: error.message });
  }
};

// Update faculty (Webmaster, HOD, or Self)
export const updateFaculty = async (req: any, res: Response) => {
  try {
    const { id } = req.params;
    const { name, photo, bio, department, role, designation } = req.body;
    const user = req.user;

    const existing = await prisma.faculty.findUnique({ where: { id } });
    if (!existing) return res.status(404).json({ error: "Faculty not found" });

    // Auth check
    if (
      user.role !== "webmaster" &&
      user.role !== "hod" &&
      user.username.toLowerCase() !== existing.email.split("@")[0].toLowerCase()
    ) {
      // return res.status(403).json({ error: "Permission denied" });
```

```
    }

    const faculty = await prisma.faculty.update({
      where: { id },
      data: {
        name,
        photo,
        bio,
        department: department?.toUpperCase(),
        designation,
        role,
      },
    });
    res.json(faculty);
  } catch (error: any) {
    res.status(500).json({ error: error.message });
  }
};

// Delete faculty (Webmaster only)
export const deleteFaculty = async (req: Request, res: Response) => {
  try {
    const { id } = req.params;
    await prisma.faculty.delete({
      where: { id },
    });
    res.json({ message: "Faculty deleted successfully" });
  } catch (error: any) {
    res.status(500).json({ error: error.message });
  }
};

// Update Role (Webmaster only)

// ... [Truncated to optimize compile time] ...
```

apps/uniz-academics/src/controllers/queue.controller.ts

```
import { Request, Response } from "express";
import { processNextBatch } from "../services/upload.service";
import axios from "axios";

export const processQueue = async (req: Request, res: Response) => {
  const secret = req.headers["x-internal-secret"];
  const INTERNAL_SECRET = process.env.INTERNAL_SECRET || "uniz-core";

  const receivedSecret = Array.isArray(secret) ? secret[0] : secret;

  if (receivedSecret !== INTERNAL_SECRET) {
    console.warn(
      `[Queue] Unauthorized trigger attempt. Received length: ${receivedSecret ? receivedSecret.length : 0}, Expected length: ${INTERNAL_SECRET.length}`,
    );
    console.warn(
      `[Queue] Received starts with: ${receivedSecret ? receivedSecret.substring(0, 3) : "NULL"}`,
    );
    console.warn(
      `[Queue] Expected starts with: ${INTERNAL_SECRET.substring(0, 3)}`,
    );
    return res.status(401).json({ error: "Unauthorized" });
  }

  try {
    const result = await processNextBatch();
    if (!result) {
      return res.status(200).json({ message: "No jobs in queue" });
    }

    // CRITICAL: Trigger next batch BEFORE sending response
    // Otherwise Vercel terminates the function immediately after res.json()
```

```
    if (result.status === "continued") {
      const port = process.env.PORT || 3004;
      const url = `http://localhost:${port}/api/queue/process`;

      // Loop Buster: Add random param to avoid 508 Loop Detected
      const triggerUrl = `${url}?lb=${Math.random().toString(36).substring(7)}`;
      console.log(
        `[Queue]      Triggering next batch recursively: ${triggerUrl}`,
      );

      // Fire and forget to avoid HTTP recursion depth limits (508 Loop Detected)
      // and to free up this worker's Redis connection faster.
      axios
        .post(
          triggerUrl,
          {},
          {
            headers: { "x-internal-secret": INTERNAL_SECRET },
            timeout: 5000,
          },
        )
        .catch((e) => {
          // Only log if it's not a timeout (timeouts are expected if we don't wait)
          if (e.code !== "ECONNABORTED") {
            console.error(`[Queue]     Recursive trigger error: ${e.message}`);
          }
        });

      // Increased delay to ensure request is dispatched and proxy "rests"
      await new Promise((resolve) => setTimeout(resolve, 500));
      console.log(`[Queue]  Next batch dispatched.`);
    }

    return res.status(200).json(result);
  } catch (err: any) {
    console.error("Queue Worker Error:", err);
    return res.status(500).json({ error: err.message });
  }
};
```

2.12 Component: Uniz Auth

apps/uniz-auth/src/controllers/auth.controller.ts

```
import { Request, Response } from "express";
import axios from "axios";
import { prisma } from "../utils/prisma";
import { signToken, verifyToken } from "../utils/token.util";
import { AuthenticatedRequest } from "../middlewares/auth.middleware";
import { redis } from "../utils/redis.util";
import {
  sendOtpEmail,
  sendOtpPush,
  sendLoginNotification,
  sendPasswordChangeNotification,
} from "../utils/email.util";
import { comparePassword, hashPassword } from "../utils/password.util";
import { ErrorCode } from "../shared/error-codes";
import { UserRole } from "../shared/roles.enum";
import { UAParser } from "ua-parser-js";
import { verifyTurnstileToken } from "../utils/turnstile.util";

export const login = async (req: Request, res: Response) => {
  const username = (req.body.username || "").trim(); // Case-insensitive: handled by Prisma
    mode:"insensitive" below
  const password = req.body.password;
  const captchaToken = req.body.captchaToken;

  // Cloudflare Turnstile Verification
  const isHuman = await verifyTurnstileToken(captchaToken, req.ip);

```

```
27    if (!isHuman) {
28      return res.status(400).json({
29        code: "AUTH_CAPTCHA_FAILED",
30        message: "Security verification failed. Please try again.",
31      });
32    }
33
34    try {
35      const user = await prisma.authCredential.findFirst({
36        where: { username: { equals: username, mode: "insensitive" } },
37      });
38
39      if (!user) {
40        return res.status(401).json({
41          code: ErrorCode.AUTH_INVALID_CREDENTIALS,
42          message: "Invalid username or password",
43        });
44      }
45
46      if (user.isDisabled) {
47        console.warn(
48          `[AUTH-LOGIN]     Login blocked for suspended user: ${user.username}`,
49        );
50        return res.status(403).json({
51          code: "AUTH_SUSPENDED",
52          message:
53            "Your account is currently suspended. Please contact the administrator for
      assistance.",
54        });
55      }
56
57      const isValid = await comparePassword(password, user.passwordHash);
58      if (!isValid) {
59        return res.status(401).json({
60          code: ErrorCode.AUTH_INVALID_CREDENTIALS,
61          message: "Invalid username or password",
62        });
63      }
64
65      const inferDepartment = (uname: string): string => {
66        const parts = uname.toUpperCase().split("_");
67        const commonBranches = [
68          "CSE",
69          "ECE",
70          "ME",
71          "CE",
72          "MME",
73          "CHEM",
74          "EEE",
75          "CIVIL",
76          "MET",
77          "MEC",
78        ];
79        if (parts.length > 1) {
80          const lastPart = parts[parts.length - 1];
81          if (commonBranches.includes(lastPart)) return lastPart;
82        }
83        return "";
84      };
85
86      const normalizedUsername = user.username.toUpperCase();
87      let department = "";
88      try {
89        const rawUserUrl = (
90          process.env.USER_SERVICE_URL || "http://localhost:3002"
91        ).trim();
92        const USER_SERVICE = rawUserUrl.endsWith("/health")
93          ? rawUserUrl.slice(0, -7)
94          : rawUserUrl;
95        const INTERNAL_SECRET = (
96          process.env.INTERNAL_SECRET || "uniz-core"
97        ).trim();
98
```

```
      const userType = user.role === UserRole.STUDENT ? "student" : "faculty";
      // Try faculty first, then admin if faculty fails/returns empty for non-students
      const endpoints =
        user.role === UserRole.STUDENT
          ? [`student/${normalizedUsername}`]
          : [`faculty/${normalizedUsername}`, `admin/${normalizedUsername}`];

      for (const endpoint of endpoints) {
        try {
          const userRes = await axios.get(`${USER_SERVICE}/admin/${endpoint}`, {
            headers: { "x-internal-secret": INTERNAL_SECRET },
            timeout: 3000,
          });
          const data =
            userRes.data?.student ||
            userRes.data?.faculty ||
            userRes.data?.data ||
            userRes.data;
          const foundDept = data?.department || data?.branch;
          if (foundDept) {
            department = foundDept;
            break;

// ... [Truncated to optimize compile time] ...
```

2.13 Component: Uniz Files

apps/uniz-files/src/controllers/file.controller.ts

```
import { Response } from "express";
import { AuthenticatedRequest } from "../middlewares/auth.middleware";
import { PrismaClient } from "@prisma/client";
import ExcelJS from "exceljs";
import { v2 as cloudinary } from "cloudinary";

// Configure Cloudinary (ensure env vars are set: CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY,
    CLOUDINARY_API_SECRET)
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET,
});

const prisma = new PrismaClient();

// --- Templates --- (No changes)

import axios from "axios";

const GATEWAY_URL =
  (process.env.DOCKER_ENV === "true"
    ? "http://uniz-gateway-api:3000/api/v1"
    : process.env.GATEWAY_URL) || "http://localhost:3000/api/v1";

export const downloadAttendanceTemplate = async (
  req: AuthenticatedRequest,
  res: Response,
) => {
  try {
    const { branch, year, semester, subjectCode } = req.query;

    let rows = [];

    // If criteria provided, try to fetch students and pre-fill
    if (
      branch &&
      year &&
      semester &&
      subjectCode &&
      req.headers.authorization
    ) {
```

```
      try {
        // Fetch students from User Service (Inter-service call)
        const response = await axios.post(
          `${GATEWAY_URL}/profile/student/search`,
          {
            branch,
            year,
            limit: 1000, // Get all
          },
          {
            headers: { Authorization: req.headers.authorization },
          },
        );

        if (response.data.success && response.data.students) {
          rows = response.data.students.map((s: any) => [
            s.username,
            String(subjectCode).toUpperCase(),
            String(semester).toUpperCase(),
            "", // Attended (Blank)
            "", // Total (Blank)
          ]);
        }
      } catch (e) {
        console.warn("Failed to fetch students for template pre-fill", e);
      }
    }

    // Define headers
    const headers = [
      [
        "Student ID",
        "Subject Code",
        "Semester",
        "Attended Classes",
        "Total Classes",
      ],
    ];
    const data =
      rows.length > 0
        ? [...headers, ...rows]
        : [
            ...headers,
            ["O210001", "E1-SEM-1-CSE-1", "SEM-1", 25, 30], // Example if no data found
          ];

    const workbook = new ExcelJS.Workbook();
    const worksheet = workbook.addWorksheet("Attendance Template");

    data.forEach((row) => {
      worksheet.addRow(row);
    });

    // Style header
    worksheet.getRow(1).font = { bold: true };

    // Auto-width columns
    worksheet.columns = [
      { width: 15 },
      { width: 25 },
      { width: 15 },
      { width: 20 },
      { width: 20 },
    ];

    res.setHeader(
      "Content-Disposition",
      'attachment; filename="Attendance_Template.xlsx"',
    );
    res.setHeader(
      "Content-Type",
      "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    );
```

```
    const buffer = await workbook.xlsx.writeBuffer();
    res.send(buffer);
  } catch (e: any) {
    res.status(500).json({ success: false, message: e.message });
  }
};

// ... [Truncated to optimize compile time] ...
```

2.14 Component: Uniz Mail

apps/uniz-mail/src/controllers/email.controller.ts

```
import { Request, Response } from "express";
import {
  sendOtpEmail,
  sendResultEmail,
  sendLoginNotification,
  sendOutpassRequestNotification,
  sendOutingRequestNotification,
  sendOutpassApprovalNotification,
  sendOutingApprovalNotification,
  sendNewRequestAlertToAdmin,
  sendActionConfirmationToAdmin,
  sendCheckpointNotification,
  sendPasswordChangeNotification,
  sendProfileUpdateNotification,
  sendAttendanceReportEmail,
  sendGrievanceSubmissionNotification,
} from "../services/email.service";

export const sendEmail = async (req: Request, res: Response) => {
  const { type, to, data } = req.body;
  console.log(`[MAIL] Received request: type=${type}, to=${to}`);

  try {
    let success = false;
    switch (type) {
      case "otp":
        success = await sendOtpEmail(to, data.username, data.otp);
        break;
      case "results":
        success = await sendResultEmail(
          to,
          data.username,
          data.name,
          data.branch,
          data.campus,
          data.semesterId,
          data.grades,
        );
        break;
      case "attendance_report":
        success = await sendAttendanceReportEmail(
          to,
          data.username,
          data.name,
          data.branch,
          data.campus || "RGUKT Ongole",
          data.semesterId,
          data.records,
        );
        break;
      case "login_alert":
        success = await sendLoginNotification(to, data.username, data.ip);
        break;
      case "outpass_request":
        success = await sendOutpassRequestNotification(
          to,
          data.username,
          data.reason,
```

```
          data.fromDate,
          data.toDate,
        );
        break;
      case "outing_request":
        success = await sendOutingRequestNotification(
          to,
          data.username,
          data.reason,
          data.fromDate,
          data.toDate,
        );
        break;
      case "outpass_approval":
        success = await sendOutpassApprovalNotification(
          to,
          data.username,
          data.status,
          data.approver,
          data.comment,
        );
        break;
      case "outing_approval":
        success = await sendOutingApprovalNotification(
          to,
          data.username,
          data.status,
          data.approver,
          data.comment,
        );
        break;
      case "admin_alert":
        success = await sendNewRequestAlertToAdmin(
          to,
          data.studentName,
          data.studentId,
          data.reason,
          data.type,
        );
        break;
      case "admin_action_confirmation":
        success = await sendActionConfirmationToAdmin(
          to,
          data.action,
          data.studentName,
          data.studentId,
          data.type,
        );
        break;
      case "checkpoint":
        success = await sendCheckpointNotification(
          to,
          data.username,
          data.type,
          data.time,
        );
        break;
      case "password_change":
        success = await sendPasswordChangeNotification(to, data.username);
        break;
      case "profile_update":
        success = await sendProfileUpdateNotification(

// ... [Truncated to optimize compile time] ...
```

2.15 Component: Uniz Outpass

apps/uniz-outpass/src/controllers/grievance.controller.ts

```
import { Response } from "express";
import { PrismaClient } from "@prisma/client";
```

```
import { AuthenticatedRequest } from "../middlewares/auth.middleware";
import { ErrorCode } from "../shared/error-codes";
import { z } from "zod"; // Assuming zod is available as per package.json
import axios from "axios";

const prisma = new PrismaClient();

const GrievanceSchema = z.object({
  category: z.string().min(1, "Category is required"),
  description: z.string().min(10, "Description must be at least 10 characters"),
  isAnonymous: z.boolean().default(false),
});

const NOTIFICATION_SERVICE_URL = (
  (process.env.DOCKER_ENV === "true"
    ? "http://uniz-notification-service:3007"
    : process.env.NOTIFICATION_SERVICE_URL) || "http://localhost:3007"
)
  .trim()
  .replace(/\/health$/, "");

// Helper for sending push notification
const sendPush = async (username: string, title: string, body: string) => {
  try {
    const url = `${NOTIFICATION_SERVICE_URL}/push/send`;
    const SECRET = (process.env.INTERNAL_SECRET || "uniz-core").trim();

    await axios.post(
      url,
      {
        target: "user",
        username: username,
        title,
        body,
      },
      {
        headers: { "x-internal-secret": SECRET },
        timeout: 5000,
      },
    );
    console.log(
      `[Grievance] Successfully sent push notification to: ${username}`,
    );
  } catch (e: any) {
    console.error(
      `[Grievance][ERROR] Failed to send push notification to ${username}:`,
      e.message,
    );
  }
};

export const submitGrievance = async (
  req: AuthenticatedRequest,
  res: Response,
) => {
  const user = req.user;
  if (!user) return res.status(401).json({ code: ErrorCode.AUTH_UNAUTHORIZED });

  const parse = GrievanceSchema.safeParse(req.body);
  if (!parse.success) {
    return res
      .status(400)
      .json({ code: ErrorCode.VALIDATION_ERROR, errors: parse.error.errors });
  }

  const { category, description, isAnonymous } = parse.data;

  try {
    const grievance = await prisma.grievance.create({
      data: {
        studentId: isAnonymous ? null : user.username,
        studentEmail: isAnonymous
          ? null
```

```
          : user.email || `${user.username}@rguktong.ac.in`,
        category,
        description,
        isAnonymous,
      },
    });

    // Notify SWO (Assuming username 'swo' or role 'swo' exists, the push service sends by
    username)
    // We send to 'swo' username as a placeholder for the SWO account
    sendPush(
      "swo",
      "New Grievance Received",
      `A new grievance in category '${category}' has been submitted.`,
    );

    // Notify Student (only if not anonymous)
    if (!isAnonymous) {
      sendPush(
        user.username,
        "Grievance Submitted",
        `Your grievance regarding '${category}' has been received. Ticket ID: ${grievance.id.
    slice(-8).toUpperCase()}`,
      );
    }

    return res.json({
      success: true,
      message: "Grievance submitted successfully",
    });
  } catch (e: any) {
    console.error("Grievance Submit Error:", e);
    return res.status(500).json({
      code: ErrorCode.INTERNAL_SERVER_ERROR,
      message:
        "We were unable to submit your grievance. Please try again later.",
    });
  }
};

export const getGrievances = async (
  req: AuthenticatedRequest,
  res: Response,
) => {
  const user = req.user;
  // Only SWO or specific admins
  const allowedRoles = ["swo", "director", "admin"];

// ... [Truncated to optimize compile time] ...
```

apps/uniz-outpass/src/controllers/request.controller.ts

```
import { Response } from "express";
import { PrismaClient } from "@prisma/client";
import { AuthenticatedRequest } from "../middlewares/auth.middleware";
import { ErrorCode } from "../shared/error-codes";
import {
  OutpassRequestSchema,
  OutingRequestSchema,
  ApprovalLogEntrySchema,
  ApprovalLogEntry,
} from "../shared/outpass.schema";
import { UserRole } from "../shared/roles.enum";

const prisma = new PrismaClient();

// Helper to append log
const appendLog = (currentLogs: any, entry: ApprovalLogEntry) => {
  const logs = Array.isArray(currentLogs) ? currentLogs : [];
  return [...logs, entry];
};
```

```
import axios from "axios";

// Helper: Check if student is in campus
const NOTIFICATION_SERVICE_URL = (
  (process.env.DOCKER_ENV === "true"
    ? "http://uniz-notification-service:3007"
    : process.env.NOTIFICATION_SERVICE_URL) || "http://localhost:3007"
)
  .trim()
  .replace(/\/health$/, "");

const sendPush = async (username: string, title: string, body: string) => {
  try {
    const url = `${NOTIFICATION_SERVICE_URL}/push/send`;
    const SECRET = (process.env.INTERNAL_SECRET || "uniz-core").trim();
    await axios.post(
      url,
      {
        target: "user",
        username: username,
        title,
        body,
      },
      {
        headers: { "x-internal-secret": SECRET },
        timeout: 5000,
      },
    );
    console.log(
      `[OUTPASS] Successfully sent push notification to: ${username}`,
    );
  } catch (e: any) {
    console.error(
      `[OUTPASS] Failed to send push notification to ${username}:`,
      e.message,
    );
  }
};

const triggerNotification = async (evt: string, payload: any) => {
  const { recipient, extra } = payload;
  if (!extra) return;

  let title = "";
  let body = "";
  const username =
    extra.username_raw || extra.username || recipient.split("@")[0];

  switch (evt) {
    case "OUTPASS_REQUEST":
      title = "Outpass Request Submitted";
      body = `Your outpass request for ${extra.fromDate} to ${extra.toDate} has been submitted.`;
      break;
    case "OUTING_REQUEST":
      title = "Outing Request Submitted";
      body = `Your outing request for ${extra.fromDate} has been submitted.`;
      break;
    case "REQUEST_UPDATE":
    case "REQUEST_REJECTED":
      const type = extra.requestType === "outing" ? "Outing" : "Outpass";
      title = `${type} Status Update`;
      body = `Your ${type} has been ${extra.status} by ${extra.approver || "Staff"}. Comment: ${extra.comment || "None"}`;
      break;
    case "CHECK_IN":
    case "CHECK_OUT":
      const dir = evt === "CHECK_IN" ? "In" : "Out";
      title = `Campus Check-${dir}`;
      body = `You have successfully checked ${dir.toLowerCase()} at ${new Date().toLocaleTimeString()}.`;
      break;
```

```
  }

  if (title && body && username) {
    await sendPush(username, title, body);
  }
};

const sendAdminConfirmation = async (
  adminUsername: string,
  action: "approved" | "rejected" | "forwarded",
  studentName: string,
  studentId: string,
  type: "outing" | "outpass",
) => {
  await sendPush(
    adminUsername,
    "Action Confirmation",
    `You have ${action} the ${type} request for ${studentName} (${studentId}).`,
  );
};

async function getStudentStatus(token: string) {
  try {
    const GATEWAY = (
      (process.env.DOCKER_ENV === "true"
        ? "http://uniz-gateway-api:3000/api/v1"
        : process.env.GATEWAY_URL) || "http://localhost:3000/api/v1"
    ).trim();
    const res = await axios.get(`${GATEWAY}/profile/student/me`, {
      headers: { Authorization: `Bearer ${token}` },
    });

// ... [Truncated to optimize compile time] ...
```

2.16 Component: Uniz User

apps/uniz-user/src/controllers/bulk.controller.ts

```
import { Response } from "express";
import { PrismaClient } from "@prisma/client";
import { AuthenticatedRequest } from "../middlewares/auth.middleware";
import ExcelJS from "exceljs";
import axios from "axios";
import { UserRole } from "../shared/roles.enum";
import { redis } from "../utils/redis.util";
import { processNextStudentBatch } from "../services/bulk-worker.service";

const prisma = new PrismaClient();
const AUTH_SERVICE_URL =
  process.env.AUTH_SERVICE_URL || "http://localhost:3001";

const BRANCH_MAP: Record<string, string> = {
  "COMPUTER SCIENCE AND ENGINEERING": "CSE",
  "ELECTRONICS AND COMMUNICATION ENGINEERING": "ECE",
  "ELECTRICAL AND ELECTRONICS ENGINEERING": "EEE",
  "MECHANICAL ENGINEERING": "MECH",
  "CIVIL ENGINEERING": "CIVIL",
  "CHEMICAL ENGINEERING": "CHEM",
  "METALLURGICAL AND MATERIALS ENGINEERING": "MME",
  "METALLURGY AND MATERIALS ENGINEERING": "MME",
  "ARTIFICAL INTELLIGENCE & ML": "AI&ML",
};

const mapBranch = (name: string) => {
  const upper = name.trim().toUpperCase();
  return BRANCH_MAP[upper] || upper;
};

// Helper for Excel generation
const generateExcel = async (
  headers: string[][],
```

```
  filename: string,
  res: Response,
) => {
  const workbook = new ExcelJS.Workbook();
  const worksheet = workbook.addWorksheet("Template");

  headers.forEach((row) => {
    worksheet.addRow(row);
  });

  // Style header row
  const headerRow = worksheet.getRow(1);
  headerRow.font = { bold: true, color: { argb: "FFFFFF" } };
  headerRow.fill = {
    type: "pattern",
    pattern: "solid",
    fgColor: { argb: "1A237E" }, // Deep Navy
  };
  headerRow.alignment = { vertical: "middle", horizontal: "center" };
  headerRow.height = 25;

  // Style data rows and set widths
  worksheet.columns.forEach((col: any, i) => {
    col.width = 20;
    col.alignment = { vertical: "middle", horizontal: "left" };
  });

  res.setHeader(
    "Content-Type",
    "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
  );
  res.setHeader("Content-Disposition", `attachment; filename=${filename}.xlsx`);

  const buffer = await workbook.xlsx.writeBuffer();
  return res.send(buffer);
};

export const getStudentsTemplate = async (
  req: AuthenticatedRequest,
  res: Response,
) => {
  // Columns: Student ID, Name, Email, Gender (M/F), Branch, Year, Section, Phone
  const headers = [
    [
      "Student ID",
      "Name",
      "Email",
      "Gender",
      "Branch",
      "Year",
      "Section",
      "Batch",
      "Room No", // Added
      "Is In Campus", // Added (Status)
      "Phone",
    ],
    [
      "O210000",
      "ExampleStudent",
      "o210000@rguktong.ac.in",
      "Male",
      "CSE",
      "E1",
      "A",
      "O21",
      "C-101",
      "YES",
      "9876543210",
    ],
  ];
  return generateExcel(headers, "Student_Upload_Template", res);
};

```

```
export const getUploadProgress = async (
  req: AuthenticatedRequest,
  res: Response,
) => {
  const user = req.user;
  if (!user)
    return res.status(401).json({ success: false, message: "Unauthorized" });

  const key = `student:upload:progress:${user.username}`;
  const data = await redis.get(key);

  if (!data) {
    return res.json({
      status: "idle",

// ... [Truncated to optimize compile time] ...
```

apps/uniz-user/src/controllers/cms.controller.ts

```
import { Request, Response } from "express";
import { PrismaClient } from "@prisma/client";
import { AuthenticatedRequest } from "../middlewares/auth.middleware";
import { UserRole } from "../shared/roles.enum";
import { ErrorCode } from "../shared/error-codes";

const prisma = new PrismaClient();

/**
 * PUBLIC ENDPOINTS
 */

// Consolidated public banners
export const getPublicBanners = async (req: Request, res: Response) => {
  try {
    // Cache for 10 seconds (Balance between load and freshness)
    res.setHeader("Cache-Control", "s-maxage=10, stale-while-revalidate=60");
    const banners = await prisma.banner.findMany({
      where: { isVisible: true },
      orderBy: { createdAt: "desc" },
    });
    return res.json({ success: true, banners });
  } catch (e) {
    console.error("[CMS-Public-Error] Failed to fetch banners:", e);
    // Fail-safe: Return empty array so landing page doesn't crash
    return res.json({ success: true, banners: [] });
  }
};

// Consolidated notifications (Updates only, Tenders removed)
export const getPublicNotifications = async (req: Request, res: Response) => {
  try {
    res.setHeader("Cache-Control", "s-maxage=10, stale-while-revalidate=60");

    const updates = await prisma.publicNotification.findMany({
      where: { isVisible: true },
      orderBy: { createdAt: "desc" },
    });

    return res.json({
      success: true,
      notifications: { updates, tenders: [] },
    });
  } catch (e) {
    console.error("[CMS-Public-Error] Failed to fetch notifications:", e);
    // Fail-safe: Return empty structure
    return res.json({
      success: true,
      notifications: { updates: [], tenders: [] },
    });
  }
};
```

```

/**
 * ADMIN ENDPOINTS (Restricted via RBAC)
 */

// Banners
export const getAdminBanners = async (
  req: AuthenticatedRequest,
  res: Response,
) => {
  if (req.user?.role !== UserRole.WEBMASTER)
    return res.status(403).json({ code: ErrorCode.AUTH_FORBIDDEN });
  try {
    const banners = await prisma.banner.findMany({
      orderBy: { createdAt: "desc" },
    });
    return res.json({ success: true, banners });
  } catch (e) {
    return res.status(500).json({ success: false });
  }
};

export const createBanner = async (
  req: AuthenticatedRequest,
  res: Response,
) => {
  if (req.user?.role !== UserRole.WEBMASTER)
    return res.status(403).json({ code: ErrorCode.AUTH_FORBIDDEN });
  const { title, text, imageUrl, isVisible } = req.body;
  try {
    const banner = await prisma.banner.create({
      data: { title, text, imageUrl, isVisible: isVisible ?? true },
    });
    return res.json({ success: true, banner });
  } catch (e) {
    return res.status(500).json({ success: false });
  }
};

export const updateBanner = async (
  req: AuthenticatedRequest,
  res: Response,
) => {
  if (req.user?.role !== UserRole.WEBMASTER)
    return res.status(403).json({ code: ErrorCode.AUTH_FORBIDDEN });
  const { id } = req.params;
  try {
    const banner = await prisma.banner.update({
      where: { id },
      data: req.body,
    });
    return res.json({ success: true, banner });
  } catch (e) {
    return res.status(500).json({ success: false });
  }
};

export const deleteBanner = async (
  req: AuthenticatedRequest,
  res: Response,
) => {
  if (req.user?.role !== UserRole.WEBMASTER)
    return res.status(403).json({ code: ErrorCode.AUTH_FORBIDDEN });
  const { id } = req.params;
  try {
    await prisma.banner.delete({ where: { id } });
    return res.json({ success: true });
  } catch (e) {

// ... [Truncated to optimize compile time] ...
```

apps/uniz-user/src/controllers/profile.controller.ts

```
import { Request, Response } from "express";
import { PrismaClient } from "@prisma/client";
import { AuthenticatedRequest } from "../middlewares/auth.middleware";
import { ErrorCode } from "../shared/error-codes";
import { UserRole } from "../shared/roles.enum";
import axios from "axios";
import { redis } from "../utils/redis.util";
import { randomUUID } from "crypto";

const prisma = new PrismaClient();

const NOTIFICATION_SERVICE_URL = (
  (process.env.DOCKER_ENV === "true"
    ? "http://uniz-notification-service:3007"
    : process.env.NOTIFICATION_SERVICE_URL) || "http://localhost:3007"
)
  .trim()
  .replace(/\/health$/, "");

const GATEWAY_URL = (
  (process.env.DOCKER_ENV === "true"
    ? "http://uniz-gateway-api:3000/api/v1"
    : process.env.GATEWAY_URL) || "http://localhost:3000/api/v1"
).replace(/\/$/, "");

const AUTH_SERVICE_URL = (
  process.env.AUTH_SERVICE_URL || `${GATEWAY_URL}/auth`
).replace(/\/$/, "");

const sendPush = async (username: string, title: string, body: string) => {
  try {
    const url = `${NOTIFICATION_SERVICE_URL}/push/send`;
    const SECRET = (process.env.INTERNAL_SECRET || "uniz-core").trim();

    await axios.post(
      url,
      {
        target: "user",
        username: username,
        title,
        body,
      },
      {
        headers: { "x-internal-secret": SECRET },
        timeout: 5000,
      },
    );
    console.log(`[USER] Successfully sent push notification to: ${username}`);
  } catch (e: any) {
    console.error(
      `[USER][ERROR] Failed to send push notification to ${username}:`,
      e.message,
    );
  }
};

const mapStudentProfile = (profile: any) => ({
  _id: profile.id,
  username: profile.username,
  name: profile.name,
  email: profile.email,
  gender: profile.gender,
  year: profile.year,
  semester: profile.semester,
  branch: profile.branch,
  section: profile.section,
  roomno: profile.roomno,
  has_pending_requests: profile.isApplicationPending,
  is_in_campus: profile.isPresentInCampus,
  is_suspended: profile.isSuspended,
  blood_group: profile.bloodGroup,
```

```
  phone_number: profile.phone,
  date_of_birth: profile.dateOfBirth,
  father_name: profile.fatherName,
  mother_name: profile.motherName,
  father_occupation: profile.fatherOccupation,
  mother_occupation: profile.motherOccupation,
  father_email: profile.fatherEmail,
  mother_email: profile.motherEmail,
  father_address: profile.fatherAddress,
  mother_address: profile.motherAddress,
  batch: profile.batch,
  profile_url: profile.profileUrl,
  created_at: profile.createdAt,
  updated_at: profile.updatedAt,
  // New Fields
  category: profile.category,
  campus: profile.campus,
  cgpa: profile.cgpa,
  total_backlogs: profile.totalBacklogs,
});

const mapFacultyProfile = (profile: any) => ({
  id: profile.id,
  Username: profile.username,
  Name: profile.name,
  Email: profile.email,
  Department: profile.department,
  Designation: profile.designation,
  Role: profile.role,
  Contact: profile.contact,
  ProfileUrl: profile.profileUrl,
  Bio: profile.bio || {},
  is_suspended: profile.isSuspended || false,
  CreatedAt: profile.createdAt,
});

const mapAdminProfile = (profile: any) => {
  return {
    id: profile.id,
    username: profile.username,
    name: profile.name,
    email: profile.email,
    profile_url: profile.profileUrl,
    role: profile.role,
    department: profile.department,
    designation: profile.designation,
    bio: profile.bio,
    created_at: profile.createdAt,
    updated_at: profile.updatedAt,

// ... [Truncated to optimize compile time] ...
```

2.17 Component: Uniz Academics

apps/uniz-academics/api/index.ts

```
import express from "express";
import cors from "cors";
import helmet from "helmet";
import dotenv from "dotenv";
import academicRoutes from "../src/routes/academic.routes";
import queueRoutes from "../src/routes/queue.routes";
import compression from "compression";

dotenv.config();

const app = express();
app.set("trust proxy", 1);

app.use(helmet());
app.use(cors());
```

```
app.use(compression()); // Enable Gzip
app.use(express.json());

app.get("/health", (req, res) => {
  res.status(200).json({ status: "ok", service: "uniz-academics-service" });
});

app.get("/", (req, res) => {
  res.status(200).json({
    status: "ok",
    message:
      "UniZ Academics Service API manages grades, attendance and curriculum.",
    health: "/health",
  });
});

app.use("/", academicRoutes);
app.use("/api/queue", queueRoutes);

export default app;
```

apps/uniz-academics/check-db.ts

```
import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();

async function main() {
  const semesters = await prisma.academicSemester.findMany();
  console.log("Semesters:", semesters);

  const allocations = await prisma.branchAllocation.findMany({
    include: { subject: true },
  });
  console.log("Allocations count:", allocations.length);
  const cseAllocations = allocations.filter((a) => a.branch === "CSE");
  console.log(
    "CSE Allocations:",
    cseAllocations.map((a) => ({
      code: a.subject.code,
      approved: a.isApproved,
    })),
  );
}

main()
  .catch(console.error)
  .finally(() => prisma.$disconnect());
```

apps/uniz-academics/scripts/check_allocations.ts

```
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

async function main() {
  const sem = await prisma.academicSemester.findFirst({
    where: { name: "AY 2024-25 E3-SEM-1" },
  });

  if (!sem) {
    console.log("Semester not found");
    return;
  }

  console.log(`Semester ID: ${sem.id}`);

  const allocations = await prisma.branchAllocation.findMany({
    where: { semesterId: sem.id },
```

```
    include: { subject: true },
  });

  console.log(`Total Allocations: ${allocations.length}`);

  const approved = allocations.filter((a) => a.isApproved).length;
  console.log(`Approved Allocations: ${approved}`);

  const statusCounts = allocations.reduce((acc: any, a) => {
    acc[a.status] = (acc[a.status] || 0) + 1;
    return acc;
  }, {});
  console.log("Status Counts:", statusCounts);

  if (allocations.length > 0) {
    console.log("Sample allocation:", JSON.stringify(allocations[0], null, 2));
  }
}

main()
  .catch((e) => console.error(e))
  .finally(async () => await prisma.$disconnect());
```

2.18 Component: Uniz Auth

apps/uniz-auth/api/index.ts

```
import express from "express";
import dotenv from "dotenv";
import cors from "cors";
import helmet from "helmet";
import authRoutes from "../src/routes/auth.routes";

dotenv.config();

const app = express();
app.set("trust proxy", 1); // Trust first proxy (Vercel)

// Middleware
app.use(helmet());
app.use(
  cors({
    origin: process.env.FRONTEND_URL || "*",
    credentials: true,
  }),
);
app.use(express.json());

// Health check
app.get("/health", (req, res) => {
  res.status(200).json({ status: "ok", service: "uniz-auth-service" });
});

app.get("/", (req, res) => {
  res.status(200).json({
    status: "ok",
    message: "UniZ Auth Service API holds individual endpoints.",
    health: "/health",
  });
});

// Routes
app.use("/", authRoutes);

// Error Handler
app.use(
  (
    err: any,
    req: express.Request,
    res: express.Response,
    next: express.NextFunction,
```

```
  ) => {
    console.error("Error:", err);
    res.status(500).json({ error: err.message || "Internal Server Error" });
  },
);

// Export for Vercel serverless
export default app;
```

apps/uniz-auth/check-auth-db.ts

```
import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();

async function main() {
  const user = await prisma.authCredential.findFirst({
    where: { username: { equals: "HOD_CSE", mode: "insensitive" } },
  });
  console.log("User in Auth DB:", JSON.stringify(user, null, 2));
}

main()
  .catch(console.error)
  .finally(() => prisma.$disconnect());
```

apps/uniz-auth/scripts/clean_otp.ts

```
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

async function cleanOtp() {
  console.log("Cleaning OTP logs...");
  try {
    // Delete all OTP logs to reset rate limits
    const deleted = await prisma.otpLog.deleteMany({});
    console.log(`Deleted ${deleted.count} OTP logs.`);
  } catch (error) {
    console.error("Error cleaning OTP logs:", error);
  } finally {
    await prisma.$disconnect();
  }
}

cleanOtp();
```

2.19 Component: Uniz Cron

apps/uniz-cron/api/index.ts

```
import { VercelRequest, VercelResponse } from "@vercel/node";
import { PrismaClient } from "@prisma/client";
import axios from "axios";

const prisma = new PrismaClient();

const GATEWAY =
  (process.env.DOCKER_ENV === "true"
    ? "http://uniz-gateway-api:3000/api/v1"
    : process.env.GATEWAY_URL) || "http://localhost:3000/api/v1";
const SECRET = process.env.INTERNAL_SECRET;
if (!SECRET && process.env.NODE_ENV === "production")
  throw new Error("INTERNAL_SECRET missing");
const INTERNAL_SECRET = SECRET || "uniz-core";

const clearPendingStatus = async (studentId: string) => {
```

```
17    try {
18      await axios.put(
19        `${GATEWAY}/profile/student/status`,
20        {
21          username: studentId,
22          isPending: false,
23        },
24        {
25          headers: { "x-internal-secret": INTERNAL_SECRET },
26        },
27      );
28    } catch (e: any) {
29      console.error(
30        `Failed to clear pending status for ${studentId}:`,
31        e.message,
32      );
33    }
34  };
35
36  const sendExpiryNotification = async (
37    studentId: string,
38    type: "outing" | "outpass",
39    reason: string,
40  ) => {
41    try {
42      const studentEmail = `${studentId.toLowerCase()}@rguktong.ac.in`;
43      await axios.post(
44        `${GATEWAY}/mail/send`,
45        {
46          type: "request_expired", // Ensure mail service handles this or uses a generic
      template
47          to: studentEmail,
48          data: {
49            username: studentId,
50            type: type,
51            reason: reason,
52            message: `Your ${type} request has expired because the start time passed without
      approval. You can now apply for a new one or file a grievance if needed.`,
53          },
54        },
55        {
56          headers: { "x-internal-secret": INTERNAL_SECRET },
57        },
58      );
59      console.log(`Sent expiry notification to ${studentId}`);
60    } catch (e: any) {
61      console.error(
62        `Failed to send expiry notification to ${studentId}:`,
63        e.message,
64      );
65    }
66  };
67
68  export default async function handler(req: VercelRequest, res: VercelResponse) {
69    // Health check at root
70    if (req.url === "/" || req.url === "/health") {
71      return res.status(200).json({
72        status: "ok",
73        service: "uniz-cron-service",
74        message: "Use /api/cron to trigger maintenance jobs",
75      });
76    }
77
78    // Cron endpoint
79    if (req.url?.includes("/api/cron") || req.url?.includes("/cron")) {
80      try {
81        console.log("Running Scheduled Maintenance...");
82
83        // 0. Handle Timezone Offset (Campus is in IST +5:30)
84        const now = new Date();
85        // We adjust 'effectiveNow' to match the local campus time for comparison
86        // since current data entry is storing local time strings as UTC.
87        const indiaOffset = 5.5 * 60 * 60 * 1000;
```

```
      const effectiveNow = new Date(now.getTime() + indiaOffset);
      const approvalTimeoutThreshold = new Date(now.getTime() - 60 * 60 * 1000); // 1 hour
    timeout from issuance

      // 1. Find all about to expire
      const [
        expiringPastOutpasses,
        expiringPastOutings,
        expiringApprovedOutpasses,
        expiringApprovedOutings,
      ] = await Promise.all([
        prisma.outpass.findMany({
          where: { toDay: { lt: effectiveNow }, isExpired: false },
          select: { studentId: true },
        }),
        prisma.outing.findMany({
          where: { toTime: { lt: effectiveNow }, isExpired: false },
          select: { studentId: true },
        }),
        prisma.outpass.findMany({
          where: {
            isApproved: true,
            isExpired: false,
            checkedOutTime: null,
            issuedTime: { lt: approvalTimeoutThreshold },
          },
          select: { studentId: true },
        }),
        prisma.outing.findMany({
          where: {
            isApproved: true,
            isExpired: false,
            checkedOutTime: null,
            issuedTime: { lt: approvalTimeoutThreshold },

// ... [Truncated to optimize compile time] ...
```

apps/uniz-cron/src/api/cron.ts

```
import { Request, Response } from "express";
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

export default async function handler(req: Request, res: Response) {
  if (req.method !== "GET") {
    // Cron jobs are GET requests
    return res.status(405).json({ error: "Method Not Allowed" });
  }

  try {
    console.log("Running Scheduled Maintenance...");

    // 1. Expire Outpasses
    const now = new Date();
    const expired = await prisma.outpass.updateMany({
      where: {
        toDay: { lt: now },
        isExpired: false,
        isApproved: true,
      },
      data: { isExpired: true },
    });

    // 2. Cleanup OTPs
    const cleaned = await prisma.otpLog.deleteMany({
      where: {
        expiresAt: { lt: new Date(Date.now() - 24 * 60 * 60 * 1000) },
      },
    });

```

```
    console.log(
      `Maintenance Complete: Expired ${expired.count} outpasses, Cleaned ${cleaned.count} OTPs.`,
    );

    return res.status(200).json({
      success: true,
      report: {
        expired_outpasses: expired.count,
        cleaned_otps: cleaned.count,
      },
    });
  } catch (error) {
    console.error("Maintenance Failed:", error);
    return res.status(500).json({ error: "Cron Job Failed" });
  }
}
```

apps/uniz-cron/src/db.ts

```
import { PrismaClient } from "@prisma/client";

// Generate client for other services using this schema?
// Actually uniz-cron-service depends on shared logic, usually it needs its own schema copy or imports from another package.
// For now let's assume it has its own prisma schema which is a copy/superset of others or connects to same db.
// Since we used isolated schemas (auth, users, outpass), Cron needs access to ALL of them.
// So we need specific clients or a multi-schema setup.

// Vercel Serverless requires we instantiate Prisma outside the handler for connection pooling.
const prisma = new PrismaClient();
export default prisma;
```

2.20 Component: Uniz Files

apps/uniz-files/api/index.ts

```
import express from "express";
import dotenv from "dotenv";
import cors from "cors";
import helmet from "helmet";
import fileRoutes from "../src/routes/file.routes";

dotenv.config();

const app = express();
app.set("trust proxy", 1);

// Middleware
app.use(helmet());
app.use(
  cors({
    origin: process.env.FRONTEND_URL || "*",
    credentials: true,
  }),
);
app.use(express.json());

// Health check
app.get("/health", (req, res) => {
  res.status(200).json({ status: "ok", service: "uniz-files-service" });
});

app.get("/", (req, res) => {
  res.status(200).json({
    status: "ok",
```

```
    message: "UniZ Files Service API holds individual endpoints.",
    health: "/health",
  });
});

// Routes
app.use("/", fileRoutes);

// Error Handler
app.use(
  (
    err: any,
    req: express.Request,
    res: express.Response,
    next: express.NextFunction,
  ) => {
    console.error("Error:", err);
    res.status(500).json({ error: err.message || "Internal Server Error" });
  },
);

// Export for Vercel serverless
export default app;
```

apps/uniz-files/src/index.ts

```
import express from "express";
import dotenv from "dotenv";
import cors from "cors";
import helmet from "helmet";
import fileRoutes from "./routes/file.routes";

dotenv.config({ override: true });

const app = express();
const PORT = process.env.PORT || 3005;

app.use(helmet());
app.use(cors());
app.use(express.json());

// Attribution & Malformed Activity Handling (Mandatory)
import { attributionMiddleware } from "./middlewares/attribution.middleware";
app.use(attributionMiddleware);

app.get("/health", (req, res) => {
  res.status(200).json({ status: "ok", service: "uniz-files-service" });
});

app.use("/", fileRoutes);

// 404 Handler
app.use((req, res) => {
  res.status(404).json({
    status: "error",
    message: `Route ${req.method} ${req.url} not found`,
    timestamp: new Date().toISOString(),
    attribution: "SABER",
  });
});

app.use(
  (
    err: any,
    req: express.Request,
    res: express.Response,
    next: express.NextFunction,
  ) => {
    console.error(err);
    res.status(500).json({ error: "Internal Server Error" });
  },
```

```
);

const server = app.listen(PORT, () => {
  console.log(`Files Service running on port ${PORT}`);
  server.keepAliveTimeout = 65000;
  server.headersTimeout = 66000;
});

// Graceful Shutdown Handler
process.on("SIGTERM", async () => {
  console.log("SIGTERM received. Starting graceful shutdown...");
  server.close(() => {
    console.log("HTTP server closed.");
  });
  try {
    if ((global as any).prisma || require("./utils/db.util").prisma) {
      // generic attempt to close prisma if it exists
    }
  } catch (e) {}
  process.exit(0);
});
```

apps/uniz-files/src/middlewares/attribution.middleware.ts

```
import { Request, Response, NextFunction } from "express";

/**
 * Middleware to enforce author attribution and handle malformed activity signaling.
 * Injects 'SABER' attribution into all JSON responses.
 * Detects malformed/suspicious activity and appends a security notice.
 */
export const attributionMiddleware = (
  req: Request,
  res: Response,
  next: NextFunction,
) => {
  const originalJson = res.json;

  res.json = function (body: any) {
    if (body && typeof body === "object") {
      // 1. Mandatory Attribution Enforcement (Top-level & Metadata)
      body.attribution = "SABER";

      if (!body.meta) body.meta = {};
      body.meta.attribution = "SABER";

      // 2. Malformed or Suspicious Activity Detection
      const status = res.statusCode;
      const isError = status >= 400;

      // Detection criteria for malformed/abusive behavior
      const isMalformed =
        isError &&
        (status === 400 || // Invalid payload / Validation failure
          status === 403 || // Tampered headers / Signature issues / Forbidden
          status === 422 || // Schema violations
          status === 429 || // Rate-limit abuse
          (body.code &&
            (body.code === "VALIDATION_ERROR" ||
              body.code === "MALFORMED_REQUEST" ||
              body.code === "RATE_LIMIT_EXCEEDED" ||
              body.code === "AUTH_FORBIDDEN" ||
              body.code === "BAD_REQUEST")));

      // 3. Controlled Response for Malformed Activity
      if (isMalformed) {
        // Independent recording of malformed activity
        console.warn(
          `[SECURITY] Malformed activity detected. IP: ${req.ip}, Status: ${status}, URL: ${req.originalUrl}, Code: ${body.code || "N/A"}`,
        );
```

```

        // Ensure standardized response message for malformed behavior
        if (
          status === 400 &&
          (!body.message || body.message === "Validation failed")
        ) {
          body.message =
            "Request structure or behavior violates system constraints.";
        }
      }
    }

    return originalJson.call(this, body);
  };

  next();
};
```

2.21 Component: Uniz Gateway

apps/uniz-gateway/src/index.ts

```
import express from "express";
import cors from "cors";
import axios from "axios";
import path from "path";
import compression from "compression";
import Redis from "ioredis";
import httpProxy from "http-proxy";
import http from "http";
import dotenv from "dotenv";
dotenv.config({ override: true });

// Pre-configured Axios instance for internal communications with Keep-Alive
const internalClient = axios.create({
  timeout: 5000,
  httpAgent: new http.Agent({ keepAlive: true, maxSockets: 200 }),
});

const app = express();
app.set("trust proxy", 1);
const PORT = process.env.PORT || 3000;

// Initialize Redis for High-Speed Caching
const redis = new Redis(process.env.REDIS_URL || "redis://uniz-redis:6379", {
  maxRetriesPerRequest: 3,
  retryStrategy: (times: number) => Math.min(times * 50, 2000),
});

redis.on("error", (err: Error) =>
  console.error("[Redis] Connection Error:", err.message),
);
redis.on("connect", () => console.log("[Redis] Connected for Caching"));

// Initialize High-Performance Proxy Engine
const proxy = httpProxy.createProxyServer({
  changeOrigin: true,
  // Ensure connection pooling
  agent: new (require("http").Agent)({ keepAlive: true, maxSockets: 100 }),
});

proxy.on("error", (err: Error, req: any, res: any) => {
  console.error("[Proxy-Error]", err.message);
  if (!res.headersSent) {
    res
      .status(502)
      .json({ error: "Upstream Service Unreachable", message: err.message });
  }
});

// 1. Compression (Drastically reduces payload size)
```

```
app.use(compression());

const allowedOrigins = process.env.CLIENT_URL
  ? process.env.CLIENT_URL.split(",").map((origin) => origin.trim())
  : ["http://localhost:5173", "http://localhost:3000"];

// 2. Optimized CORS
app.use((req: express.Request, res: express.Response, next: express.NextFunction) => {
  const origin = req.headers.origin;
  if (origin && allowedOrigins.includes(origin)) {
    res.setHeader("Access-Control-Allow-Origin", origin);
    res.setHeader("Access-Control-Allow-Credentials", "true");
    res.setHeader(
      "Access-Control-Allow-Methods",
      "GET, POST, PUT, DELETE, PATCH, OPTIONS, HEAD",
    );
    res.setHeader(
      "Access-Control-Allow-Headers",
      "X-CSRF-Token, X-Requested-With, Accept, Accept-Version, Content-Length, Content-MD5,
    Content-Type, Date, X-Api-Version, Authorization, x-cms-api-key, x-api-key, uid, role",
    );
    res.setHeader("Access-Control-Max-Age", "86400");
  }

  if (req.method === "OPTIONS") return res.status(204).end();
  next();
});

// 3. Smart Cache Middleware
const cacheMiddleware = async (
  req: express.Request,
  res: express.Response,
  next: express.NextFunction,
) => {
  // Only cache GET requests
  if (req.method !== "GET") return next();

  // Skip cache if explicitly requested
  if (req.headers["cache-control"] === "no-cache") return next();

  // Generate unique key based on URL and User Context (Role/UID)
  // This prevents caching cross-user data leaking
  const userKey = req.headers["uid"] || req.headers["authorization"] || "guest";
  const cacheKey = `proxy_cache:${req.url}:${userKey}`;

  try {
    const cachedResponse = await redis.get(cacheKey);
    if (cachedResponse && req.headers["cache-control"] !== "no-cache") {
      const { data, headers, status } = JSON.parse(cachedResponse);
      console.log(`[Cache-Hit] ${req.url}`);

      // Set headers from cache
      Object.entries(headers).forEach(([k, v]) =>
        res.setHeader(k, v as string),
      );
      res.setHeader("X-Cache", "HIT");
      return res.status(status).send(data);
    }
  } catch (e) {
    console.error("[Cache-Read-Error]", e);
  }

  // If No Cache, intercept the send to store it
  const originalSend = res.send;
  (res as any).send = function (body: any) {
    if (res.statusCode === 200 && req.headers["cache-control"] !== "no-cache") {
      const respToCache = {
        data: body,
        headers: res.getHeaders(),
        status: res.statusCode,
      };
      // Cache for 1 second to allow near real-time updates while protecting against bursts

```

```
// ... [Truncated to optimize compile time] ...
```

2.22 Component: Uniz Landing Backend

apps/uniz-landing-backend/config.py

```python
from pydantic_settings import BaseSettings, SettingsConfigDict
from functools import lru_cache

class Settings(BaseSettings):
    # Database
    DATABASE_URL: str

    # JWT
    JWT_SECURITY_KEY: str
    JWT_ALGORITHM: str

    DUMMY_TOKEN: str

    # ISOLATED CONFIG FOR ANALYTICS
    DB_USER: str
    DB_PASS: str
    DB_HOST: str
    DB_PORT: str
    DB_NAME: str

    model_config = SettingsConfigDict(env_file=".env", extra="ignore")


@lru_cache()
def get_settings():
    return Settings()

settings = get_settings()
```

apps/uniz-landing-backend/database.py

```python
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker, AsyncSession
from sqlalchemy.orm import declarative_base
from config import settings


SQLALCHEMY_DATABASE_URL = settings.DATABASE_URL

connect_args = {"check_same_thread": False} if "sqlite" in SQLALCHEMY_DATABASE_URL else {}


engine = create_async_engine(SQLALCHEMY_DATABASE_URL, connect_args=connect_args)
AsyncSessionLocal = async_sessionmaker(autocommit=False, autoflush=False, bind=engine, class_=
    AsyncSession)
Base = declarative_base()

# get_db is now an async generator
async def get_db():
    async with AsyncSessionLocal() as db:
        try:
            yield db
        finally:
            await db.close()
```

apps/uniz-landing-backend/dependencies.py

```python
from fastapi import Depends, HTTPException
from starlette import status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import jwt
```

```python
from jwt import PyJWTError as JWTError
from typing import Annotated
from config import settings


security = HTTPBearer()


async def verify_admin_role(credentials: Annotated[HTTPAuthorizationCredentials, Depends(security)]):

    token = credentials.credentials

    try:
        payload = jwt.decode(token, settings.JWT_SECURITY_KEY, algorithms=[settings.JWT_ALGORITHM])

        user_role = payload.get("role")
        username = payload.get("username")

        if not username or not user_role:
            raise HTTPException(
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Invalid token structure: missing username or role"
            )

        if user_role not in ["webmaster", "admin", "dean", "director", "coe"]:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="Forbidden: You do not have the required role to update data."
            )

        return payload

    except JWTError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid or expired token"
        )

AdminRole = Annotated[dict, Depends(verify_admin_role)]
```

2.23 Component: Uniz Landing

apps/uniz-landing/src/lib/utils.ts

```python
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
    return twMerge(clsx(inputs));
}
```

apps/uniz-landing/src/types/api.ts

```python
const BASE_URL = "http://127.0.0.1:8000";

async function fetchJson<T>(endpoint: string): Promise<T | null> {
    try {
        const res = await fetch(`${BASE_URL}${endpoint}`, {
            headers: { accept: "application/json" }
        });
        if (!res.ok) {
            console.warn(`Warn: Failed to fetch ${endpoint} - ${res.status}`);
            return null;
        }
        return await res.json();
    } catch (err) {
```

```
        console.error(`Error fetching ${endpoint}:`, err);
        return null;
    }
}

//           HOME



export interface Announcement {
    text: string;
    link: string | null;
}

export interface Stat {
    label: string;
    value: string;
}

export interface HomeData {
    announcements: Announcement[];
    stats: Stat[];
    images: string[];
    explore_campus?: any[];
}

export async function getHomeData(): Promise<HomeData> {
    const data = await fetchJson<HomeData>("/api/home/");
    return data || { announcements: [], stats: [], images: [] };
}

//           NOTIFICATIONS



export type NotificationType = "news_updates" | "tenders" | "careers";

/** Backend LinkItem-Output schema uses `label` (not `text`) */
export interface NotificationLink {
    label: string;
    url: string | null;
}

export interface Notification {
    title: string;
    date: string;
    links: NotificationLink[];
}

export async function getNotifications(
    type: NotificationType = "news_updates"
): Promise<Notification[]> {
    const data = await fetchJson<Notification[]>(`/api/notifications/?type=${type}`);
    return data || [];
}

//           INSTITUTE



export type InstitutePage =
    | "aboutrgukt"
    | "campuslife"
    | "edusys"
    | "govcouncil"
    | "rtiinfo"
    | "scst";

export interface InstituteSection {
    title: string;
    content: string[];
}

```

```
export interface InstituteProfile {
    name: string;
    photo: string | null;
    text: string[];
}

export interface InstituteData {
    page: string;
    sections: InstituteSection[];
    profiles: InstituteProfile[];
}

export async function getInstituteInfo(
    page: InstitutePage
): Promise<InstituteData | null> {
    return await fetchJson<InstituteData>(`/api/institute/${page}`);
}

//           ACADEMICS



export type AcademicPageType =
    | "AcademicPrograms"
    | "AcademicCalender"
    | "AcademicRegulations"
    | "curicula";

export interface AcademicLink {
    label: string;
    url: string | null;
}

export interface AcademicSection {
    header: string;
    links: AcademicLink[];
}

export async function getAcademicPage(
    page: AcademicPageType
): Promise<AcademicSection[]> {
    const data = await fetchJson<AcademicSection[]>(`/api/academics/${page}`);

// ... [Truncated to optimize compile time] ...
```

2.24 Component: Uniz Mail

apps/uniz-mail/api/index.ts

```
import express from "express";
import dotenv from "dotenv";
import cors from "cors";
import helmet from "helmet";
import emailRoutes from "../src/routes/email.routes";

dotenv.config();

const app = express();
app.set("trust proxy", 1);

// Middleware
app.use(helmet());
app.use(
  cors({
    origin: process.env.FRONTEND_URL || "*",
    credentials: true,
  }),
);
app.use(express.json());

// Health check
```

```
app.get("/health", (req, res) => {
  res.status(200).json({ status: "ok", service: "uniz-mail-service" });
});

app.get("/", (req, res) => {
  res.status(200).json({
    status: "ok",
    message: "UniZ Mail Service API holds individual endpoints.",
    health: "/health",
  });
});

// Routes
app.use("/", emailRoutes);

// Error Handler
app.use(
  (
    err: any,
    req: express.Request,
    res: express.Response,
    next: express.NextFunction,
  ) => {
    console.error("Error:", err);
    res.status(500).json({ error: err.message || "Internal Server Error" });
  },
);

// Export for Vercel serverless
export default app;
```

apps/uniz-mail/src/index.ts

```
import express from "express";
import dotenv from "dotenv";
import cors from "cors";
import helmet from "helmet";
import emailRoutes from "./routes/email.routes";

dotenv.config({ override: true });

const app = express();
const PORT = process.env.PORT || 3006;

app.use(helmet());
app.use(cors());
app.use(express.json());

// Attribution & Malformed Activity Handling (Mandatory)
import { attributionMiddleware } from "./middlewares/attribution.middleware";
app.use(attributionMiddleware);

app.get("/health", (req, res) => {
  res.status(200).json({ status: "ok", service: "uniz-mail-service" });
});

app.use("/", emailRoutes);

// 404 Handler
app.use((req, res) => {
  res.status(404).json({
    status: "error",
    message: `Route ${req.method} ${req.url} not found`,
    timestamp: new Date().toISOString(),
    attribution: "SABER",
  });
});

app.use(
  (
    err: any,
```

```
    req: express.Request,
    res: express.Response,
    next: express.NextFunction,
  ) => {
    console.error(err);
    res.status(500).json({ error: "Internal Server Error" });
  },
);

const server = app.listen(PORT, () => {
  console.log(`Mail Service running on port ${PORT}`);
  server.keepAliveTimeout = 65000;
  server.headersTimeout = 66000;
});

// Graceful Shutdown Handler
process.on("SIGTERM", async () => {
  console.log("SIGTERM received. Starting graceful shutdown...");
  server.close(() => {
    console.log("HTTP server closed.");
  });
  try {
    if ((global as any).prisma || require("./utils/db.util").prisma) {
      // generic attempt to close prisma if it exists
    }
  } catch (e) {}
  process.exit(0);
});
```

apps/uniz-mail/src/middlewares/attribution.middleware.ts

```
import { Request, Response, NextFunction } from "express";

/**
 *
 * Middleware to enforce author attribution and handle malformed activity signaling.
 * Injects 'SABER' attribution into all JSON responses.
 * Detects malformed/suspicious activity and appends a security notice.
 */
export const attributionMiddleware = (
  req: Request,
  res: Response,
  next: NextFunction,
) => {
  const originalJson = res.json;

  res.json = function (body: any) {
    if (body && typeof body === "object") {
      // 1. Mandatory Attribution Enforcement (Top-level & Metadata)
      body.attribution = "SABER";

      if (!body.meta) body.meta = {};
      body.meta.attribution = "SABER";

      // 2. Malformed or Suspicious Activity Detection
      const status = res.statusCode;
      const isError = status >= 400;

      // Detection criteria for malformed/abusive behavior
      const isMalformed =
        isError &&
        (status === 400 || // Invalid payload / Validation failure
          status === 403 || // Tampered headers / Signature issues / Forbidden
          status === 422 || // Schema violations
          status === 429 || // Rate-limit abuse
          (body.code &&
            (body.code === "VALIDATION_ERROR" ||
              body.code === "MALFORMED_REQUEST" ||
              body.code === "RATE_LIMIT_EXCEEDED" ||
              body.code === "AUTH_FORBIDDEN" ||
              body.code === "BAD_REQUEST")));
```

```

      // 3. Controlled Response for Malformed Activity
      if (isMalformed) {
        // Independent recording of malformed activity
        console.warn(
          `[SECURITY] Malformed activity detected. IP: ${req.ip}, Status: ${status}, URL: ${req.originalUrl}, Code: ${body.code || "N/A"}`,
        );

        // Ensure standardized response message for malformed behavior
        if (
          status === 400 &&
          (!body.message || body.message === "Validation failed")
        ) {
          body.message =
            "Request structure or behavior violates system constraints.";
        }
      }
    }

    return originalJson.call(this, body);
  };

  next();
};
```

2.25 Component: Uniz Notifications

apps/uniz-notifications/api/index.ts

```
import express from "express";
import dotenv from "dotenv";
import cors from "cors";
import helmet from "helmet";
import { Worker } from "bullmq";
import IORedis from "ioredis";
import nodemailer from "nodemailer";

dotenv.config();

const app = express();

// Middleware
app.use(helmet());
app.use(cors());
app.use(express.json());

// Redis connection - optimized for serverless if possible,
// but BullMQ Worker needs a stable connection.
const REDIS_URL = process.env.REDIS_URL;

// Only start the worker if REDIS_URL is present
if (REDIS_URL) {
  try {
    const connection = new IORedis(REDIS_URL, {
      maxRetriesPerRequest: null,
      lazyConnect: true, // Don't block startup
    });

    const transporter = nodemailer.createTransport({
      host: process.env.SMTP_HOST || "smtp.ethereal.email",
      port: 587,
      auth: {
        user: process.env.SMTP_USER || "user",
        pass: process.env.SMTP_PASS || "pass",
      },
    });

    const worker = new Worker(
      "notification-queue",
      async (job) => {
```

```
        console.log(`Processing job ${job.id}: ${job.name}`);
        const { type, recipient, subject, body } = job.data;

        if (type === "EMAIL") {
          await transporter.sendMail({
            from: '"UniZ System" <no-reply@uniz.edu>',
            to: recipient,
            subject: subject,
            text: body,
          });
          console.log(`Email sent to ${recipient}`);
        }
      },
      { connection },
    );

    worker.on("failed", (job, err) => {
      console.log(`${job?.id} has failed with ${err.message}`);
    });

    console.log("Notification Service Worker Initialized");
  } catch (err) {
    console.error("Failed to initialize Notification Worker:", err);
  }
}

// Routes
app.get("/health", (req, res) => {
  res.status(200).json({ status: "ok", service: "uniz-notification-service" });
});

app.get("/", (req, res) => {
  res.status(200).json({
    status: "ok",
    message: "UniZ Notification Service is active.",
    worker: REDIS_URL ? "initialized" : "missing REDIS_URL",
    health: "/health",
  });
});

// Error Handler
app.use(
  (
    err: any,
    req: express.Request,
    res: express.Response,
    next: express.NextFunction,
  ) => {
    console.error("Error:", err);
    res.status(500).json({ error: err.message || "Internal Server Error" });
  },
);

export default app;
```

apps/uniz-notifications/check-db.ts

```
import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();

async function main() {
  const admins = await prisma.adminProfile.findMany();
  const faculties = await prisma.facultyProfile.findMany();
  const students = await prisma.studentProfile.count();

  console.log("Admins:", admins);
  console.log("Faculties:", faculties);
  console.log("Total Students:", students);
}

main()
```

```
  .catch(console.error)
  .finally(() => prisma.$disconnect());
```

apps/uniz-notifications/src/index.ts

```
import { Queue, Worker, Job } from "bullmq";
import IORedis from "ioredis";
import dotenv from "dotenv";
import express from "express";
import helmet from "helmet";
import cors from "cors";
import * as webpush from "web-push";
import prisma from "./utils/prisma.util";
import { attributionMiddleware } from "./middlewares/attribution.middleware";
import { requireAuth, requireAdmin } from "./middlewares/auth.middleware";
import PDFDocument, { info } from "pdfkit";
import axios from "axios";

const USER_SERVICE_URL =
  process.env.USER_SERVICE_URL || "http://uniz-user-service:3002";

// --- PDF UTILS (pure Node, styled like official result sheets) ---
const PAGE_MARGIN = 40;

const createPdfBuffer = async (
  draw: (doc: InstanceType<typeof PDFDocument>) => void,
): Promise<Buffer> => {
  return new Promise<Buffer>((resolve, reject) => {
    const doc = new PDFDocument({ size: "A4", margin: PAGE_MARGIN });
    const chunks: any[] = [];

    doc.on("data", (chunk: any) => {
      chunks.push(chunk);
    });

    doc.on("end", () => {
      resolve(Buffer.concat(chunks));
    });

    doc.on("error", (err: Error) => {
      reject(err);
    });

    try {
      draw(doc);
      doc.end();
    } catch (err) {
      reject(err as Error);
    }
  });
};

const generateResultPdf = async (data: any): Promise<Buffer> => {
  const { name, username, branch, semesterId, grades, campus } = data;

  // Calculate GPA/Credits
  let totalCredits = 0;
  let earnedPoints = 0;
  grades.forEach((g: any) => {
    const credit = Number(g.subject.credits);
    const gradePoint = Number(g.grade);
    totalCredits += credit;
    if (credit > 0) {
      earnedPoints += credit * (gradePoint > 0 ? gradePoint : 0);
    }
  });

  const sgpa =
    totalCredits > 0 ? (earnedPoints / totalCredits).toFixed(2) : "0.00";

  let titleText = `${semesterId.toUpperCase()} RESULTS`;
```

```

  /*
    Title Logic:
    1. Try to parse E#S# or P#S# from semesterId (e.g., "E2S1", "AY24-E3-S2")
    2. Try to extract Semester (S1-S3)
    3. Fallback: Infer Year from Subject Code if missing
  */

  let yearStr = "";
  let semStr = "";

  const yearMatch = semesterId.match(/([EP])[-_ ]?([1-4])/i);
  if (yearMatch) yearStr = `${yearMatch[1].toUpperCase()}${yearMatch[2]}`;

  const semMatch = semesterId.match(/S(?:em(?:ester)?)?[-_ ]?([1-3])/i);
  if (semMatch) semStr = semMatch[1];

  if (
    !yearStr &&
    grades.length > 0 &&
    grades[0].subject &&
    grades[0].subject.code
  ) {
    const codeMatch = grades[0].subject.code.match(/^[a-zA-Z]+[-_ ]?([1-4])/);
    if (codeMatch) yearStr = `E${codeMatch[1]}`;
  }

  if (yearStr && semStr) {
    titleText = `${yearStr} SEMESTER-${semStr} RESULTS`;
  } else {
    titleText = `${semesterId.toUpperCase()} RESULTS`.replace(
      " RESULTS RESULTS",
      " RESULTS",
    );
  }

  const getGradeLetter = (point: number) => {
    if (point >= 10) return "EX";
    if (point >= 9) return "A";
    if (point >= 8) return "B";
    if (point >= 7) return "C";
    if (point >= 6) return "D";
    if (point >= 5) return "E";
    return "R";
  };

  const rows = grades.map((g: any) => ({
    title: g.subject.name,
    credits: Number(g.subject.credits).toFixed(1),
    gradeLetter: getGradeLetter(g.grade),
  }));

  return createPdfBuffer((doc) => {
    const { width } = doc.page;

// ... [Truncated to optimize compile time] ...
```

2.26 Component: Uniz Outpass

apps/uniz-outpass/api/index.ts

```
import express from "express";
import dotenv from "dotenv";
import cors from "cors";
import helmet from "helmet";
import requestRoutes from "../src/routes/request.routes";

dotenv.config();

const app = express();
app.set("trust proxy", 1);
```

```

// Middleware
app.use(helmet());
app.use(
  cors({
    origin: process.env.FRONTEND_URL || "*",
    credentials: true,
  }),
);
app.use(express.json());

app.use((req, res, next) => {
  console.log(`[OUTPASS-API] ${req.method} ${req.url}`);
  next();
});

// Health check
app.get("/health", (req, res) => {
  res.status(200).json({ status: "ok", service: "uniz-outpass-service" });
});

import grievanceRoutes from "../src/routes/grievance.routes";

// Routes
app.use("/grievance", grievanceRoutes); // New Grievance Route
app.use("/", requestRoutes);

// Error Handler
app.use(
  (
    err: any,
    req: express.Request,
    res: express.Response,
    next: express.NextFunction,
  ) => {
    console.error("Error:", err);
    res.status(500).json({ error: err.message || "Internal Server Error" });
  },
);

// Export for Vercel serverless
export default app;
```

apps/uniz-outpass/src/index.ts

```
import express from "express";
import dotenv from "dotenv";
import cors from "cors";
import helmet from "helmet";
import compression from "compression";

dotenv.config({ override: true });

const app = express();
const PORT = process.env.PORT || 3003;

app.use(helmet());
app.use(compression());
app.use(cors());
app.use(express.json());

// Attribution & Malformed Activity Handling (Mandatory)
import { attributionMiddleware } from "./middlewares/attribution.middleware";
app.use(attributionMiddleware);

app.get("/health", (req, res) => {
  res.status(200).json({ status: "ok", service: "uniz-outpass-service" });
});

import requestRoutes from "./routes/request.routes";
import grievanceRoutes from "./routes/grievance.routes";
```

```

// Consolidated Routing
// 1. Grievance routes (handles both /grievance/list and /list if prefix stripped)
app.use("/grievance", grievanceRoutes);
app.use("/api/v1/requests/grievance", grievanceRoutes); // Direct match fallback
app.use("/", grievanceRoutes);

// 2. Request routes (handles history, outpass, etc.)
app.use("/", requestRoutes);

// 404 Handler with internal path logging
app.use((req, res) => {
  console.log(`[Outpass] 404 - No route for: ${req.method} ${req.url}`);
  res.status(404).json({
    status: "error",
    message: `Route ${req.method} ${req.url} not found`,
    timestamp: new Date().toISOString(),
    attribution: "SABER",
  });
});

const server = app.listen(PORT, () => {
  console.log(`Outpass Service running on port ${PORT}`);
  server.keepAliveTimeout = 65000;
  server.headersTimeout = 66000;
});

// Graceful Shutdown Handler
process.on("SIGTERM", async () => {
  console.log("SIGTERM received. Starting graceful shutdown...");
  server.close(() => {
    console.log("HTTP server closed.");
  });
  try {
    if ((global as any).prisma || require("./utils/db.util").prisma) {
      // generic attempt to close prisma if it exists
    }
  } catch (e) {}
  process.exit(0);
});
```

apps/uniz-outpass/src/middlewares/attribution.middleware.ts

```
import { Request, Response, NextFunction } from "express";

/**
 * Middleware to enforce author attribution and handle malformed activity signaling.
 * Injects 'SABER' attribution into all JSON responses.
 * Detects malformed/suspicious activity and appends a security notice.
 */
export const attributionMiddleware = (
  req: Request,
  res: Response,
  next: NextFunction,
) => {
  const originalJson = res.json;

  res.json = function (body: any) {
    if (body && typeof body === "object") {
      // 1. Mandatory Attribution Enforcement (Top-level & Metadata)
      body.attribution = "SABER";

      if (!body.meta) body.meta = {};
      body.meta.attribution = "SABER";

      // 2. Malformed or Suspicious Activity Detection
      const status = res.statusCode;
      const isError = status >= 400;

      // Detection criteria for malformed/abusive behavior
      const isMalformed =
```

```
        isError &&
        (status === 400 || // Invalid payload / Validation failure
          status === 403 || // Tampered headers / Signature issues / Forbidden
          status === 422 || // Schema violations
          status === 429 || // Rate-limit abuse
          (body.code &&
            (body.code === "VALIDATION_ERROR" ||
              body.code === "MALFORMED_REQUEST" ||
              body.code === "RATE_LIMIT_EXCEEDED" ||
              body.code === "AUTH_FORBIDDEN" ||
              body.code === "BAD_REQUEST")));

      // 3. Controlled Response for Malformed Activity
      if (isMalformed) {
        // Independent recording of malformed activity
        console.warn(
          `[SECURITY] Malformed activity detected. IP: ${req.ip}, Status: ${status}, URL: ${req.originalUrl}, Code: ${body.code || "N/A"}`,
        );

        // Ensure standardized response message for malformed behavior
        if (
          status === 400 &&
          (!body.message || body.message === "Validation failed")
        ) {
          body.message =
            "Request structure or behavior violates system constraints.";
        }
      }
    }

    return originalJson.call(this, body);
  };

  next();
};
```

2.27 Component: Uniz Portal

apps/uniz-portal/src/api/apiClient.ts

```
import { toast } from "@/utils/toast-ref";
import { ANALYTICS_KEY } from "./endpoints";

export enum ErrorCode {
  AUTH_UNAUTHORIZED = "AUTH_UNAUTHORIZED",
  AUTH_FORBIDDEN = "AUTH_FORBIDDEN",
  AUTH_TOKEN_EXPIRED = "AUTH_TOKEN_EXPIRED",
  AUTH_INVALID_CREDENTIALS = "AUTH_INVALID_CREDENTIALS",
  VALIDATION_ERROR = "VALIDATION_ERROR",
  RESOURCE_NOT_FOUND = "RESOURCE_NOT_FOUND",
  RESOURCE_ALREADY_EXISTS = "RESOURCE_ALREADY_EXISTS",
  INTERNAL_SERVER_ERROR = "INTERNAL_SERVER_ERROR",
  DATABASE_ERROR = "DATABASE_ERROR",
  OUTPASS_EXPIRED = "OUTPASS_EXPIRED",
  OUTPASS_ALREADY_APPROVED = "OUTPASS_ALREADY_APPROVED",
  RATE_LIMIT_EXCEEDED = "RATE_LIMIT_EXCEEDED",
  AUTH_SUSPENDED = "AUTH_SUSPENDED",
  AUTH_CAPTCHA_FAILED = "AUTH_CAPTCHA_FAILED",
}

interface ApiResponse<T = any> {
  success: boolean;
  data?: T;
  message?: string;
  code?: string;
  errors?: any[];
}

export interface ApiOptions extends RequestInit {
  params?: Record<string, any>;
```

```
31 }
32
33 export async function apiClient<T = any>(
34   endpoint: string,
35   options: ApiOptions = {},
36   showToast = true,
37 ): Promise<T | null> {
38   const token =
39     localStorage.getItem("admin_token") ||
40     localStorage.getItem("faculty_token") ||
41     localStorage.getItem("student_token");
42   const cleanToken = token ? token.replace(/^"|"$/g, "") : null;
43
44   const isFormData = options.body instanceof FormData;
45
46   const defaultHeaders: Record<string, string> = {
47     ...(cleanToken ? { Authorization: `Bearer ${cleanToken}` } : {}),
48     ...(ANALYTICS_KEY ? { "x-api-key": ANALYTICS_KEY } : {}),
49   };
50
51   if (!isFormData) {
52     defaultHeaders["Content-Type"] = "application/json";
53   }
54
55   // Handle Query Parameters
56   let url = endpoint;
57   if (options.params) {
58     const searchParams = new URLSearchParams();
59     Object.entries(options.params).forEach(([key, value]) => {
60       if (value !== undefined && value !== null) {
61         searchParams.append(key, String(value));
62       }
63     });
64     const queryString = searchParams.toString();
65     if (queryString) {
66       url += (url.includes("?") ? "&" : "?") + queryString;
67     }
68   }
69
70   try {
71     const response = await fetch(url, {
72       ...options,
73       headers: {
74         ...defaultHeaders,
75         ...options.headers,
76       },
77     });
78
79     const contentType = response.headers.get("content-type");
80     if (!contentType || !contentType.includes("application/json")) {
81       handleHttpError(response.status, { message: "Unexpected response format (Expected JSON)"
      }, showToast);
82       return null;
83     }
84
85     const data: ApiResponse<T> = await response.json();
86
87     if (!response.ok) {
88       handleHttpError(response.status, data, showToast, url);
89       return null;
90     }
91
92     return (data.data || data) as T;
93   } catch (error: any) {
94     if (showToast) {
95       toast.error(
96         error.message ||
97           "Network connection error. Please check your internet.",
98       );
99     }
100    return null;
101  }
102 }
```

```

export async function downloadFile(
  endpoint: string,
  fileName: string,
  params?: Record<string, any>,
): Promise<void> {
  const token =
    localStorage.getItem("admin_token") ||
    localStorage.getItem("faculty_token") ||
    localStorage.getItem("student_token");
  const cleanToken = token ? token.replace(/^"|"$/g, "") : null;

  let url = endpoint;
  if (params) {
    const searchParams = new URLSearchParams();
    Object.entries(params).forEach(([key, value]) => {
      if (value !== undefined && value !== null) {
        searchParams.append(key, String(value));

// ... [Truncated to optimize compile time] ...
```

apps/uniz-portal/src/api/endpoints.ts

```
// Set this to your Azure VM IP or Domain in .env as VITE_API_URL
export const BASE_URL = import.meta.env.VITE_API_URL || "/api/v1";
export const ANALYTICS_BASE_URL = import.meta.env.VITE_ANALYTICS_URL || "https://college-
    analytics.vercel.app";

// Notification service URL - routes through same-origin nginx proxy
export const NOTIFICATION_SERVICE_URL = `${BASE_URL}/notifications`;

// New Microservices Architecture Endpoints
// Auth
export const SIGNIN = (type: "student" | "admin" | "faculty") =>
  type === "admin" ? `${BASE_URL}/auth/login/admin` : `${BASE_URL}/auth/login`;
export const SIGNUP = (_type: "student" | "admin" | "faculty") =>
  `${BASE_URL}/auth/signup`;
export const FORGOT_PASS_ENDPOINT = `${BASE_URL}/auth/otp/request`;
export const REQUEST_OTP_EMAIL_ENDPOINT = `${BASE_URL}/auth/otp/request-email`;
export const VERIFY_OTP_ENDPOINT = `${BASE_URL}/auth/otp/verify`;
export const SET_NEW_PASS_ENDPOINT = `${BASE_URL}/auth/password/reset`;
export const CHANGE_PASS_ENDPOINT = `${BASE_URL}/auth/password/change`;

// Profile (User Service)
export const STUDENT_INFO = `${BASE_URL}/profile/student/me`;
export const UPDATE_DETAILS = `${BASE_URL}/profile/student/update`;
export const FACULTY_INFO = `${BASE_URL}/profile/faculty/me`;
export const ADMIN_INFO = `${BASE_URL}/profile/admin/me`;
export const SEARCH_STUDENTS = `${BASE_URL}/profile/student/search`;
export const GET_AVAILABLE_BATCHES = `${BASE_URL}/profile/admin/batches`;

// Outpass (Requests Service)
export const REQUEST_OUTING = `${BASE_URL}/requests/outing`;
export const REQUEST_OUTPASS = `${BASE_URL}/requests/outpass`;
export const STUDENT_HISTORY = `${BASE_URL}/requests/history`;
export const ADMIN_STUDENT_HISTORY = (id: string) =>
  `${BASE_URL}/requests/history/${id}`;

// Admin / Approvals (Through Requests Service)
export const APPROVE_OUTING = (id: string) =>
  `${BASE_URL}/requests/${id}/approve`;
export const REJECT_OUTING = (id: string) =>
  `${BASE_URL}/requests/${id}/reject`;
export const FORWARD_OUTING = (id: string) =>
  `${BASE_URL}/requests/${id}/forward`;
export const APPROVE_OUTPASS = (id: string) =>
  `${BASE_URL}/requests/${id}/approve`;
export const REJECT_OUTPASS = (id: string) =>
  `${BASE_URL}/requests/${id}/reject`;
export const FORWARD_OUTPASS = (id: string) =>
  `${BASE_URL}/requests/${id}/forward`;
```

```

// Bulk fetch
export const GET_OUTING_REQUESTS = `${BASE_URL}/requests/outing/all`;
export const GET_OUTPASS_REQUESTS = `${BASE_URL}/requests/outpass/all`;
export const SECURITY_SUMMARY = `${BASE_URL}/requests/security/summary`;
export const SECURITY_CHECKOUT = (id: string) =>
  `${BASE_URL}/requests/${id}/checkout`;
export const SECURITY_CHECKIN = (id: string) =>
  `${BASE_URL}/requests/${id}/checkin`;

// Notes:
// The frontend expects many endpoints that were present in the monolith.
// The new Microservices (Phase 3) implemented the CORE flow: Login, Profile, Request, History, Approve.
// Some admin bulk fetch endpoints are not yet in uniz-outpass-service (Phase 3 constraint was "Core Business Logic").
// This file is updated to point to the AVAILABLE microservices endpoints.
// Endpoints that are missing in MS are marked or mapped to stubs/comments.

// Legacy / Placeholder Endpoints for Build Compatibility
export const ADMIN_RESET_PASS = `${BASE_URL}/auth/admin/reset-password`;
export const ADMIN_GLOBAL_RESET_PASS = `${BASE_URL}/auth/admin/global-reset-password`;
export const RESET_PASS = `${BASE_URL}/auth/reset-password`;

// Academic / Grades (Pending Microservice Migration)
export const GET_ATTENDANCE = `${BASE_URL}/academics/attendance`;
export const GET_GRADES = `${BASE_URL}/academics/grades`;
export const DOWNLOAD_GRADES = (semesterId: string) =>
  `${BASE_URL}/academics/grades/download/${semesterId}`;
export const DOWNLOAD_ATTENDANCE = (semesterId: string) =>
  `${BASE_URL}/academics/attendance/download/${semesterId}`;

// Additional Legacy Endpoints
export const UPDATE_STUDENT_STATUS = `${BASE_URL}/profile/student/status`;
export const STUDENT_OUTSIDE_CAMPUS = `${BASE_URL}/requests/outside`;
export const CREATE_FACULTY = `${BASE_URL}/profile/faculty/create`;
export const SEARCH_FACULTY = `${BASE_URL}/profile/faculty/search`;
export const UPDATE_FACULTY = (username: string) =>
  `${BASE_URL}/profile/admin/faculty/${username}`;
export const ADMIN_SUSPEND_FACULTY = (username: string) =>
  `${BASE_URL}/profile/admin/faculty/${username}/suspend`;
export const BULK_CREATE_FACULTY = `${BASE_URL}/profile/admin/faculty/bulk-create`;
export const BULK_UPDATE_FACULTY = `${BASE_URL}/profile/admin/faculty/bulk-update`;
export const BULK_DELETE_FACULTY = `${BASE_URL}/profile/admin/faculty/bulk-delete`;

// Webmaster / Specialized Admin Endpoints
export const ADMIN_VIEW_STUDENT = (id: string) =>
  `${BASE_URL}/profile/admin/student/${id}`;
export const GET_SUBJECTS = `${BASE_URL}/academics/subjects`;
export const ADD_SUBJECT = `${BASE_URL}/academics/subjects/add`;
export const UPLOAD_ATTENDANCE = `${BASE_URL}/academics/attendance/upload`;
export const UPLOAD_GRADES = `${BASE_URL}/academics/grades/upload`;
export const UPLOAD_IMAGE = `${BASE_URL}/files/image/upload`;

export const GET_ATTENDANCE_TEMPLATE = (
  branch: string,
  year: string,
  semesterId: string,
  batch?: string,
) =>
  `${BASE_URL}/academics/attendance/template?branch=${branch}&year=${year}&semesterId=${semesterId}${batch ? `&batch=${batch}` : ""}`;

export const GET_GRADES_TEMPLATE = (
  branch: string,
  year: string,
  semesterId: string,
  subjectCode: string,
  remedialsOnly: boolean,
  batch?: string,
) =>
  `${BASE_URL}/academics/grades/template?branch=${branch}&year=${year}&semesterId=${semesterId}&subjectCode=${subjectCode}&remedialsOnly=${remedialsOnly}${batch ? `&batch=${batch}` : "
```

```
    "}`;

export const ADMIN_SUSPEND_STUDENT = (id: string) =>
  `${BASE_URL}/profile/admin/student/${id}/suspend`;
export const ADMIN_UPDATE_STUDENT = (id: string) =>

// ... [Truncated to optimize compile time] ...
```